UNIVERSITÀ DEGLI STUDI DELL'INSUBRIA

DIPARTIMENTO DI SCIENZA E ALTA TECNOLOGIA

CORSO DI LAUREA MAGISTRALE IN
MATEMATICA

Fast Fourier Transform in zk-STARK Protocols: Interpolation, Encoding and Proximity Proofs.

Supervisors:
Prof. Marco Donatelli
Prof. Alberto Trombetta

Student:
Paolo De Marinis
Matricola 759708

Academic Year:
2024/2025

To my family.

Contents

Chapter 1

Introduction

1.1 Context

In its classic sense, a mathematical *proof* is a sequence of logical deductions that a Verifier can re-execute to be convinced of the validity of a statement. In the same way, the traditional *proof* of the integrity of a computation is its re-execution.

This approach is infeasible when there is a strong asymmetry in computational capability, for example when:

- the computation is prohibitively expensive for a Verifier that lacks the resources (time, energy, hardware) to re-execute it;
- the Verifier intentionally operates with a limited computational capacity.

This led to the need to decouple verification from execution, introducing a new type of probabilistic verification that allows to check the integrity of a computation with a cost that is at most poly-logarithmic compared to the cost of re-execution.

The existence of this type of proofs, called *Probabilistically Checkable Proofs (PCPs)*, was proven in 1992 by Arora et al. [AS92] and this breakthrough started a whole new research line in Cryptography focused on the development of efficient proof systems whose principles resemble closely the ones of Numerical Analysis.

Both disciplines are focused on answering the same question:

> "How to guarantee the efficiency and reliability of a computational process in an imperfect world?"

While Numerical Analysis studies the convergence of a method, that is its theoretical ability to provide a valid approximation, and the efficiency of the algorithms to *execute* the computation, the field of Proof Systems studies the completeness, that is the theoretical ability of an honest Prover to provide a valid proof, and the efficiency of the protocols used to *verify* the computation.

Both disciplines are also founded on an analysis of robustness under non-ideal conditions that is the study of stability against numerical error on one hand and the analysis of soundness against a dishonest Prover on the other.

This parallelism also extends to their historical catalysts. Just as Numerical Analysis, despite being an ancient discipline, was accelerated by the advent of computers thanks to the interest and capitals generated by its practical application, the study of Proof Systems is now undergoing a similar acceleration driven by the advent of blockchains.

In particular, the birth of Ethereum, the first worldwide decentralized computer, has made the scenario of asymmetry in computational capability a reality, due to the

high costs of executing computations on-chain and the subsequent need to offload those computations off-chain to untrusted parties in a verifiable manner.

In this context, the *STARK (Scalable Transparent ARguments of Knowledge)* protocols emerged as one of the most promising proof systems.

Their name describes their distinguishing features:

- **Scalable** because the computational cost for the Prover is quasi-linear ($O(T \log T)$) and the one for the Verifier is poly-logarithmic ($O(\log^2 T)$);
- **Transparent** because they don't require a *trusted setup*, unlike the more widespread SNARK protocols;
- **ARguments** because their security is based on the assumption that the Prover that generates the proof is unable to break the standard cryptographic assumptions;
- **of Knowledge** because the protocol ensures the property of *knowledge soundness*.

In addition to those properties STARKs are an example of *post-quantum* cryptography because they rely only on collision resistant hash functions, that are conjectured to be *post-quantum secure*.

Those protocols are referred by the acronym *zk-STARK* when in addition to the properties already listed they have a property known as *zero-knowledge*, that is that the interactions between the Prover and the Verifier reveal nothing about the solution that the Prover possesses.

The choice to focus on this specific type of proof protocol is motivated not only by the popularity of zk-STARKs in the blockchain space but also by their unique architecture, which is heavily based on polynomial interpolation and on the *Fast Fourier Transform (FFT)*.

1.2 Motivations and Contributions

The central theme in the thesis is computational efficiency, zk-STARK protocols require the manipulation of very high-degree polynomials and that would be infeasible without the use of FFTs. In particular, the process of translating the execution trace into its algebraic equivalent occurs via polynomial interpolation and all the Reed Solomon encoding operations are by definition evaluations.

The FFT adoption is not a simple implementation detail, but the whole proof system is built around it and its influence extends to the field chosen for the computation and conceptually shapes the FRI proximity test that is inspired by the decimation phase of a Radix-2 FFT.

Therefore, this thesis was initially conceived as a work of analysis of a proof system whose architecture is shaped by the needs and conceptual structure of the FFT.

However, while studying this connection, I noticed that the available literature on the subject has several problems that make the zk-STARK protocol extremely difficult to grasp:

- **Didactic Gap**: being a very new technology, the available resources present a stark dichotomy. On one hand, there are introductory guides that are example based or focused on implementation and lack formal proofs. On the other hand, there are the academic papers and their summaries, which are written for experts.

  The difficulty with the academic literature is twofold:

  - the foundational papers, where the basic principles are proven, are dense and written before more modern and simplifying techniques and abstractions were discovered;
  - contemporary papers build upon this foundation and focus on their novel results using them to directly prove the most difficult cases.

  There's a lack of a modern source that explains the fundamentals from the ground up using the clearer concepts available today.

- **Inconsistent Formalism:** because the field is evolving rapidly, there is a lack of standardized definitions, in particular the *Algebraic Intermediate Representation (AIR)*, that is a central point in the STARK protocol, is defined in different variants across papers, making it difficult to compare and evaluate the different approaches. Furthermore, the literature lacks a complete theoretical derivation that starts from the first principles of an FSM and arrives to the definition of an AIR.

- **Lack of an Optimization Framework:** over time, different optimization techniques have been developed to make zk-STARKs efficient. However, each optimization is presented or as part of a new protocol or as an entirely new AIR model. Although these optimizations are all based on the same underlying principles, the literature lacks a single and unified theoretical framework that allows for systematic reasoning about the different types of optimizations and for their safe and modular composition.

This thesis aims to bridge the identified gaps through a theoretical reconstruction from first principles. During this process the following contributions were produced:

- **Formal FSM-based AIR Derivation:** Chapter 2 presents a complete derivation of the Algebraic Intermediate Representation (AIR) starting from the formal model of a Finite State Machine (FSM). It first defines the *Computational Integrity Statement (CI statement)* associated to the computation and then derives an associated AIR. This process culminates in a proof of the logical equivalence between the satisfiability of the AIR and the validity of the CI statement.

  This derivation formalizes and expands the ideas presented in a more intuitive way in the work of Alan Szepieniec *Anatomy of a STARK* [Sze]. Unlike constraint-focused approaches, our derivation emphasizes polynomials as the fundamental objects of the proof system.

- **General AIR definition:** Chapter 2 introduces a general AIR definition that is designed to unify different approaches found in literature. This definition recovers the explicit use of permutations from the original STARK paper [BBHR18b] and combines it with the flexibility of distinct evaluation domains for each constraint, a technique used in *ethSTARK* [Tea21]. The resulting definition is expressive enough to serve as a unifying model for the main AIR variants.

- **Optimization Framework:** Chapter 2 ends proposing a distinction between the vertical approach of remodeling and the horizontal approach of optimizing an existent AIR, providing a theoretical framework composed of two preorder relations that allows to reason on the nature of optimizations and to compose them in a secure way using the transitivity property.

In other words while research has focused on *how to optimize* an AIR introducing specific techniques, this part focuses on the question of *what it means to optimize* an existing AIR, providing the theoretical framework to reason about it.

- **Analysis and Reinterpretation of Optimizations:** starting from Chapter 3, optimization techniques are systematically analyzed using the introduced framework. It is first proven that the trace padding is a valid extension and then Chapter 4 analyzes the optimizations necessary to build a zk-VM, including *permutation arguments* and *preprocessing* described respectively in *eSTARK* [AMGM24] and in *Study of Arithmetization Methods for STARKs* [MF23].

  The new view offered by this thesis reveals that *permutation arguments* aren't simply a replacement of permutation constraint polynomials with local ones, but rather an elegant reduction of an arbitrary permutation to a shift.

- **Constraint Unrolling:** also in Chapter 4, an optimization for reducing the degree of constraints is formalized. This method combines the unrolling philosophy behind *Rank 1 Constraint Systems (R1CS)* [Tha22, p. 130] to the use of intermediate variables present in STARKs, that is presented only through examples in *eSTARK* [AMGM24] and *ethSTARK*[Tea21], defining low-degree constrainable operations.

  Furthermore, a re-aggregation of the resulting constraint polynomials is then formalized. This is done via a random linear combination to enable the use of the preprocessing technique.

- **zk-VM:** Chapter 4 ends showing how to compose the analyzed optimizations to derive an AIR for a *Zero-Knowledge Virtual Machine (ZK-VM)* based on an *Instruction Set Architecture (ISA)*, obtaining what is known as *Randomized AIR with Preprocessing (RAP)* [Gab].

  While not providing a specific instruction set, the resulting AIR can be used to build general-purpose proof systems.

- **Analysis of the Proof Protocol:** Chapter 5 provides a derivation of the entire cryptographic protocol, from the construction of the quotient polynomial to the proximity test. It follows the list of protocol steps presented in *eSTARK* [AMGM24] expanding upon them with proofs and explanations.

  In particular, an accessible analysis of the efficiency and soundness of FRI is presented, deriving the soundness analysis from the *Correlated Agreement Theorem* in the unique decoding regime.

  This approach differs from the literature for three reasons:

  – the foundational proof found in the original FRI paper [BBHR17] for the unique decoding radius regime is technically dense because it could not rely on this more modern result;

  – the paper that introduced the Correlated Agreement Theorem [BCI+23] and the recent summary on FRI [Hab22] directly treat the soundness for the more advanced batched FRI version in the list decoding regime;

  – all other resources omit a formal proof for the FRI soundness, given its notorious complexity.

1.3 Methods and Limits

This thesis aims to be as self-contained as possible, with the objective to be an introductory material to zk-STARKs. Due to the scope of this work, some more advanced topics such as the *circle FFT*, the *Batched FRI* and the soundness analysis of the FRI protocol in the *list decoding regime* are omitted.

For the same reason the *zero-knowledge* property is not formally proven and then we will refer to the protocol as a STARK, omitting the "zk" prefix.

We will analyze the interactive version of the protocol known in literature as *STIK (Scalable Transparent IOP of Knowledge)* which can be made non-interactive using the Fiat-Shamir heuristic [AMGM24, p. 36]. We will refer to the interactive protocol with the acronym STARK using the same abuse of notation of the original paper [BBHR19, p. 10].

Finally, the security analysis of a proof system can be done at different levels

- **Soundness:** It protects the Verifier by ensuring that an invalid witness is detected.

$$\text{Dishonest Prover} \implies \text{Verifier rejects with high probability}$$

- **Knowledge Soundness:** It guarantees that the Prover knows a valid witness.

$$\text{Verifier accepts} \implies \text{Prover knows a valid witness}$$

Proving knowledge soundness traditionally requires a constructive proof with the explicit construction of a *knowledge extractor* [Tea21, p. 32] that is an algorithm that can extract the secret witness by interacting with the Prover.

However, a result in the literature for Polynomial IOPs [CBBZ22, p. 13] (full version of [CBBZ23]), the family to which STARKs belong, shows that, for this class of protocols, soundness and knowledge soundness are equivalent. Given this result and the fact that it is also used in [AMGM24, p. 7], we will only prove standard soundness.

Chapter 2

Arithmetization

In this chapter, we describe how a deterministic and sequential computation, initially modeled as a Finite-state Machine (FSM), can be translated into an algebraic representation called *Algebraic Intermediate Representation (AIR)*.

This representation is derived using two components:

- an *execution trace*, algebraically represented by polynomial interpolation;
- a set of *polynomial constraints*, that enforce the correctness of the trace with respect to the modeled computation.

2.1 Modeling the Computation: Finite-State Machines

To establish the computational model that we are going to algebraically represent, we begin by introducing the general definition of finite-state machine and the associated computational integrity statement.

2.1.1 Finite-State Machines (FSM)

We first need the definitions of state, deterministic state transition function and sequence of states.

Definition 2.1.1 (State)
Let $S \neq \emptyset$ be a finite set called state space. *A* state *is an element $s \in S$.*

Definition 2.1.2 (Deterministic State Transition Function)
A deterministic state transition function *is a function*

$$\delta : S \times \Sigma \to S,$$

where $\Sigma \neq \emptyset$ is a finite set called input alphabet.

Remark 2.1.3 (Deterministic Nature)
Because δ is a function, for each pair $(s, a) \in S \times \Sigma$ a unique state $\delta(s, a) \in S$ is defined, that is

$$\forall (s, a), (t, b) \in S \times \Sigma, \quad (s, a) = (t, b) \implies \delta(s, a) = \delta(t, b).$$

The adjective "deterministic" is used to distinguish this function from the non-deterministic ones [Sip12, p. 53]

$$\delta : S \times \Sigma \to \mathcal{P}(S),$$

in which each state can have multiple successor states.

Definition 2.1.4 (Deterministic State Sequence)
Let $\delta : S \times \Sigma \to S$ be a deterministic state transition function and let $s_0 \in S$ be an initial state. Given an input sequence

$$a : \{0, 1, \dots, T-1\} \to \Sigma,$$

the corresponding deterministic state sequence *is the recursively defined sequence*

$$s : \{0, 1, \dots, T\} \to S,$$

defined as

$$s(t) := \begin{cases} s_0 & \text{if } t = 0 \\ \delta(s(t-1), a(t-1)) & \text{if } t \in \{1, \dots, T\} \end{cases}$$

To ensure that Definition 2.1.4 is well-defined, we need to verify that the sequence $s(t)$ exists and is uniquely determined by the initial state and the input sequence. This property is analogous to the existence and uniqueness of the orbit for an initial state in a discrete dynamical system [Com15]. For the sake of completeness, we provide a brief proof.

Proposition 2.1.5 (State Sequence Existence and Uniqueness)
Given a deterministic state transition function $\delta : S \times \Sigma \to S$, an initial state $s_0 \in S$ and an input sequence

$$a : \{0, 1, \dots, T-1\} \to \Sigma,$$

the sequence $s : \{0, 1, \dots, T\} \to S$ recursively defined as in Definition 2.1.4 exists and is unique.

Proof.

- **Existence:** We proceed by induction on t.
  - *Base Case ($t = 0$):* By hypothesis, $s(0) = s_0$ is defined.
  - *Inductive Step:* Assume that, for a certain $t \in \{0, \dots, T-1\}$, $s(t)$ is defined. Because δ is a function, given the input $a(t) \in \Sigma$, the pair $(s(t), a(t))$ uniquely determines
    $$s(t+1) = \delta(s(t), a(t)) \in S.$$
    Then the sequence is well-defined for $t + 1$.

  By induction, there exists a sequence $\{s(t)\}_{t=0}^{T}$ defined for each $t \in \{0, \dots, T\}$.

- **Uniqueness:** Assume, by contradiction, that two sequences $\{s(t)\}_{t=0}^{T}$ and $\{s'(t)\}_{t=0}^{T}$ exist and that they satisfy the recursive definition with the same initial state s_0 and the same input sequence $a(t)$. Consider the set
  $$I = \{t \in \{0, 1, \dots, T\} : s(t) = s'(t)\}.$$
  Because $s(0) = s'(0) = s_0$, we get $0 \in I$, so I is non-empty.

  Assume that $I \neq \{0, 1, \dots, T\}$ and let $t_0 \in \{1, \dots, T\}$ be the minimum index such that $t_0 \notin I$, that is $s(t_0) \neq s'(t_0)$. By the minimality of t_0, we have $t_0 - 1 \in I$, that is $s(t_0 - 1) = s'(t_0 - 1)$.

  Applying the state transition function with input $a(t_0 - 1)$, we get
  $$s(t_0) = \delta(s(t_0 - 1), a(t_0 - 1)) = \delta(s'(t_0 - 1), a(t_0 - 1)) = s'(t_0).$$
  and that is a contradiction because by hypothesis we have $s(t_0) \neq s'(t_0)$.

  We conclude that $I = \{0, 1, \dots, T\}$ and so the sequence is unique.

□

We can now state the definition of FSM as seen in [Sip12, p. 35]

Definition 2.1.6 (Deterministic Finite-state Machine)
A deterministic finite-state machine *(FSM) is defined as a* 5*-tuple*

$$(\Sigma, S, s_0, \delta, F),$$

where:

- Σ *is the input alphabet;*
- S *is the state space;*
- $s_0 \in S$ *is the initial state;*
- $\delta : S \times \Sigma \to S$ *is the deterministic transition function;*
- $F \subseteq S$ *is the set of final states.*

2.1.2 Algebraic FSM and Computational Integrity Statements

The arithmetization process for the STARK protocol happens in finite fields [Tea21, pp. 7-8] e [Sze].

Let $\mathbb{F}_q$ be a finite field of order q, where q is a power of a prime number, and let $w_{state}, w_{input} \in \mathbb{N}$ be such that $w_{state} \geq 1$ and $w_{input} \geq 0$.

We set the space state as $S = \mathbb{F}_q^{w_{state}}$ and the input alphabet as $\Sigma = \mathbb{F}_q^{w_{input}}$.

The state transition function and the input sequence are then

$$\delta : \mathbb{F}_q^{w_{state}} \times \mathbb{F}_q^{w_{input}} \to \mathbb{F}_q^{w_{state}}$$

and $a(t) \in \mathbb{F}_q^{w_{input}}$, together those determine the state sequence $s(t) \in \mathbb{F}_q^{w_{state}}$.

For the arithmetization, we now introduce the *execution trace* (or *discrete witness*)

$$x : \{0, \ldots, T\} \to \mathbb{F}_q^w$$

that combines state and input

$$x(t) = (s(t), a(t)), \quad \text{where } w = w_{state} + w_{input}.$$

This trace $x(t)$ is the object upon which we will define the algebraic constraints.

We now formally define the computational integrity statement, adapting the general definition found in [BBHR18b, p. 19] [BBHR19, p. 23] and [Sze] to our case.

Definition 2.1.7 (Computational Integrity Statement)
A computational integrity statement *is a tuple* $\mathcal{A} = (\delta, w_{state}, w_{input}, T, B)$*, where:*

- $\delta : \mathbb{F}_q^{w_{state}} \times \mathbb{F}_q^{w_{input}} \to \mathbb{F}_q^{w_{state}}$ *is the state transition function.*
- $w_{state} \geq 1$ *and* $w_{input} \geq 0$ *are the state and input width.*
- $T \geq 1$ *is the number of computation steps.*
- $B \subseteq \{1, \ldots, w\} \times \{0, \ldots, T\} \times \mathbb{F}_q$ *is the set of constraint specifications, where each tuple* $(k, t, v_{k,t}) \in B$ *specifies the condition* $x_k(t) = v_{k,t}$*. The set B must uniquely define the initial state* $s(0)$*.*

Remark 2.1.8 (Public Components)
The tuple $\mathcal{A} = (\delta, w_{state}, w_{input}, T, B)$ *defines the* public statement *of the problem, known by both the Prover and the Verifier.*

Remark 2.1.9 (Proof Scenarios)
The set B allows us to model different scenarios by choosing which parts of $x(t)$ to constrain:

- Proof of Execution*: only $s(0)$ is fixed. The inputs $a(t)$ for $t \in \{0, \dots, T-1\}$ and the states $s(t)$ for $t \in \{1, \dots, T\}$ remain private and are part of the witness.*
- Proof of Correct State Transition*: the initial state $s(0)$, the final state $s(T)$ and optionally the inputs $a(t)$ for $0 \leq t < T$ are fixed.*
- Proof of Knowledge*: is a Proof of Correct State Transition where at least one $a(t)$ is not fixed. This type of statement can be used to prove that the Prover knows an input sequence that brings the computation from a given initial state to a given final state.*
- Intermediate Conditions*: we can constrain specific states or inputs during execution imposing conditions on $x_k(t)$ for $0 < t < T$.*

This flexibility can be then used to adapt the CI statement to various requisites of public/private knowledge.

Following [Tea21, p. 10],[Sze] we now state when a CI statement is valid.

Definition 2.1.10 (Valid CI Statement)
The CI statement $\mathcal{A}$ is valid *if there is a trace $x : \{0, \dots, T\} \to \mathbb{F}_q^w$ such that:*

1. ***Transition Conditions:** For each $t \in \{0, \dots, T-1\}$, $x(t)$ and $x(t+1)$ satisfy the relation $s(t+1) = \delta(s(t), a(t))$, where $s(t)$ and $s(t+1)$ are the first w_{state} components and $a(t)$ are the remaining w_{input} components of $x(t)$.*
2. ***Point Conditions:** For each $(k, t, v_{k,t}) \in B$, the equality $x_k(t) = v_{k,t}$ holds.*

We denote by $\mathcal{T}_{\mathcal{A}}$ the set of valid execution traces for the CI statement $\mathcal{A}$.

Remark 2.1.11 (Terminology)
We use the general term point conditions*, however, as can be seen in [Sze], they are often referred as* boundary conditions *because these conditions are usually applied only on the initial and final states.*

Remark 2.1.12 (Convincing the Verifier)
In the STARK protocol the Prover, who knows the execution trace $x(t)$, needs to convince a Verifier of the validity of the CI statement $\mathcal{A}$ without revealing $x(t)$ itself. The inputs $a(t)$ for $t \geq 1$ can be part of the secret if not fixed by B.

The next step is translating the CI statement validity into a set of polynomial constraints.

2.2 The FSM-based Algebraic Intermediate Representation

2.2.1 Univariate Polynomial Interpolation

The first step in translating the trace $x(t)$ into an algebraic object is to represent each of its w components $x_k(t)$ using an univariate polynomial that interpolates it on a fixed evaluation domain.

We recall the basics of univariate polynomial interpolation [BBCM92, p. 352],[Cap16]

Definition 2.2.1 (Evaluation Domain)
Let $\mathbb{F}$ be a finite field and $T \geq 1$ an integer. An evaluation domain *of cardinality $T+1$ is an ordered subset $H = \{h_0, h_1, \dots, h_T\} \subset \mathbb{F}$ of $T+1$ distinct elements of $\mathbb{F}$.*
The elements $h_t \in H$ are called evaluation nodes*.*

Remark 2.2.2 (Choosing the Field and the Evaluation Domain)
For a CI statement defined on $\mathbb{F}_q$, the field $\mathbb{F}$ that is used for interpolation can be chosen as $\mathbb{F}_q$ itself or one of its extensions to ensure that $|\mathbb{F}| \geq T + 1$.

The evaluation domain H can be chosen as a cyclic subgroup of the multiplicative subgroup $\mathbb{F}^*$ but, to ensure that such a subgroup of order $T + 1$ exists, by the Lagrange Theorem [DF04, p. 89] for groups we have that its order must divide the order of $\mathbb{F}^*$, that is $T + 1 \mid |\mathbb{F}^*|$.

The choice of the evaluation domain is public.

Definition 2.2.3 (Lagrange Basis Polynomials)
Given an evaluation domain $H = \{h_0, h_1, \ldots, h_T\} \subset \mathbb{F}$, for each $t \in \{0, 1, \ldots, T\}$, the Lagrange basis polynomial *$\ell_t(X) \in \mathbb{F}[X]$ associated to the node h_t is the unique polynomial of degree $\deg \ell = T$ such that*

$$\ell_t(h_s) = \delta_{s,t} = \begin{cases} 1 & \text{if } s = t \\ 0 & \text{if } s \neq t \end{cases} \quad (\textit{Kronecker delta})$$

for each $s \in \{0, 1, \ldots, T\}$. Its explicit form is

$$\ell_t(X) = \prod_{\substack{j=0 \\ j \neq t}}^{T} \frac{X - h_j}{h_t - h_j}.$$

Theorem 2.2.4 (Lagrange Univariate Interpolation)
Given an evaluation domain $H = \{h_0, \ldots, h_T\} \subset \mathbb{F}$ and $T + 1$ values $y_0, \ldots, y_T \in \mathbb{F}$. There exists a unique polynomial $Q(X) \in \mathbb{F}[X]$ with degree $\deg Q \leq T$ such that

$$Q(h_t) = y_t \quad \text{for each } t \in \{0, \ldots, T\}.$$

This polynomial can be written in the form

$$Q(X) = \sum_{t=0}^{T} y_t \ell_t(X),$$

where $\ell_t(X)$ are the Lagrange basis polynomials.

Proof. We prove existence and uniqueness.

- **Existence:** Given the polynomial $Q(X) = \sum_{t=0}^{T} y_t \ell_t(X)$ we have $\deg Q \leq T$ because $Q(X)$ is a linear combination of the polynomials $\ell_t(X)$ that have degree $\deg \ell_t = T$.

  Evaluating $Q(X)$ on a node $h_s \in H$ we get

  $$Q(h_s) = \sum_{t=0}^{T} y_t \ell_t(h_s) = \sum_{t=0}^{T} y_t \delta_{t,s} = y_s,$$

  using the basic property of the Lagrange basis polynomials. Hence the polynomial $Q(X)$ satisfies the required conditions.

- **Uniqueness:** Suppose that there exist two polynomials $Q_1(X)$ and $Q_2(X)$, each with degree $\deg Q_1, \deg Q_2 \leq T$, such that

  $$Q_1(h_t) = Q_2(h_t) = y_t \quad \text{for each } t \in \{0, \ldots, T\}.$$

Defining the difference between the two polynomials as

$$R(X) = Q_1(X) - Q_2(X).$$

we get that $\deg R \leq T$, given that $\deg Q_1, \deg Q_2 \leq T$.

Then we can write $R(X)$ as

$$R(X) = a_0 + a_1 X + \cdots + a_T X^T,$$

with coefficients $a_i \in \mathbb{F}$. For each $t \in \{0, \ldots, T\}$, we have

$$R(h_t) = Q_1(h_t) - Q_2(h_t) = y_t - y_t = 0.$$

Substituting $R(X)$, we get a linear homogeneous system of equations in $a_0, \ldots, a_T$,

$$a_0 + a_1 h_t + a_2 h_t^2 + \cdots + a_T h_t^T = 0 \quad \text{for each } t \in \{0, \ldots, T\}.$$

This can be written in matrix form as $V\mathbf{a} = \mathbf{0}$, where $\mathbf{a} = (a_0, a_1, \ldots, a_T)^{\mathrm{T}}$ is the vector of coefficients and V is the Vandermonde matrix associated to the nodes $h_0, \ldots, h_T$

$$V = \begin{pmatrix} 1 & h_0 & h_0^2 & \cdots & h_0^T \\ 1 & h_1 & h_1^2 & \cdots & h_1^T \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & h_T & h_T^2 & \cdots & h_T^T \end{pmatrix}.$$

The determinant of V is

$$\det(V) = \prod_{0 \leq i < j \leq T} (h_j - h_i)$$

and $h_0, \ldots, h_T$ are distinct by the definition of evaluation domain H, so we get $h_j - h_i \neq 0$ for $i \neq j$. Therefore we have $\det(V) \neq 0$ and so the matrix V is invertible.

The unique solution of the linear homogeneous system $V\mathbf{a} = \mathbf{0}$ is the trivial solution $\mathbf{a} = \mathbf{0}$, meaning that $a_0 = a_1 = \cdots = a_T = 0$ and so $R(X)$ is the zero polynomial.

Therefore $Q_1(X) = Q_2(X)$ which proves the uniqueness of $Q(X)$.

□

Remark 2.2.5 (Computational Time Complexity)
Given $T+1$ points (h_t, y_t), computing the polynomial $Q(X)$, using for example its Newton form and computing the divided differences, needs $O((T+1)^2)$ arithmetic operations in the field $\mathbb{F}$ (see for example [BBCM92, p. 371] or [Cap16, p. 10]). This justifies the introduction of fast transforms as suggested in [BBHR18b, p.17] that we will discuss in detail in Chapter 3.

2.2.2 Trace Polynomials

We now use Theorem 2.2.4 to interpolate each of the w components of the execution trace $x(t)$ and define the trace polynomials as in [Sze], [Tea21, p. 8], [RIS] and [MF23, p. 3].

Definition 2.2.6 (Trace Polynomials)
Let $x : \{0, \ldots, T\} \to \mathbb{F}_q^w$ be the execution trace $x(t) = (x_1(t), \ldots, x_w(t))$ and let $H = \{h_0, \ldots, h_T\} \subset \mathbb{F}$ be an evaluation domain. For each $j \in \{1, \ldots, w\}$, the trace polynomial *$W_j(X) \in \mathbb{F}[X]$ is the unique polynomial of degree $\deg W_j \leq T$ such that*

$$W_j(h_t) = x_j(t) \quad \textit{for each } t \in \{0, \ldots, T\}.$$

We will refer to H as the trace domain *and to the vector $\mathbf{W}(X) = (W_1(X), \ldots, W_w(X)) \in \mathbb{F}[X]^w$ as the* polynomial witness.

Remark 2.2.7 (From Discrete to Polynomial Witness)
If the execution trace $x(t)$ is the discrete witness of a valid CI statement $\mathcal{A}$, the vector $\mathbf{W}(X)$ is its algebraic representation.

The Prover, who knows the entire execution trace $x(t)$, can compute the polynomial witness $\mathbf{W}(X)$ and has to keep it secret from the Verifier.

Remark 2.2.8 (Trace Polynomials and Transition Function)
The transition condition $s(t+1) = \delta(x(t))$ implies the following pointwise relation for $\mathbf{W}(X)$

$$W_k(h_{t+1}) = \delta_k(\mathbf{W}(h_t)) \quad \text{for each } k \in \{1, \dots, w_{state}\} \text{ and } t \in \{0, \dots, T-1\}.$$

This identity must hold on the nodes contained in $H \setminus \{h_T\} = \{h_0, \dots, h_{T-1}\}$ and will be used to derive the polynomial constraints in Section 2.2.4.

The components $W_j(X)$ for $j > w_{state}$ are not directly constrained by δ.

2.2.3 Polynomial Canonical Representative of Functions between Finite Fields

To algebraically express the transition condition $s(t+1) = \delta(s(t), a(t))$, we need to represent the function $\delta : \mathbb{F}_q^{w_{state}} \times \mathbb{F}_q^{w_{input}} \to \mathbb{F}_q^{w_{state}}$ as a polynomial.

Lagrange interpolation can be extended to represent δ, using [LN97, pp. 368-369] for existence and [Alo99, pp. 2-3] for uniqueness we prove the following Theorem.

Theorem 2.2.9 (Polynomial Representative of Functions $\mathbb{F}_q^w \to \mathbb{F}_q$)
Let $\mathbb{F}_q$ be a finite field. For each function $f : \mathbb{F}_q^w \to \mathbb{F}_q$ there exists a unique polynomial

$$Q(X_1, \dots, X_w) \in \mathbb{F}_q[X_1, \dots, X_w]$$

such that:

1. *$Q(a_1, \dots, a_w) = f(a_1, \dots, a_w)$ for each $(a_1, \dots, a_w) \in \mathbb{F}_q^w$;*
2. *the degree of Q in each variable X_k, denoted by $\deg_{X_k}(Q)$, is at most $q-1$.*

This polynomial is the polynomial representative *of f on $\mathbb{F}_q^w$.*

Proof. We prove existence and uniqueness.

- **Existence:** For each $a \in \mathbb{F}_q$, define $\ell_a(X) \in \mathbb{F}_q[X]$ as the degree $q-1$ univariate Lagrange basis polynomial that satisfies $\ell_a(b) = \delta_{a,b}$ for each $b \in \mathbb{F}_q$.

  Given a vector $\mathbf{a} = (a_1, \dots, a_w) \in \mathbb{F}_q^w$, we build the following multivariate basis polynomial

  $$\ell_{\mathbf{a}}(X_1, \dots, X_w) = \prod_{k=1}^{w} \ell_{a_k}(X_k).$$

  This polynomial has degree $\deg_{X_k} \ell_{\mathbf{a}} = q-1$ for each k and we have

  $$\ell_{\mathbf{a}}(\mathbf{b}) = \delta_{\mathbf{a},\mathbf{b}} = \begin{cases} 1 & \text{if } \mathbf{a} = \mathbf{b} \\ 0 & \text{if } \mathbf{a} \neq \mathbf{b} \end{cases}$$

  where $\delta_{\mathbf{a},\mathbf{b}}$ is the Kronecker delta between vectors. We can now write a candidate Q as

  $$Q(X_1, \dots, X_w) = \sum_{\mathbf{a} \in \mathbb{F}_q^w} f(\mathbf{a})\, \ell_{\mathbf{a}}(X_1, \dots, X_w).$$

and evaluating it in $\mathbf{b} \in \mathbb{F}_q^w$ we get

$$Q(\mathbf{b}) = \sum_{\mathbf{a} \in \mathbb{F}_q^w} f(\mathbf{a})\, \ell_{\mathbf{a}}(\mathbf{b}) = f(\mathbf{b}) \cdot \ell_{\mathbf{b}}(\mathbf{b}) = f(\mathbf{b}).$$

At last, the candidate Q has degree $\deg_{X_k} Q \leq q-1$ in each variable because $\deg_{X_k} \ell_{\mathbf{a}} = q-1$ and so all the requirements are met.

- **Uniqueness:** Suppose that there exist two polynomials $Q_1, Q_2 \in \mathbb{F}_q[X_1, \ldots, X_w]$, both with degree at most $q-1$ in each variable, such that $Q_1(\mathbf{a}) = Q_2(\mathbf{a}) = f(\mathbf{a})$ for each $\mathbf{a} \in \mathbb{F}_q^w$.

  Let $R = Q_1 - Q_2$, then

  - $R(\mathbf{a}) = 0$ for each $\mathbf{a} \in \mathbb{F}_q^w$,
  - $\deg_{X_k} R \leq q-1$ for each k.

  We need to prove that $R \equiv 0$. We proceed by induction on w:

  - *Base Case ($w = 1$):* $R(X_1)$ is an univariate polynomial with degree $\deg R \leq q-1$ that vanishes on each of the q elements of $\mathbb{F}_q$. Each non-zero polynomial with degree at most $q-1$ has at most $q-1$ roots, so we must have $R \equiv 0$.
  - *Inductive Step:* Suppose that the conclusion holds for $w-1$ and write

    $$R(X_1, \ldots, X_w) = \sum_{k=0}^{\deg_{X_w} R} R_k(X_1, \ldots, X_{w-1}) X_w^k,$$

    where $R_k \in \mathbb{F}_q[X_1, \ldots, X_{w-1}]$ and $\deg_{X_i}(R_k) \leq q-1$ for each $i \in \{1, \ldots, w-1\}$.
    For each fixed $\mathbf{a}' = (a_1, \ldots, a_{w-1}) \in \mathbb{F}_q^{w-1}$ we define

    $$R_{\mathbf{a}'}(X_w) := R(a_1, \ldots, a_{w-1}, X_w) = \sum_{k=0}^{\deg_{X_w} R} R_k(\mathbf{a}') X_w^k.$$

    Given that by hypotesis $R(\mathbf{a}', a_w) = 0$ for each $a_w \in \mathbb{F}_q$, we note that $R_{\mathbf{a}'}(X_w)$ is a polynomial of degree $\deg R_{\mathbf{a}'} \leq q-1$ with q roots, so it follows that $R_{\mathbf{a}'}(X_w) \equiv 0$.
    This implies $R_k(\mathbf{a}') = 0$ for each $k \in \{0, \ldots, \deg_{X_w} R\}$.
    Because the choice of $\mathbf{a}'$ is arbitrary we get $R_k(\mathbf{a}') = 0$ for each $\mathbf{a}' \in \mathbb{F}_q^{w-1}$.
    By the inductive hypothesis, it follows that $R_k \equiv 0$ for each k, therefore

    $$R(X_1, \ldots, X_w) = \sum_{k=0}^{\deg_{X_w} R} 0 \cdot X_w^k \equiv 0.$$

    We proved that $R \equiv 0$ and this implies $Q_1 \equiv Q_2$.

□

Corollary 2.2.10 (Polynomial Representative of Functions $\mathbb{F}_q^w \to \mathbb{F}_q^z$)
For each function $f : \mathbb{F}_q^w \to \mathbb{F}_q^z$ there exists a unique vector of z polynomials $\mathbf{F} = (F_1, \ldots, F_z)$, where each component $F_j \in \mathbb{F}_q[X_1, \ldots, X_w]$ is the polynomial representation of the component $f_j : \mathbb{F}_q^w \to \mathbb{F}_q$ and has $\deg_{X_k} F_j \leq q-1$ for each j, k.

Proof. The conclusion immediately follows by applying Theorem 2.2.9 to each component

$$f_j : \mathbb{F}_q^w \to \mathbb{F}_q,$$

defined as $(a_1, \ldots, a_w) \mapsto (f(a_1, \ldots, a_w))_j$. □

Remark 2.2.11 (Equivalence Classes)
Multiple polynomials in $\mathbb{F}_q[X_1, \ldots, X_w]$ can represent the same function f on $\mathbb{F}_q^w$.

By the Lagrange Theorem for groups, we get $x^q = x$ for each $x \in \mathbb{F}_q$ [LN97, p. 48], and that implies that these polynomials form an equivalence class modulo the ideal

$$I = \langle X_1^q - X_1, \ldots, X_w^q - X_w \rangle.$$

Theorem 2.2.9 ensures that each equivalence class contains a unique polynomial representative $Q(X_1, \ldots, X_w)$ of $\deg_{X_k} Q \leq q - 1$ for each k.

We now recall the definition of total degree for a multivariate polynomial [LN97, pp. 28-29]

Definition 2.2.12 (Multivariate Polynomial Total Degree)
Let $P \in \mathbb{F}_q[X_1, \ldots, X_w]$ be a polynomial $P = \sum_{\mathbf{a}=(a_1,\ldots,a_w)} c_{\mathbf{a}} X_1^{a_1} \cdots X_w^{a_w}$ with $c_{\mathbf{a}} \in \mathbb{F}_q$.

The total degree *of P, denoted by* $\deg P$*, is the maximum degree between its non-zero monomials*

$$\deg P = \max\{a_1 + \cdots + a_w \mid c_{(a_1,\ldots,a_w)} \neq 0\}.$$

If $P \equiv 0$, one sets $\deg P = -\infty$.

For practical purposes we introduce the following definition of degree of a vector of multivariate polynomials.

Definition 2.2.13 (Vector of Multivariate Polynomials Degree)
Let $\mathbf{P} = (P_1, \ldots, P_z)$ be a vector of multivariate polynomials $P_j \in \mathbb{F}_q[X_1, \ldots, X_w]$. The degree *of* $\mathbf{P}$*, denoted by* $\deg \mathbf{P}$*, is the maximum total degree of its components*

$$\deg \mathbf{P} = \max_{1 \leq j \leq z} \deg P_j.$$

By extension, the degree of a tuple of vectors is the degree of the vector obtained concatenating the elements of the tuple.

Remark 2.2.14 (State Transition Function Maximum Degree)
Given the state transition function $\delta : \mathbb{F}_q^w \to \mathbb{F}_q^{w_{state}}$ and its polynomial representative

$$\mathbf{D} = (D_1, \ldots, D_{w_{state}}),$$

we know that $\deg_{X_j}(D_k) \leq q - 1$ for each $k \in \{1, \ldots, w_{state}\}$ and $j \in \{1, \ldots, w\}$.

The total degree of D_k is then upper bounded by $\sum_{j=1}^{w}(q-1) = w(q-1)$ so it follows that $\deg \mathbf{D}$ *satisfies*

$$\deg \mathbf{D} = \max_{1 \leq k \leq w_{state}} \deg D_k \leq w(q-1).$$

Remark 2.2.15 (Polynomial Representative of δ)
Although Theorem 2.2.9 ensures the polynomial representative existence and uniqueness for each $\delta : \mathbb{F}_q^w \to \mathbb{F}_q^{w_{state}}$, calculating the representative through interpolation is computationally infeasible.

This is because any such algorithm would need to operate on every point of the domain, that has cardinality $|\mathbb{F}_q^w| = q^w$ giving a minimum time complexity of $\Omega(q^w)$, which is astronomically large for the fields typically used in STARKs, as can be seen in the following table.

Field Name	Cardinality q	Bit Length	Implementation
Baby Bear	$2^{31} - 2^{27} + 1$	31	RISC Zero [BGt23] Plonky3 [The22]
KoalaBear	$2^{31} - 2^{24} + 1$	31	Plonky3 [The22]
Rescue	$2^{61} + 20 \cdot 2^{32} + 1$	62	ethSTARK [Tea21, p. 5]
f62	$2^{62} - 111 \cdot 2^{39} + 1$	62	Winterfell [Fac21]
Goldilocks	$2^{64} - 2^{32} + 1$	64	Plonky3 [The22] Winterfell [Fac21]
f128	$2^{128} - 45 \cdot 2^{40} + 1$	128	Winterfell [Fac21]
Rescue-Prime	$407 \cdot 2^{119} + 1$	128	Anatomy of a STARK [Sze]
STARK	$2^{251} + 17 \cdot 2^{192} + 1$	252	Starknet [Sta]

Table 2.1: Finite fields used across STARK implementations.

Even for low values of w we will still get astronomically large values for q^w making the interpolation infeasible. Therefore, we assume that the state transition function δ, defined in the CI statement $\mathcal{A}$, is already in its canonical representative form $\mathbf{D} = (D_1, \ldots, D_{w_{state}})$, where each $D_k \in \mathbb{F}_q[X_1, \ldots, X_w]$ has degree $\deg_{X_j} D_k \leq q - 1$. In other words, we consider $\mathbf{D}$ public.

When the logic of the transition function δ is not too complex, we can use the properties of $\mathbb{F}_q$ to derive the canonical representative analitically.

Example 2.2.16 (Analytic Derivation of the Canonical Representative)
Let $\delta_k : \mathbb{F}_q^3 \to \mathbb{F}_q$ be a state transition function component that implements the following conditional logic

$$\delta_k(x_1, x_2, x_3) = \begin{cases} x_1 + x_2 & \text{if } x_3 \neq 0, \\ x_1 - x_2 & \text{if } x_3 = 0. \end{cases}$$

The canonical polynomial representative $D_k(X_1, X_2, X_3)$ can be derived using the fact that, as a consequence of the Lagrange theorem for groups, we have

$$X_3^{q-1} = \begin{cases} 1 & \text{if } X_3 \neq 0 \\ 0 & \text{if } X_3 = 0 \end{cases} .$$

So we have

$$D_k(X_1, X_2, X_3) = (X_1 + X_2)X_3^{q-1} + (X_1 - X_2)(1 - X_3^{q-1}).$$

that is

$$D_k(X_1, X_2, X_3) = X_1 - X_2 + 2X_2X_3^{q-1}.$$

This polynomial is indeed the canonical representative because it has degrees $\deg_{X_1} D_k = \deg_{X_2} D_k = 1$ and $\deg_{X_3} D_k = q - 1$ that are all at most $q - 1$.

2.2.4 Transition and Point Constraint Polynomials

Following [Sze] and the general principles found in [Tea21, p. 10] and [MF23] we now formalize the constrains that the polynomial witness $\mathbf{W}(X)$ must satisfy, making use of the state transition function representative $\mathbf{D}$ and the point conditions B.

Transition Constraints

Definition 2.2.17 (Transition Constraint Polynomial)
Let $\mathbf{D} = (D_1, \ldots, D_{w_{state}})$ *be the canonical polynomial representative of the state transition function* δ*. The set of* transition constraint polynomials *associated to* δ *is* $\mathcal{P}_{\mathbf{D}} = \{P_1, \ldots, P_{w_{state}}\}$*, where each* $P_k \in \mathbb{F}[X_1, \ldots, X_w, Y_1, \ldots, Y_w]$ *is a polynomial in* $2w$ *variables defined by*

$$P_k(X_1, \ldots, X_w, Y_1, \ldots, Y_w) := Y_k - D_k(X_1, \ldots, X_w) \quad \text{for } k \in \{1, \ldots, w_{state}\}.$$

Those w_{state} *polynomials algebraically express the condition* $s_k(t+1) = \delta_k(x(t))$ *linking the next state to the previous execution trace row.*

Remark 2.2.18 (Transition Constraint Polynomials are Public)
Given the fact that the polynomials P_k *are directly derived from* $\mathbf{D}$ *which is public, the set* $\mathcal{P}_{\mathbf{D}}$ *is also public.*

Definition 2.2.19 (Shift Polynomial)
Given an evaluation domain $H = \{h_0, h_1, \ldots, h_T\} \subset \mathbb{F}$*, we call* shift polynomial *the unique polynomial* $\sigma(X) \in \mathbb{F}[X]$ *with* $\deg \sigma \leq T$ *such that*

$$\sigma(h_t) = h_{t+1} \quad \text{for each } t \in \{0, \ldots, T-1\}.$$

Remark 2.2.20 (Public Shift Polynomial)
The shift polynomial $\sigma(X)$ *is uniquely determined by interpolation over the evaluation domain* H *so it uniquely depends on the structure of* H*. If, for example,* H *is a cyclic subgroup*

$$H = \{\omega^0, \omega^1, \ldots, \omega^T\}$$

with $\omega^{T+1} = 1$*, then we have* $\sigma(X) = \omega X$*.*

The evaluation domain H *is public and this implies that* $\sigma(X)$ *is also public.*

Definition 2.2.21 (Transition Constraint)
Let $\mathbf{W}(X)$ *be a polynomial witness over the evaluation domain* H*. Given the shift polynomial* $\sigma(X)$ *associated with* H *and the transition constraint polynomials* $\mathcal{P}_{\mathbf{D}} = \{P_1, \ldots, P_{w_{state}}\}$ *we say that* $\mathbf{W}(X)$ *satisfies the k-th* transition constraint *if*

$$P_k(\mathbf{W}(h_t), \mathbf{W}(\sigma(h_t))) = 0 \quad \text{for each } h_t \in H \setminus \{h_T\}.$$

Definition 2.2.22 (Transition Check Polynomial)
Let $\mathbf{W}(X)$ *be the polynomial witness over the evaluation domain* H*. Given the shift polynomial* $\sigma(X)$ *associated with* H *and the transition constraint polynomials* $\mathcal{P}_{\mathbf{D}} = \{P_1, \ldots, P_{w_{state}}\}$ *we define the k-th* transition check polynomial *as the univariate polynomial*

$$Check_k(X) := P_k(\mathbf{W}(X), \mathbf{W}(\sigma(X))) = W_k(\sigma(X)) - D_k(\mathbf{W}(X)) \in \mathbb{F}[X]$$

Remark 2.2.23 (Check Polynomials Role and Properties)
The w_{state} *transition check polynomials are computed by the Prover using his private polynomial witness* $\mathbf{W}(X)$ *and the public components* $\sigma(X)$ *and* $\mathcal{P}_{\mathbf{D}}$*, so the polynomials* $Check_k(X)$ *are private.*

If $\mathbf{W}(X)$ *is the polynomial witness of a valid execution trace* $x(t)$*, the transition check polynomials must vanish over* $H \setminus \{h_T\}$*. Hence, the Prover's objective is to convince the Verifier that all the transition check polynomials vanish over* $H \setminus \{h_T\}$ *without revealing them.*

Remark 2.2.24 (Transition Check Polynomials Degree)
Being that $\deg(W_j) \leq T$ *for each* j*, if we assume that* H *is cyclic, we get*

$$\deg(Check_k(X)) = \deg(W_k(\sigma(X)) - D_k(\mathbf{W}(X))) \leq \max(T, \deg D_k \cdot T)$$

where we used the fact that $\deg \sigma = 1$.

If $\deg D_k > 1$ *we have*

$$\deg Check_k(X) \leq \deg D_k \cdot T \leq w(q-1)T.$$

Point Constraints

In the same way, we now introduce the point constraint polynomials derived from the point conditions B given in the CI statement $\mathcal{A} = (\delta, w_{state}, w_{input}, T, B)$.

Definition 2.2.25 (Point constraint Polynomial)
Let $(k, t, v_{k,t}) \in B$ *be a constraint specification. We define the* point constraint polynomial *associated to* $(k, t, v_{k,t})$ *as the polynomial* $B_{k,t} \in \mathbb{F}[X_1, \ldots, X_w]$ *defined as*

$$B_{k,t}(X_1, \ldots, X_w) := X_k - v_{k,t}.$$

Those polynomials algebraically express the condition $X_k = v_{k,t}$.

Remark 2.2.26 (Point Constraint Polynomials are Public)
Given that B *is public, we have that the set* $\mathcal{B} = \{B_{k,t}(\mathbf{X}) \mid (k, t, v_{k,t}) \in B\}$ *of all the point constraint polynomials derived from* B *is also public.*

Definition 2.2.27 (Point Constraints)
Let $\mathbf{W}(X)$ *be a polynomial witness over the evaluation domain* H*. Given the point constraint polynomials* $\mathcal{B} = \{B_{k,t}(\mathbf{X}) \mid (k, t, v_{k,t}) \in B\}$ *we say that* $\mathbf{W}(X)$ *satisfies the* point constraints *if*

$$B_{k,t}(\mathbf{W}(h_t)) = 0 \text{ for each } B_{k,t} \in \mathcal{B}.$$

Definition 2.2.28 (Point Check Polynomial)
Let $\mathbf{W}(X)$ *be a polynomial witness over the evaluation domain* H *and let*

$$\mathcal{B} = \{B_{k,t}(\mathbf{X}) \mid (k, t, v_{k,t}) \in B\}$$

be the set of point constraint polynomials.

For each $(k, t, v_{k,t}) \in B$ *we define the associated* point check polynomial *as the univariate polynomial*

$$Check^{\mathcal{B}}_{k,t}(X) := B_{k,t}(\mathbf{W}(X)) = W_k(X) - v_{k,t}.$$

Remark 2.2.29 (Point Polynomials Properties)
Unlike transition constraints, which must hold over the entire transition domain $H \setminus \{h_T\}$*, each point constraint is a condition that applies to a single node* h_t.

The Prover must convince the Verifier that, for each specification $(k, t, v_{k,t}) \in B$*, the polynomial* $W_k(X) - v_{k,t}$ *vanishes at* h_t.

Being $\mathbf{W}(X)$ *private, the polynomials* $Check^{\mathcal{B}}_{k,t}(X)$ *are also private.*

2.2.5 Algebraic Intermediate Representation (AIR) associated to $\mathcal{A}$

An Algebraic Intermediate Representation (AIR) [MF23, BBHR18b, Tea21] associated to a CI statement $\mathcal{A}$ translates $\mathcal{A}$ into an algebraic structure, by defining the context and the rules that a valid witness must satisfy.

Definition 2.2.30 (Algebraic Intermediate Representation (AIR) associated to $\mathcal{A}$)
Let $\mathcal{A} = (\delta, w_{state}, w_{input}, T, B)$ be a CI statement.

An Algebraic Intermediate Representation (AIR) *associated to $\mathcal{A}$ is a tuple*

$$\mathcal{AIR}(\mathcal{A}) = (\mathbb{F}, T, w_{state}, w_{input}, H, \sigma(X), \mathcal{P}_{\mathbf{D}}, \mathcal{B})$$

where:

- $\mathbb{F}$ *is a finite field chosen for the arithmetization, such that $\mathbb{F} = \mathbb{F}_q$ or one of its extensions and satisties $|\mathbb{F}| \geq T + 1$ (as in Remark 2.2.2).*
- T *is the number of computation steps (from $\mathcal{A}$).*
- w_{state} *is the state width (from $\mathcal{A}$).*
- w_{input} *is the input width (from $\mathcal{A}$).*
- $H = \{h_0, \ldots, h_T\} \subset \mathbb{F}$ *is the trace evaluation domain (Definition 2.2.1).*
- $\sigma(X) \in \mathbb{F}[X]$ *is the shift polynomial for H (Definition 2.2.19).*
- $\mathcal{P}_{\mathbf{D}} = \{P_1, \ldots, P_{w_{state}}\}$ *is the set of w_{state} transition constraint polynomials (Definition 2.2.17), derived from the canonical representative $\mathbf{D}$ of δ specified in $\mathcal{A}$.*
- $\mathcal{B} = \{B_{k,t}(\mathbf{X}) \mid (k, t, v_{k,t}) \in B\}$ *is the set of point constraint polynomials (Definition 2.2.25), derived from the set B specified in $\mathcal{A}$.*

Remark 2.2.31 (AIR as Algebraic Public Specification)
The entire tuple $\mathcal{AIR}(\mathcal{A})$ is public and is the algebraic specification that the private polynomial witness $\mathbf{W}(X)$, if it exists, must satisfy.

Now we state the AIR satisfiability following [Tea21, p. 26]

Definition 2.2.32 (AIR Satisfiability)
An AIR $\mathcal{AIR}(\mathcal{A}) = (\mathbb{F}, T, w_{state}, w_{input}, H, \sigma(X), \mathcal{P}_{\mathbf{D}}, \mathcal{B})$ is said to be satisfiable *if there exists a vector of polynomials $\mathbf{W}(X) = (W_1(X), \ldots, W_w(X)) \in \mathbb{F}[X]^w$, where $w = w_{state} + w_{input}$, such that the following conditions hold:*

1. ***Low Degree:*** $\deg(W_j) \leq T$ *for each $j \in \{1, \ldots, w\}$.*
2. ***Transition Constraints:*** *For each transition constraint polynomial $P_k \in \mathcal{P}_{\mathbf{D}}$ and for each $h_t \in H \setminus \{h_T\} = \{h_0, \ldots, h_{T-1}\}$, we have*
$$P_k(\mathbf{W}(h_t), \mathbf{W}(\sigma(h_t))) = 0.$$
3. ***Point Constraints:*** *For each $B_{k,t}(\mathbf{X}) \in \mathcal{B}$ associated to $(k, t, v_{k,t}) \in B$, we have*
$$B_{k,t}(\mathbf{W}(h_t)) = 0.$$

Remark 2.2.33 (AIR Satisfiability and Polynomial Witness)
As we already observed the polynomial witness $\mathbf{W}(X)$ is the algebraic representation of the execution trace $x(t)$ calculated by executing the computation described in $\mathcal{A}$.

The AIR satisfiability is then a statement about the existence and the properties of $\mathbf{W}(X)$ that the Prover must prove without revealing $\mathbf{W}(X)$ itself.

2.2.6 AIR Satisfiability and CI Statement Validity Equivalence

We now formally prove the equivalence between the validity of the CI statement $\mathcal{A}$ and the satisfiability of its algebraic representation $\mathcal{AIR}(\mathcal{A})$.

Proposition 2.2.34 (Equivalence between AIR Satisfiability and CI Statement Validity)
Let $\mathcal{A} = (\delta, w_{state}, w_{input}, T, B)$ be a CI statement and consider a corresponding AIR

$$\mathcal{AIR}(\mathcal{A}) = (\mathbb{F}, T, w_{state}, w_{input}, H, \sigma(X), \mathcal{P}_{\mathbf{D}}, \mathcal{B}),$$

then the CI statement $\mathcal{A}$ is valid if and only if $\mathcal{AIR}(\mathcal{A})$ is satisfiable.

Proof. We prove the two implications.

$\implies$ Assume that $\mathcal{A}$ is valid, with discrete witness $x(t)$. Let $\mathbf{W}(X)$ be the interpolant of $x(t)$ over H, we get:

1. **Low Degree:** $\deg(W_j) \leq T$ by construction.
2. **Transition Constraints:** We have $s(t+1) = \delta(s(t), a(t))$ by the validity of $x(t)$. Evaluating $P_k(\mathbf{X}, \mathbf{Y}) = Y_k - D_k(\mathbf{X})$ over $(\mathbf{W}(h_t), \mathbf{W}(h_{t+1}))$ we obtain
$$P_k(\mathbf{W}(h_t), \mathbf{W}(h_{t+1})) = W_k(h_{t+1}) - D_k(\mathbf{W}(h_t)) = s_k(t+1) - \delta_k(s(t), a(t)) = 0$$
for $k \in \{1, \ldots, w_{state}\}$ and $t \in \{0, \ldots, T-1\}$.
3. **Point Constraints:** We have $x_k(t) = v_{k,t}$ for each $(k, t, v_{k,t}) \in B$. Evaluating $B_{k,t}(\mathbf{X}) = X_k - v_{k,t}$ over $\mathbf{W}(h_t)$ we obtain
$$B_{k,t}(\mathbf{W}(h_t)) = W_k(h_t) - v_{k,t} = x_k(t) - v_{k,t} = 0.$$

Therefore $\mathbf{W}(X)$ satisfies $\mathcal{AIR}(\mathcal{A})$.

$\impliedby$ Assume that $\mathcal{AIR}(\mathcal{A})$ is satisfiable with polynomial witness $\mathbf{W}(X)$. Defining the candidate trace $x(t) = \mathbf{W}(h_t)$ we get:

1. **Transition Conditions:** By hypothesis we have $P_k(\mathbf{W}(h_t), \mathbf{W}(h_{t+1})) = 0$, explicitating the polynomial we get $W_k(h_{t+1}) - D_k(\mathbf{W}(h_t)) = 0$, that implies $s_k(t+1) = \delta_k(s(t), a(t))$ for each $k \in \{1, \ldots, w_{state}\}$. Therefore $s(t+1) = \delta(s(t), a(t))$.
2. **Point Conditions:** By hypothesis we have $B_{k,t}(\mathbf{W}(h_t)) = 0$, substituting we get $W_k(h_t) - v_{k,t} = 0$, that implies $x_k(t) = v_{k,t}$ for each specification in B.

Therefore the trace $x(t)$ is a valid witness for $\mathcal{A}$.

□

Remark 2.2.35 (Plurality of Solutions)
Proposition 2.2.34 states that the CI statement $\mathcal{A}$ is valid if and only if there exists at least one *polynomial witness $\mathbf{W}(X)$ that satisfies $\mathcal{AIR}(\mathcal{A})$. As we have seen, this equivalence is a direct consequence of the one-to-one correspondence between a trace $x(t)$ and its interpolant $\mathbf{W}(X)$ given by Theorem 2.2.4.*

Although each trace has a unique interpolant, the trace itself could be non-unique. If $\mathcal{A}$ is valid, there is at least one valid $x(t)$ but the uniqueness depends on how much B binds the computation. If, for example, the inputs $a(1), \ldots, a(T-1)$ are not fixed in B, multiple valid $x(t)$ can exist.

This means that we can have multiple distinct polynomial witnesses $\mathbf{W}(X)$ that satisfy $\mathcal{AIR}(\mathcal{A})$.

Given the fact that we can have multiple witnesses that satisfy $\mathcal{AIR}(\mathcal{A})$ we group them by defining the set of valid witnesses.

Definition 2.2.36 (Set of Valid Witnesses)
Given an AIR $\mathcal{AIR}(\mathcal{A})$ associated to the CI statement $\mathcal{A}$, we denote by $\mathcal{W}_{\mathcal{AIR}(\mathcal{A})}$ the set of witnesses that satisfy it

$$\mathcal{W}_{\mathcal{AIR}(\mathcal{A})} := \{\mathbf{W}(X) \in \mathbb{F}[X]^w \mid \mathbf{W}(X) \textit{ satisfies } \mathcal{AIR}(\mathcal{A})\}.$$

Remark 2.2.37 (Set of Valid Traces)
Evaluating over H we obtain again the set of valid execution traces

$$\begin{aligned}\mathcal{T}_{\mathcal{A}} &= \{x \in \mathbb{F}_q^{(T+1)\times w} \mid x \textit{ satisfies } \mathcal{A}\} \\ &= \{x \in \mathbb{F}_q^{(T+1)\times w} \mid x(t) = \mathbf{W}(h_t) \textit{ for } h_t \in H \textit{ and } \mathbf{W} \in \mathcal{W}_{\mathcal{AIR}(\mathcal{A})}\}\end{aligned}$$

that we will also denote by $\mathcal{T}_{\mathcal{AIR}(\mathcal{A})}$

Remark 2.2.38 (Prover's Objective)
So we can reformulate again the Provers's objective, the Prover must convince the Verifier that $\mathcal{W}_{\mathcal{AIR}(\mathcal{A})} \neq \emptyset$, without revealing which specific element of $\mathcal{W}_{\mathcal{AIR}(\mathcal{A})}$ he has.

2.3 Example: Arithmetizing the Fibonacci Sequence

We now consider a concrete example using the Fibonacci sequence to show how a CI statement $\mathcal{A}$ can be translated into an associated AIR $\mathcal{AIR}(\mathcal{A})$.

We consider the special case in which the original field $\mathbb{F}_q$ is not big enough to satisfy the condition $|\mathbb{F}_q| \geq T+1$ and therefore a field extension is needed for the arithmetization process.

Example 2.3.1 (Fibonacci Sequence)
We divide the process into multiple steps.

1. ***Describing the computation and CI statement:***

   The Fibonacci sequence is defined by the following recurrence relation

   $$F_t = \begin{cases} 0 & \text{for } t = 0 \\ 1 & \text{for } t = 1 \\ F_{t-1} + F_{t-2} & \text{for } t \geq 2 \end{cases}.$$

   Asssume that the computation takes place in the field $\mathbb{F}_q = \mathbb{F}_3$ and define the state at time t as $s(t) = (s_1(t), s_2(t)) \in \mathbb{F}_3^2$, where $s_1(t) = F_t$ and $s_2(t) = F_{t+1}$.

   The state width is then $w_{state} = 2$ and there is no input value so $w_{input} = 0$, this means that the execution trace has width $w = w_{state} + w_{input} = 2$.

   Therefore we get $x(t) = s(t)$ and the state transition function is

   $$\begin{aligned} \delta : &\mathbb{F}_3^2 \to \mathbb{F}_3^2 \\ &(x_1, x_2) \mapsto (x_2, x_1 + x_2) \end{aligned}.$$

   We choose to compute the sequence up to the element F_4. Being that the trace $x(t)$ contains F_t and F_{t+1}, to obtain F_4 we need to compute three computation steps, therefore we have $T = 3$.

   The set of point conditions B must define the initial state, so it contains $x(0)$.

   In particular, B contains $(1, 0, v_{1,0})$ and $(2, 0, v_{2,0})$, with $v_{1,0} = 0, v_{2,0} = 1 \in \mathbb{F}_3$.

   The CI statement, with δ and B defined over $\mathbb{F}_3$ and $T = 3$, is then

   $$\mathcal{A}_{Fib} = (\delta, w_{state} = 2, w_{input} = 0, T = 3, B = \{(1,0,0), (2,0,1)\}).$$

2. ***Choosing $\mathcal{AIR}(\mathcal{A}_{Fib})$:*** *Following Definition 2.2.30, we can define $\mathcal{AIR}(\mathcal{A}_{Fib})$ as it follows:*

   - ***Arithmetization Field $\mathbb{F}$:*** *The CI statement is defined over $\mathbb{F}_q = \mathbb{F}_3$ that contains only 3 elements. However, the field $\mathbb{F}$ chosen for arithmetization must be big enough to satisfy $|\mathbb{F}| \geq T + 1$.*

     In our case, $\mathbb{F}_3$ has not enough elements to be directly used and we must choose a field extension that has at least $T + 1 = 4$ elements.

     The smallest extension $\mathbb{F}_{3^n}$ that satisfies the condition is $\mathbb{F}_{3^2} = \mathbb{F}_9$, we then choose $\mathbb{F} = \mathbb{F}_9$.

     To represent the elements of $\mathbb{F}_9$, we follow [LN97, p. 67].

     We consider the polynomial $P(K) = K^2 + 1$, that is irreducible over $\mathbb{F}_3$ because it has no roots in $\mathbb{F}_3$ given that

- $P(0) = 0^2 + 1 = 1 \neq 0$,
- $P(1) = 1^2 + 1 = 2 \neq 0$,
- $P(2) = 2^2 + 1 = 5 \equiv 2 \pmod 3 \neq 0$.

Let α be a root of $P(K) = K^2 + 1$ in $\mathbb{F}_9$.
By definition, in $\mathbb{F}_9$ we have $\alpha^2 + 1 = 0$, that is $\alpha^2 = -1$.
Given that $-1 \in \mathbb{F}_3 \subset \mathbb{F}_9$ we have $-1 \equiv 2 \pmod 3$, so we can write $\alpha^2 = 2$ in $\mathbb{F}_9$.
Given that α is a root of an irreducible polynomial of degree $n = 2$ over $\mathbb{F}_3$, each element β in the extension $\mathbb{F}_{3^2} = \mathbb{F}_9$ can be uniquely represented as a polynomial in the variable α with degree lower than 2 (see [LN97, p. 52]), that is

$$\beta = a_0 + a_1\alpha \quad \text{with } a_0, a_1 \in \mathbb{F}_3.$$

This gives $3 \times 3 = 9$ distinct elements and so

$$\mathbb{F}_9 = \{0, 1, 2, \alpha, 1 + \alpha, 2 + \alpha, 2\alpha, 1 + 2\alpha, 2 + 2\alpha\}$$

- ***Trace Domain** $H \leq \mathbb{F}_9^*$: To obtain a cyclic subgroup of order $T + 1 = 4$ we can consider the one generated by $\alpha \in \mathbb{F}_9$.*
  This element has order 4 because $\alpha^0 = 1, \alpha^1 = \alpha, \alpha^2 = 2$ and $\alpha^3 = 2\alpha, \alpha^4 = 1$.
  Therefore we choose $H = \langle\alpha\rangle = \{\alpha^0, \alpha^1, \alpha^2, \alpha^3\} = \{1, \alpha, 2, 2\alpha\}$.
- ***Shift Polynomial** $\sigma(Z) \in \mathbb{F}_9[Z]$: Being H cyclic and generated by α, the shift polynomial is simply $\sigma(Z) = \alpha Z$.*
- ***Transition Constraint Polynomials** $\mathcal{P}_{\mathbf{D}} \subset \mathbb{F}_9[X_1, X_2, Y_1, Y_2]$:*
  The state transition function δ, that has coefficients in $\mathbb{F}_3 \subset \mathbb{F}_9$, is already the canonical representative because it is in polynomial form and with low enough degree.
  So we have

  $$\mathbf{D}(\mathbf{X}) = (D_1(X_1, X_2), D_2(X_1, X_2)) = (X_2, X_1 + X_2).$$

  Each polynomial component is in $\mathbb{F}_3[X_1, X_2]$ so they both are in $\mathbb{F}_9[X_1, X_2]$.
  The degree is $\deg(\mathbf{D}) = 1$.
  We finally get $\mathcal{P}_{\mathbf{D}} = \{P_1, P_2\}$ where

  $$P_1(\mathbf{X}, \mathbf{Y}) = Y_1 - X_2,$$

  $$P_2(\mathbf{X}, \mathbf{Y}) = Y_2 - (X_1 + X_2).$$

  As we already observed, the coefficients of those polynomials are in $\mathbb{F}_3$ but we can consider them as elements in $\mathbb{F}_9$.
- ***Point Constraint Polynomials** $\mathcal{B} \subset \mathbb{F}_9[X_1, X_2]$:*
  Given $B = \{(1, 0, 0), (2, 0, 1)\}$ with $0, 1 \in \mathbb{F}_3 \subset \mathbb{F}_9$ we get $\mathcal{B} = \{B_{1,0}, B_{2,0}\}$ where

  $$B_{1,0}(\mathbf{X}) = X_1,$$

  $$B_{2,0}(\mathbf{X}) = X_2 - 1.$$

  Again, the polynomials are in $\mathbb{F}_3[X_1, X_2]$ so they both are in $\mathbb{F}_9[X_1, X_2]$.

The complete AIR is then

$$\mathcal{AIR}(\mathcal{A}_{Fib}) = (\mathbb{F}_9, T = 3, w_{state} = 2, w_{input} = 0, H = \{1, \alpha, 2, 2\alpha\}, \sigma(Z) = \alpha Z, \mathcal{P}_{\mathbf{D}}, \mathcal{B}).$$

3. ***AIR Satisfiability***

   The AIR $\mathcal{AIR}(\mathcal{A}_{Fib})$ is satisfiable if there exists $\mathbf{W}(Z) = (W_1(Z), W_2(Z))$, with $W_j(Z) \in \mathbb{F}_9[Z]$ and $\deg(W_j) \leq T = 3$, such that the following hold

   (a) ***Transition Constraints:*** *For each $h_t \in H \setminus \{h_T\} = \{h_0, h_1, h_2\} = \{1, \alpha, 2\}$ we have*

   $$\begin{aligned} W_1(\alpha \cdot h_t) - W_2(h_t) &= 0 \\ W_2(\alpha \cdot h_t) - (W_1(h_t) + W_2(h_t)) &= 0 \end{aligned}$$

   (b) ***Point Constraints:*** *Given the node $h_0 = 1 \in H$:*

   $$\begin{aligned} W_1(h_0) - 0 &= 0 \\ W_2(h_0) - 1 &= 0 \end{aligned}$$

   The existence of such a witness is what the Prover must convince the Verifier of, without revealing it.

4. ***Execution Trace $x(t)$ over $\mathbb{F}_3$:***

   The Prover can now execute the computation starting from $F_0 = 0$ and $F_1 = 1$ in $\mathbb{F}_3$ and obtains

   - $x(0) = (F_0, F_1) = (0, 1)$
   - $x(1) = \delta(x(0)) = (F_1, F_0 + F_1) = (1, 0 + 1) = (1, 1)$
   - $x(2) = \delta(x(1)) = (F_2, F_1 + F_2) = (1, 1 + 1) = (1, 2)$
   - $x(3) = \delta(x(2)) = (F_3, F_2 + F_3) = (2, 1 + 2) = (2, 0)$.

   Execution traces are often represented as tables

Execution Trace of Fibonacci sequence over $\mathbb{F}_3$ (with $F_0 = 0, F_1 = 1, T = 3$)		
t	$x_1(t) = F_t \pmod 3$	$x_2(t) = F_{t+1} \pmod 3$
0	*0*	*1*
1	*1*	*1*
2	*1*	*2*
3	*2*	*0*

   but we will often represent them in a more compact way using matrices so we have $x \in \mathbb{F}_3^{4\times 2}$

   $$x = \begin{pmatrix} 0 & 1 \\ 1 & 1 \\ 1 & 2 \\ 2 & 0 \end{pmatrix}$$

   and use x_1 and x_2 to denote the columns associated to $x_1(t)$ and $x_2(t)$.

   If the execution trace $x(t)$ correctly computes the Fibonacci sequence over $\mathbb{F}_3$, then the polynomials $W_1(Z), W_2(Z) \in \mathbb{F}_9[Z]$ that interpolate $x_1 = (0, 1, 1, 2)$ and $x_2 = (1, 1, 2, 0)$ over the nodes $h_0, h_1, h_2, h_3 \in H$ will satisfy all the constraints.

5. ***Lagrange Basis Polynomials*** $\ell_t(Z)$***:***

Given the trace domain $H = \{h_0, h_1, h_2, h_3\} = \{1, \alpha, 2, 2\alpha\}$ *in* $\mathbb{F}_9$ *with* $\alpha^2 = 2$*, we recall that the Lagrange basis polynomials* $\ell_t(Z)$ *of degree* $T = 3$ *are given by*

$$\ell_t(Z) = \prod_{\substack{s=0 \\ s \neq t}}^{3} \frac{Z - h_s}{h_t - h_s}.$$

The Lagrange basis polynomial $\ell_0(Z)$ *is then*

$$\ell_0(Z) = \frac{(Z-\alpha)(Z-2)(Z-2\alpha)}{(1-\alpha)(1-2)(1-2\alpha)}.$$

We now compute the coefficient $C_0 = \frac{1}{(1-\alpha)(1-2)(1-2\alpha)}$*.*

We first find the inverses in $\mathbb{F}_9$ *of the factors in the denominator:*

- *We compute* $(1-\alpha)^{-1}$ *solving* $(1-\alpha)(x+y\alpha) = 1$*.*
  We get $(x-2y) + (y-x)\alpha = 1$ *that implies* $x = 2, y = 2$*. Therefore,* $(1-\alpha)^{-1} = 2 + 2\alpha$*.*
- $(1-2)^{-1} = (-1)^{-1} = 2^{-1} = 2$ *in* $\mathbb{F}_3$*.*
- *We compute* $(1-2\alpha)^{-1}$ *solving* $(1-2\alpha)(x+y\alpha) = 1$*, that implies* $x = 2, y = 1$*. Therefore,* $(1-2\alpha)^{-1} = 2 + \alpha$*.*

Multiplying the obtained factors we get

$$\begin{aligned} C_0 &= (2+2\alpha) \cdot 2 \cdot (2+\alpha) \\ &= (4+4\alpha)(2+\alpha) \equiv (1+\alpha)(2+\alpha) \pmod 3 \\ &= 2 + \alpha + 2\alpha + \alpha^2 = 2 + 3\alpha + 2 \quad (\textit{using } \alpha^2 = 2) \\ &= 4 \equiv 1 \pmod 3 \end{aligned}$$

Now we compute the product for the numerator $N_0(Z) = (Z-\alpha)(Z-2)(Z-2\alpha)$*, we have*

$$\begin{aligned} (Z-\alpha)(Z-2) &= Z^2 - 2Z - \alpha Z + 2\alpha \\ &= Z^2 - (2+\alpha)Z + 2\alpha \end{aligned}$$

and multiplying by $(Z - 2\alpha)$ *we get*

$$\begin{aligned} N_0(Z) &= (Z^2 - (2+\alpha)Z + 2\alpha)(Z - 2\alpha) \\ &= Z(Z^2 - (2+\alpha)Z + 2\alpha) - 2\alpha(Z^2 - (2+\alpha)Z + 2\alpha) \\ &= Z^3 - (2+\alpha)Z^2 + 2\alpha Z - 2\alpha Z^2 + 2\alpha(2+\alpha)Z - 4\alpha^2 \\ &= Z^3 - (2+\alpha+2\alpha)Z^2 + (2\alpha + 4\alpha + 2\alpha^2)Z - 4\alpha^2 \\ &= Z^3 - (2+3\alpha)Z^2 + (6\alpha + 2\alpha^2)Z - 4\alpha^2 \quad (\textit{using } \alpha^2 = 2) \\ &= Z^3 - 2Z^2 + (0\alpha + 2(2))Z - 4(2) \\ &= Z^3 - 2Z^2 + 4Z - 8 \\ &\equiv Z^3 + Z^2 + Z + 1 \pmod 3 \end{aligned}$$

We finally obtain $\ell_0(Z) = C_0 \cdot N_0(Z) = 1 \cdot (Z^3 + Z^2 + Z + 1) = Z^3 + Z^2 + Z + 1$*.*

In a similar way, on the other interpolation nodes we obtain the Lagrange basis polynomials

- $\ell_0(Z) = Z^3 + Z^2 + Z + 1$
- $\ell_1(Z) = \alpha Z^3 + 2Z^2 + 2\alpha Z + 1$
- $\ell_2(Z) = 2Z^3 + Z^2 + 2Z + 1$
- $\ell_3(Z) = 2\alpha Z^3 + 2Z^2 + \alpha Z + 1$

6. ***Trace Polynomials*** $W_1(Z)$ ***and*** $W_2(Z)$***:***

   Utilizing $x(t) = (x_1(t), x_2(t))$ and the Lagrange basis polynomials $\ell_t(Z)$, we can now find the trace polynomials $W_1(Z)$ and $W_2(Z)$.

   We compute the first one

   $$W_1(Z) = x_1(0)\ell_0(Z) + x_1(1)\ell_1(Z) + x_1(2)\ell_2(Z) + x_1(3)\ell_3(Z)$$

   using $x_1(t)$ we obtain

   $$W_1(Z) = 0 \cdot (Z^3 + Z^2 + Z + 1) + 1 \cdot (\alpha Z^3 + 2Z^2 + 2\alpha Z + 1) + 1 \cdot (2Z^3 + Z^2 + 2Z + 1) + \\ + 2 \cdot (2\alpha Z^3 + 2Z^2 + \alpha Z + 1)$$

   Simplifying the fourth term

   $$2(2\alpha Z^3 + 2Z^2 + \alpha Z + 1) = 4\alpha Z^3 + 4Z^2 + 2\alpha Z + 2 \equiv \alpha Z^3 + Z^2 + 2\alpha Z + 2 \pmod 3$$

   we get

   $$W_1(Z) = (\alpha Z^3 + 2Z^2 + 2\alpha Z + 1) + (2Z^3 + Z^2 + 2Z + 1) + (\alpha Z^3 + Z^2 + 2\alpha Z + 2)$$

   Finally grouping the coefficients

   $$W_1(Z) = (2 + 2\alpha)Z^3 + Z^2 + (2 + \alpha)Z + 1.$$

   In a similar way, we can obtain

   $$W_2(Z) = (2 + \alpha)Z^3 + 2Z^2 + (2 + 2\alpha)Z + 1$$

7. ***Check Polynomials:***

   - ***Point Check Polynomials:***
     Given the constraint polynomials in $\mathcal{B} = \{B_{1,0}, B_{2,0}\}$ the associated check polynomials are

     $$Check^B_{1,0}(Z) = W_1(Z) = (2 + 2\alpha)Z^3 + Z^2 + (2 + \alpha)Z + 1$$
     $$Check^B_{2,0}(Z) = W_2(Z) - 1 = (2 + \alpha)Z^3 + 2Z^2 + (2 + 2\alpha)Z$$

     that correctly vanish over $h_0 = 1$.
   - ***Transition Check Polynomials:***
     With $\mathbf{D}(\mathbf{X}) = (X_2, X_1 + X_2)$ and $\sigma(Z) = \alpha Z$, the check polynomials are

     $$Check_1(Z) = W_1(\alpha Z) - W_2(Z) = 0$$
     $$Check_2(Z) = W_2(\alpha Z) - (W_1(Z) + W_2(Z)) = \alpha Z^3 + Z^2 + 2\alpha Z + 2$$

     that correctly vanish over $H \setminus \{h_T\} = \{1, \alpha, 2\}$.

The Prover now has the check polynomials corresponding to the polynomial witness and will use those in the next phases of the STARK protocol.

Remark 2.3.2 (Choosing the Field)
In real world STARK scenarios, a prime field $\mathbb{F}_q$ with $q \gg T$ is directly chosen to model the computation to avoid the necessity for field extensions and this explains why in Table 2.2.15 we find only prime fields with high cardinality.

Another crucial aspect, that we already pointed out in Remark 2.2.5, is the necessity for fast transforms. This will further restrict the field choice to fields that support fast transforms, as we will explain in Chapter 3.

2.4 General AIR Definition and Optimizations

We defined $\mathcal{AIR}(\mathcal{A})$ in Definition 2.2.30 as the CI statement $\mathcal{A}$ algebraic representation. This representation directly follows from the FSM logic but it is not guaranteed to be the most efficient way to build a STARK proof.

For example, the transition constraint polynomials can have high degree or the computation lenght $T+1$ can be inadeguate to use fast transforms efficiently.

To address these problems, we introduce a more general AIR definition that more closely resembles the ones found in literature. The main idea is that multiple algebraic structures can describe the validity of the same CI statement.

2.4.1 Generalizing $\mathcal{AIR}(\mathcal{A})$

Instead of directly linking the form of the constraint polynomials to δ or to specific trace components, we treat them as generic polynomials that can describe the evolution between states that are not necessarily consecutive or algebraic properties that hold across a single state.

We will furthermore introduce specific domains for each constraint polynomial.

Definition 2.4.1 (Algebraic Intermediate Representation (AIR))
An AIR *is a tuple*

$$\mathcal{AIR} = (\mathbb{F}, T, w, H, \sigma(X), \mathcal{P}, \mathcal{P}_{\boldsymbol{\pi}}, \mathcal{B})$$

where:

- $\mathbb{F}$, T, w, H,$\sigma(X)$ *are respectively a finite field, the number of computation steps, the trace width, a trace evaluation domain with cardinality* $T+1$ *and the associated shift polynomial.*
- $\mathcal{P} = \{(P_1, H_{P_1}), \dots, (P_s, H_{P_s})\}$ *is a set of s local transition constraint specifications. Each specification is a pair where*
  - $P_j \in \mathbb{F}[\mathbf{X}, \mathbf{Y}]$ *is a local transition constraint polynomial, between two consecutive states with respect to the natural order of the computation given by* $\sigma(X)$.
  - $H_{P_j} \subseteq H$ *is the evaluation domain of the polynomial* P_j.
- $\mathcal{P}_{\boldsymbol{\pi}} = \{(P_{\pi_1}, H_{\pi_1}), \dots, (P_{\pi_r}, H_{\pi_r})\}$ *is an ordered set of r non-local transition constraint specifications. Each specification is a pair where*
  - $P_{\pi_k} \in \mathbb{F}[\mathbf{X}, \mathbf{Y}]$ *is a non-local transition constraint polynomial, between two consecutive states with respect to the order permuted according to a bijection of the nodes*

    $$\pi_k : H \to H.$$

  - $H_{P_{\pi_k}} \subseteq H$ *is the evaluation domain of the polynomial* P_{π_k}.
- $\mathcal{B} = \{(B_1, H_{B_1}\}), \dots, (B_m, H_{B_m})\}$ *is a set of m state constraint specifications. Each specification is a pair where*
  - $B_l \in \mathbb{F}[\mathbf{X}]$ *is a state constraint polynomial.*
  - $H_{B_l} \subseteq H$ *is the evaluation domain of the the polynomial* B_l.

An AIR $\mathcal{AIR}(\mathcal{A})$ associated to the CI statement $\mathcal{A}$ is a particular case of $\mathcal{AIR}$ in which:

- the set of non-local constraint specifications $\mathcal{P}_{\boldsymbol{\pi}}$ is empty;
- the set of local constraint specifications $\mathcal{P}_{\boldsymbol{\pi}}$ contains all specifications over the same evaluation domain $H \setminus \{h_T\}$ and only constraint polynomials of the form $Y_k - D_k(\mathbf{X})$;
- the set of state constraint specifications contain at most one point constraint for each element of the trace.

Remark 2.4.2 (Main Differences)
As already observed, unlike Definition 2.2.17 in this general AIR definition the transition constraint polynomials are not confined in the form $Y_k - D_k(\mathbf{X})$.

The most important difference between local and non-local constraints is given by the shift polynomial:

- *in $\mathcal{P}$ the next state is publicly defined by $\sigma(X)$;*
- *in $\mathcal{P}\boldsymbol{\pi}$ the next state is defined using a vector of permutation polynomials $\boldsymbol{\pi} = (\pi_1, \ldots, \pi_r)$. The tuple $\mathcal{AIR}$ does not specify $\boldsymbol{\pi}$ because its existence will be part of the witness.*

  The Prover, who can derive the correct order as a result of the computation, can compute $\boldsymbol{\pi}$ by interpolating the nodes in the permuted order over the original nodes.

In a similar way, a state constraint polynomial B_l is not anymore confined in the form $X_k - v_{k,t}$, but can constrain more general properties on the state at time t. Given that $\mathcal{B}$ is now a set of pairs, multiple state properties can now be enforced at the same time.

Remark 2.4.3 (In Literature)
The AIR *we defined in Definition 2.4.1 unifies two ideas that are already present in the STARK literature:*

- *the introduction of non-local constraint polynomials $\mathcal{P}_{\boldsymbol{\pi}}$ can be found in [BBHR18b, p. 38] as* Permuted Algebraic Intermediate Representation (PAIR)*;*
- *the use of different evaluation domains for each constraint polynomial can be found in [Tea21, p. 25].*

As we did in Definition 2.2.32 we now define the AIR satisfiability.

Definition 2.4.4 (AIR Satisfiability)
An AIR $\mathcal{AIR} = (\mathbb{F}, T, w, H, \sigma(X), \mathcal{P}, \mathcal{P}_{\boldsymbol{\pi}}, \mathcal{B})$ is satisfiable *if there exists a pair*

$$(\mathbf{W}(X), \boldsymbol{\pi}(X)) \in \mathbb{F}[X]^w \times \mathbb{F}[X]^r$$

of a polynomial witness and a vector of permutation polynomials, meaning that $\pi_k : H \to H$ is a bijection, such that the following hold

1. ***Low Degree:*** *For each $j \in \{1, \ldots, w\}$ we have $\deg(W_j) \leq T$ and for each $k \in \{1, \ldots, r\}$ we have $\deg(\pi_k) \leq T$, that by Definition 2.2.13 is $\deg(\mathbf{W}(X), \boldsymbol{\pi}(X)) \leq T$.*
2. ***Local Transition Constraints:*** *For each specification $(P_j, H_{P_j}) \in \mathcal{P}$ we have*

$$P_j(\mathbf{W}(h), \mathbf{W}(\sigma(h))) = 0 \quad \textit{for each } h \in H_{P_j}.$$

3. ***Non-Local Transition Constraints:*** *For each specification $(P_{\pi_k}, H_{P_{\pi_k}})$ that occupies the k-th place in the ordered set $\mathcal{P}_{\boldsymbol{\pi}}$, we have*

$$P_{\pi_k}(\mathbf{W}(h), \mathbf{W}(\pi_k(h))) = 0 \quad \textit{for each } h \in H_{\pi_k}.$$

4. ***State Constraints:*** *For each* $(B_l, H_{B_l}) \in \mathcal{B}$*, we have*

$$B_l(\mathbf{W}(h)) = 0 \quad \textit{for each } h \in H_{B_l}.$$

We will often refer to the whole $(\mathbf{W}(X), \boldsymbol{\pi}(X)) \in \mathbb{F}[X]^w \times \mathbb{F}[X]^r$ *simply as* polynomial witness.

As we did in Definition 2.2.36 we define the set of polynomial witnesses that satisfy $\mathcal{AIR}$ and its discrete analogous.

Definition 2.4.5 (Witness Sets)
Given an AIR $\mathcal{AIR}$*, we denote by* $\mathcal{W}_{\mathcal{AIR}}$ *the set of all its polyomial witnesses*

$$\mathcal{W}_{\mathcal{AIR}} := \{(\mathbf{W}(X), \boldsymbol{\pi}(X)) \in \mathbb{F}[X]^w \times \mathbb{F}[X]^r \mid (\mathbf{W}(X), \boldsymbol{\pi}(X)) \textit{ satisfy } \mathcal{AIR}\}.$$

Evaluating over the trace domain we get the discrete witness set

$$\mathcal{T}_{\mathcal{AIR}} := \{(x, p) \in \mathbb{F}^{(T+1)\times w} \times H^{(T+1)\times r} \mid (x(t), p(t)) = (\mathbf{W}(h_t), \boldsymbol{\pi}(h_t)) \textit{ for } h_t \in H \textit{ and } (\mathbf{W}, \boldsymbol{\pi}) \in \mathcal{W}_{\mathcal{AIR}}\}$$

At last, in a similar way as Section 2.2.4 we define the check polynomials

Definition 2.4.6 (Check Polynomials)
Given a polynomial witness $(\mathbf{W}(X), \boldsymbol{\pi}(X))$*, we define the following check polynomials:*

- *for each specification* $(P_j, H_{P_j}) \in \mathcal{P}$*, the* local check polynomial *is*

$$Check_{P_j}(X) := P_j(\mathbf{W}(X), \mathbf{W}(\sigma(X)))$$

- *for each specification* (P_{π_k}, H_{π_k}) *that is in the k-th place of* $\mathcal{P}_{\boldsymbol{\pi}}$*, the* non-local check polynomial *is*

$$Check_{P_{\pi_k}}(X) := P_{\pi_k}(\mathbf{W}(X), \mathbf{W}(\pi_k(X)))$$

- *for each specification* $(B_l, H_{B_l}) \in \mathcal{B}$*, the* state check polynomial *is*

$$Check_{B_l}(X) := B_l(\mathbf{W}(X))$$

2.4.2 AIR Optimization and Valid Extensions

Even if Definition 2.4.1 may seem too general, it gives the flexibility necessary to have optimized AIRs.

As we show in the following example, we can constrain a column to be a permutation of another one using the simple FSM-based AIR but this is highly inefficient, while the new AIR definition gives us a more direct and efficient way.

Example 2.4.7 (Optimizing Trough the Use of Non-Local Constraints)
We now analyze a simple case where one has to prove through a computation in the finite prime field $\mathbb{F}_q$ *that two recursively defined sequences*

$$s_i(t) = \begin{cases} v_{i,0} & \textit{if } t = 0 \\ \delta_i(s_i(t-1)) & \textit{if } t > 0 \end{cases}, \quad \textit{for } i = 1, 2$$

produce in the first T steps the same multiset of values, that is that one set is a permutation of the other.

We begin by modeling the statement with a direct but highly inefficient FSM-based approach based on Vieta's formulas and then we show the more efficient and straightforward permutation-based approach presented next.

1. ***Without Non-Local Constraints***

   We begin by modeling the CI statement. We recall that two sets $\{s_i(t)\}_{t=0}^{T-1}$ *for* $i = 1, 2$ *are one the permutation of the other if and only if they have the same vanishing polynomial*

$$\prod_{t=0}^{T-1}(X - s_i(t)).$$

   As seen in [LN97, p. 29] [DF04, p. 608], we can decompose the polynomial as

$$\prod_{t=0}^{T-1}(X - s_i(t)) = X^T - e_{i,1}X^{T-1} + e_{i,2}X^{T-2} + \cdots + (-1)^T e_{i,T}$$

   where the coefficients $e_{i,k}$ *are the elementary symmetric polynomials computed over the entire sequence* $\{s_i(t)\}_{t=0}^{T-1}$*, that is the sum of all possible combinations of k elements picked in* $\{s_i(t)\}_{t=0}^{T-1}$

$$e_{i,k} := \begin{cases} 1 & \text{if } k = 0 \\ \sum_{0 \le t_1 < t_2 < \cdots < t_k \le T-1} \prod_{j=1}^{k} s_i(t_j) & \text{if } k > 0 \end{cases}.$$

   If we denote the coefficents computed using the partial sequence $\{s_i(\tau)\}_{\tau=0}^{t}$ *by* $e_{i,k}(t)$*, we get*

$$e_{i,k}(t) := \begin{cases} 1 & \text{if } k = 0 \\ \sum_{0 \le t_1 < \cdots < t_k \le t} \prod_{j=1}^{k} s_i(t_j) & \text{if } k > 0 \end{cases}.$$

   We can now compute the coefficients using the recursive formula

$$e_{i,k}(t) = \begin{cases} 1 & \text{if } k = 0, t = 0 \\ s_i(0) & \text{if } k = 1, t = 0 \\ 0 & \text{if } k > 1, t = 0 \\ e_{i,k}(t-1) + s_i(t) \cdot e_{i,k-1}(t-1) & \text{if } t > 0 \end{cases}, \quad \text{for } i = 1, 2.$$

   This is because for the products of k elements chosen from $\{s_i(\tau)\}_{\tau=0}^{t}$ *we have two possibilities*

   - *if they do not contain the factor* $s_i(t)$ *then we have* $e_{i,k}(t-1)$ *that corresponds to the choice of elements from* $\{s_i(\tau)\}_{\tau=0}^{t-1}$*;*
   - *if they contain the factor* $s_i(t)$ *then we have a product of* $k-1$ *elements chosen from* $\{s_i(\tau)\}_{\tau=0}^{t-1}$ *and it suffices to multiply by* $s_i(t)$*.*

   This allows us to model the CI statement using a FSM that in its state contains the values of the original sequences, the values of all the coefficients $e_{i,k}(t)$ *for* $k > 0$ *of the respective polynomials and their differences. The complete state transition function is then*

$$\begin{aligned} \delta: \quad & \mathbb{F}_q^{2+3T} \to \mathbb{F}_q^{2+3T} \\ & (x_1, \ldots, x_{2+3T}) \mapsto (y_1, \ldots, y_{2+3T})' \end{aligned}$$

where the components of y are computed using the previous state x by

$$
\begin{aligned}
y_1 &= \delta_1(x_1) \\
y_2 &= \delta_2(x_2) \\
y_{2+k} &= \begin{cases} x_{2+k} + \delta_1(x_1) & \text{if } k = 1 \\ x_{2+k} + \delta_1(x_1) \cdot x_{2+k-1} & \text{if } k > 1 \end{cases} \\
y_{2+T+k} &= \begin{cases} x_{2+T+k} + \delta_2(x_2) & \text{if } k = 1 \\ x_{2+T+k} + \delta_2(x_2) \cdot x_{2+T+k-1} & \text{if } k > 1 \end{cases} \\
y_{2+2T+k} &= y_{2+k} - y_{2+T+k} = \\
&= \begin{cases} x_{2+k} - x_{2+T+k} + \delta_1(x_1) - \delta_2(x_2) & \text{if } k = 1 \\ x_{2+k} - x_{2+T+k} + \delta_1(x_1) \cdot x_{2+k-1} - \delta_2(x_2) \cdot x_{2+T+k-1} & \text{if } k > 1 \end{cases}
\end{aligned}
$$

The point conditions in B are obtained imposing the values for the initial state

$$s(0) = (v_{1,0}, v_{2,0}, \underbrace{v_{1,0}, 0, \ldots, 0}_{e_{1,k}}, \underbrace{v_{2,0}, 0, \ldots, 0}_{e_{2,k}}, \underbrace{v_{1,0} - v_{2,0}, 0, \ldots, 0}_{e_{1,k} - e_{2,k}})$$

and, to guarantee that the two sequences have generated the same multiset of values, at time T all the components relative to the difference must vanish, that is

$$s_k(T) = 0 \quad \text{for each } k \in \{2T+2, \ldots, 3T+2\}.$$

The CI statement

$$\mathcal{A} = (\delta, 2+3T, 0, T, B)$$

can now be translated into the associated $\mathcal{AIR}(\mathcal{A})$. The Prover will now have to compute $2+3T$ trace polynomials so this approach is higly inefficient.

2. ***With Non-Local constraints***

   We choose the same $\mathbb{F} = \mathbb{F}_q$ but will only use $w = 2$ columns.

   The local constraint polynomials are simply

   $$P_i(\mathbf{X}, \mathbf{Y}) = Y_i - D_i(X_1, X_2) \text{ for } i = 1, 2$$

   with evaluation domain $H \setminus \{h_T\}$, hence

   $$\mathcal{P} = \{(Y_i - D_i(X_1, X_2), H \setminus \{h_T\}) \mid i = 1, 2\}.$$

   We can then use a single non-local transition constraint polynomial

   $$P_{\boldsymbol{\pi}}(\mathbf{X}, \mathbf{Y}) = Y_2 - X_1$$

   on the evaluation domain H and we get the specification

   $$\mathcal{P}_{\boldsymbol{\pi}} = (Y_2 - X_1, H).$$

   If the AIR is satisfiable we have that there exists a polynomial witness $\mathbf{W}(X)$ such that

   $$P_{\pi_k}(\mathbf{W}(h), \mathbf{W}(\pi_k(h))) = 0 \quad \text{for each } h \in H$$

   that is

   $$W_1(h) = W_2(\pi_k(h)) \quad \text{for each } h \in H.$$

meaning that the trace columns are one the permutation of the other.

Finally for the state constraint polynomials we take $B_i(\mathbf{X}) = X_i - v_{i,0}$ with evaluation domain $\{h_0\}$, that gives the specification

$$\mathcal{B} = \{(X_i - v_{i,0}, \{h_0\}) \mid i = 1, 2\}.$$

We obtain the AIR

$$\mathcal{AIR} = (\mathbb{F}_q, T, 2, H, \sigma(X), \mathcal{P}, \mathcal{P}_\pi, \mathcal{B})$$

that requires the computation of only two trace polynomials.

We have that $|\mathcal{W}_{\mathcal{AIR}(\mathcal{A})}| = |\mathcal{W}_{\mathcal{AIR}}|$ because the auxiliary columns added in the inefficient approach are functionally dependant from the first two.

Remark 2.4.8 (Notation)
To simplify notation, we will directly refer to the set of all the constraint specifications as

$$\mathcal{C} = \{(C_1, H_{C_1}), \ldots, (C_s, H_{C_s})\}$$

and will write $\mathcal{AIR} = (\mathbb{F}, T, w, H, \sigma(X), \mathcal{C})$. We will use the complete notation only when necessary to distinguish between the different types of constraints.

Thus far, we have taken a vertical approach that consists in remodeling an AIR from scratch for a given mathematical proposition. This method requires proving the logical equivalence between the satisfiability of the AIR and the original statement on a case-by-case basis.

However, this ad-hoc modeling can be replaced by a more systematic and horizontal approach that consists in extending the trace while preserving the underlying logic.

Therefore we formalize the concept of valid extension in a definition.

Definition 2.4.9 (Valid Extension)
Let $\mathcal{AIR}_1 = (\mathbb{F}, T, w, H, \sigma(X), \mathcal{C})$ and $\mathcal{AIR}_2 = (\mathbb{F}, T', w', H', \sigma'(X), \mathcal{C}')$ be two AIRs such that $T' \geq T$ and $w' \geq w$. We will say that $\mathcal{AIR}_2$ is a valid extension *of $\mathcal{AIR}_1$ and write*

$$\mathcal{AIR}_1 \preceq \mathcal{AIR}_2$$

if it exists a function

$$\text{Ext} : \mathbb{F}^{(T+1)\times w} \times H^{(T+1)\times r} \to \mathbb{F}^{(T'+1)\times w'} \times H'^{(T'+1)\times r'}$$

such that the following hold

- ***Information Preservation:*** *given $(x, p) \in \mathbb{F}^{(T+1)\times w} \times H^{(T+1)\times r}$, then if*

  $$(x', p') = \text{Ext}(x, p)$$

  we get $x'_{[0:T;1:w]} = x$.

  Where $x'_{[0:T;1:w]}$ is the submatrix of x' that contains the elements $x'_k(t)$ for $t \in \{0, \ldots, T\}$ and $k \in \{1, \ldots, w\}$.

- ***Validity Equivalence:*** $(x, p) \in \mathcal{T}_{\mathcal{AIR}_1} \iff \text{Ext}(x, p) \in \mathcal{T}_{\mathcal{AIR}_2}$.

We now prove that the relation we just defined is a preorder.

Proposition 2.4.10 (Preorder Property)
The relation $\preceq$ is a preorder.

Proof. We prove the two properties.

- **Reflexivity:** $\mathcal{AIR} \preceq \mathcal{AIR}$, it suffices to take Ext = Id.
- **Transitivity:** Given $\mathcal{AIR}_1 \preceq \mathcal{AIR}_2$ through $\text{Ext}_{1,2}$ and $\mathcal{AIR}_2 \preceq \mathcal{AIR}_3$ through $\text{Ext}_{2,3}$.

  Let $T_1 \leq T_2 \leq T_3$, $w_1 \leq w_2 \leq w_3$ and $r_1 \leq r_2 \leq r_3$ be the respective dimensions and consider the composition $\text{Ext}_{1,3} = \text{Ext}_{2,3} \circ \text{Ext}_{1,2}$.

  We need to verify the two properties

  - **Information Preservation:** Given $(x, p) \in \mathbb{F}^{(T_1+1)\times w_1} \times H^{(T_1+1)\times r_1}$, then if $(x', p') = \text{Ext}_{1,2}(x, p)$ we get

    $$x'_{[0:T_1;1:w_1]} = x.$$

    We have that $\text{Ext}_{2,3}$ leaves unchanged the block $[0 : T_2; 1 : w_2]$ and then also leaves unchanged the block $[0 : T_1; 1 : w_1]$, therefore if $(x'', p'') = \text{Ext}_{2,3}(\text{Ext}_{1,2}(x, p))$ we have

    $$x''_{[0:T_1;1:w_1]} = (x''_{[0:T_2;1:w_2]})_{[0:T_1;1:w_1]} = x'_{[0:T_1;1:w_1]} = x.$$

  - **Validity Equivalence:**
    We have

    $$(x, p) \in \mathcal{T}_{\mathcal{AIR}_1} \iff \text{Ext}_{1,2}(x, p) \in \mathcal{T}_{\mathcal{AIR}_2} \iff \text{Ext}_{2,3}(\text{Ext}_{1,2}(x, p)) \in \mathcal{T}_{\mathcal{AIR}_3}.$$

  Both conditions hold so $\mathcal{AIR}_1 \preceq \mathcal{AIR}_3$.

□

Given that $\preceq$ is a preorder, using the transitivity we can compose multiple optimizations.

We will also need a probabilistic version of the definition.

Definition 2.4.11 (Probabilistically Sound Extension)
Let $\mathcal{AIR}_1 = (\mathbb{F}, T, w, H, \sigma(X), \mathcal{C})$ and $\mathcal{AIR}_2 = (\mathbb{F}, T', w', H', \sigma'(X), \mathcal{C}')$ be two AIRs such that $T' \geq T$ and $w' \geq w$. We will say that $\mathcal{AIR}_2$ is a probabilistically sound extension *of $\mathcal{AIR}_1$ and write*

$$\mathcal{AIR}_1 \preceq^{\epsilon}_{\text{Pr}} \mathcal{AIR}_2$$

if there exist an integer $s \in \mathbb{Z}^+$ and a function

$$\text{Ext} : \mathbb{F}^{(T+1)\times w} \times H^{(T+1)\times r} \times \mathbb{F}^s \to \mathbb{F}^{(T'+1)\times w'} \times H'^{(T'+1)\times r'}$$

such that the following hold

- ***Information Preservation:*** *given $((x, p), R) \in \mathbb{F}^{(T+1)\times w} \times H^{(T+1)\times r} \times \mathbb{F}^s$, then if $(x', p') = \text{Ext}((x, p), R)$ we get*

  $$x'_{[0:T;1:w]} = x$$

- ***Completeness:*** $(x, p) \in \mathcal{T}_{\mathcal{AIR}_1} \implies \text{Ext}((x, p), R) \in \mathcal{T}_{\mathcal{AIR}_2}$ *for each $R \in \mathbb{F}^s$*

- ***Soundness:*** *if $(x, p) \notin \mathcal{T}_{\mathcal{AIR}_1}$ then for the random parameter R drawn uniformly from $\mathbb{F}^s$ we have*

  $$\Pr_{R\in\mathbb{F}^s}[\text{Ext}((x, p), R) \in \mathcal{T}_{\mathcal{AIR}_2}] \leq \epsilon \quad \text{for } \epsilon > 0.$$

Remark 2.4.12 (Dependence on the Random Parameter)
It is worth noting that the dependence on the random parameter R can manifest in two distinct places. It can influence the behavior of the extension function $\mathrm{Ext}((x,p),R)$ *itself, and it can also be incorporated directly into the constraint polynomials of $\mathcal{AIR}_2$. For notational simplicity, we write $\mathcal{AIR}_2$ without explicitly showing this dependence on R.*

The probabilistic version is also a preorder.

Proposition 2.4.13 (Preorder Property)
The relation $\preceq^{\epsilon}_{\mathrm{Pr}}$ is a preorder.

Proof. We prove the two properties.

- **Reflexivity:** $\mathcal{AIR} \preceq^{\epsilon}_{\mathrm{Pr}} \mathcal{AIR}$, it suffices to consider $\mathrm{Ext} : ((x,p),R) \mapsto (x,p)$.
- **Transitivity:** Suppose that $\mathcal{AIR}_1 \preceq^{\epsilon_1}_{\mathrm{Pr}} \mathcal{AIR}_2$ using $\mathrm{Ext}_{1,2}$ with casual parameter $R_1 \in \mathbb{F}^{s_1}$ and that $\mathcal{AIR}_2 \preceq^{\epsilon_2}_{\mathrm{Pr}} \mathcal{AIR}_3$ using $\mathrm{Ext}_{2,3}$ with casual parameter $R_2 \in \mathbb{F}^{s_2}$.

  Let $T_1 \leq T_2 \leq T_3$ and $w_1 \leq w_2 \leq w_3$ the respective dimensions and given

  $$R = (R_1, R_2)$$

  consider the composition

  $$\mathrm{Ext}_{1,3}((x,p),R) = \mathrm{Ext}_{2,3}(\mathrm{Ext}_{1,2}((x,p),R_1),R_2).$$

  We need to verify the three conditions.

  - **Information Preservation:** let $((x,p),R_1) \in \mathbb{F}^{(T_1+1)\times w_1} \times H^{(T_1+1)\times r_1} \times \mathbb{F}^{s_1}$ then if $(x',p') = \mathrm{Ext}_{1,2}((x,p),R_1)$ we have $x'_{[0:T_1;1:w_1]} = x$.

    Now $\mathrm{Ext}_{2,3}$ leaves unchanged the block $[0:T_2;1:w_2]$ and then also leaves unchanged the block $[0:T_1;1:w_1]$, therefore if $(x'',p'') = \mathrm{Ext}_{2,3}(\mathrm{Ext}_{1,2}((x,p),R_1),R_2)$ we have

    $$x''_{[0:T_1;1:w_1]} = (x''_{[0:T_2;1:w_2]})_{[0:T_1;1:w_1]} = x'_{[0:T_1;1:w_1]} = x.$$

  - **Completeness:** We have

    $$(x,p) \in \mathcal{T}_{\mathcal{AIR}_1} \implies \forall R_1 \in \mathbb{F}^{s_1}\ \mathrm{Ext}_{1,2}((x,p),R_1) \in \mathcal{T}_{\mathcal{AIR}_2}$$

    and

    $$\mathrm{Ext}_{1,2}((x,p),R_1) \in \mathcal{T}_{\mathcal{AIR}_2} \implies \forall R_2 \in \mathbb{F}^{s_2}\ \mathrm{Ext}_{2,3}(\mathrm{Ext}_{1,2}((x,p),R_1),R_2) \in \mathcal{T}_{\mathcal{AIR}_3}.$$

    Therefore

    $$(x,p) \in \mathcal{T}_{\mathcal{AIR}_1} \implies \forall R_1 \in \mathbb{F}^{s_1} \text{ and } \forall R_2 \in \mathbb{F}^{s_2}\ \mathrm{Ext}_{2,3}(\mathrm{Ext}_{1,2}((x,p),R_1),R_2) \in \mathcal{T}_{\mathcal{AIR}_3}.$$

  - **Soundness:** Suppose that $(x,p) \notin \mathcal{T}_{\mathcal{AIR}_1}$ and define the following events

    $$A := \left\{ \mathrm{Ext}_{2,3}(\mathrm{Ext}_{1,2}((x,p),R_1),\ R_2) \in \mathcal{T}_{\mathcal{AIR}_3} \right\},$$
    $$B := \left\{ \mathrm{Ext}_{1,2}((x,p),R_1) \in \mathcal{T}_{\mathcal{AIR}_2} \right\}.$$

    We have

    $$\Pr[A] = \Pr[A \cap B] + \Pr[A \cap B^C]$$

and using the conditional probability formula

$$\Pr[A] = \Pr[A|B]\Pr[B] + \Pr[A|B^c]\Pr[B^c].$$

Because $\mathcal{AIR}_1 \preceq^{\epsilon_1}_{\Pr} \mathcal{AIR}_2$ we have $\Pr[B] \leq \epsilon_1$.
Furthermore $\mathcal{AIR}_2 \preceq^{\epsilon_2}_{\Pr} \mathcal{AIR}_3$ then we have two possibilities:

1. if the event B happens then by completeness $P[A|B] = 1$;
2. if the event B^c happens then by soundness $P[A|B^c] \leq \epsilon_2$.

Thus

$$\begin{aligned}\Pr[A] = \Pr[A|B]\Pr[B] + \Pr[A|B^c]\Pr[B^c] &\leq \Pr[B] + \epsilon_2(1 - \Pr[B]) \leq \epsilon_1 + \epsilon_2(1 - \epsilon_1) \\ &\leq \epsilon_1 + \epsilon_2\end{aligned}$$

so the soundness is satisfied with error $\epsilon = \epsilon_1 + \epsilon_2$.

We conclude that $\mathcal{AIR}_1 \preceq^{\epsilon_1+\epsilon_2}_{\Pr} \mathcal{AIR}_3$.

□

Remark 2.4.14 (Remodeling and Extensions)
In this section we proposed the distinction between the vertical approach of remodeling and the horizontal approach of extending the trace. Although in the STARK literature [BBHR18b, BBHR19, Tea21, AMGM24, MF23] various optimization techniques are described, the authors do not introduce a taxonomy to differentiate them.

In the following chapters, we will utilize the framework of valid and probabilistically sound extensions to analyze and classify the fundamental optimizations presented in STARK literature:

- *In Chapter 3 we will introduce the* execution trace padding *needed to use fast transforms and will prove that it is a valid extension.*
- *In Chapter 4, we will analyze* permutation arguments *and* constraint polynomial unrolling. *We will prove that they are respectively a probabilistically sound extension and a valid extension.*

  We will finally introduce a model for proving generic computations using Instruction Set Architectures (ISA), *to do that we will aggregate the unrolled instruction constraint polynomials with a probabilistically sound extension and, at last, we will introduce the* preprocessing *technique to encode programs, that is another example of a valid extension.*

Chapter 3

Discrete Fourier Transforms and Fast Fourier Transforms

In this chapter we introduce the Discrete Fourier Transform (DFT) in a generic algebraic context, we then explore its computation using the Fast Fourier Transform (FFT).

We will then analyze the consequences of the adoption of FFTs on the choice of the field $\mathbb{F}_q$.

At last, we will prove that padding the execution trace to a lenght suitable for the use of FFTs is a valid extension in the sense of Definition 2.4.9.

3.1 Discrete Fourier Transform (DFT)

The Discrete Fourier Transform (DFT) can be defined on an arbitrary field $\mathbb{K}$ that possesses certain algebraic properties that make it well-defined and invertible.

3.1.1 Preliminary Algebraic Conditions

We introduce the definitions of n-th root of unity and n-th primitive root of unity as in [vzGG99, p. 215]

Definition 3.1.1 (n-th Root of Unity)
Let $\mathbb{K}$ be a field and $1_{\mathbb{K}} \in \mathbb{K}$ be the multiplicative identity element. Given $n \in \mathbb{Z}^+$, an element $\omega \in \mathbb{K}$ is a n-th root of unity *if it is a root of $X^n - 1_{\mathbb{K}} \in \mathbb{K}[X]$, that is $\omega^n = 1_{\mathbb{K}}$.*

Definition 3.1.2 (Primitive n-th Root Of Unity)
Let $\mathbb{K}$ be a field and $1_{\mathbb{K}} \in \mathbb{K}$ be the multiplicative identity. Given $n \in \mathbb{Z}^+$, an n-th root of unity $\omega_n \in \mathbb{K}$ is called primitive n-th root of unity *if $\omega_n^k \neq 1_{\mathbb{K}}$ for each $0 < k < n$.*

Remark 3.1.3 (Characterization Using the Multiplicative Order)
Using the definition, it follows that an element $\omega_n \in \mathbb{K}$ is a primitive n-th root of unity if and only if $\omega_n \in \mathbb{K}^*$ and has multiplicative order n. This is true because the condition $\omega_n^n = 1_{\mathbb{K}}$ and $\omega_n^k \neq 1_{\mathbb{K}}$ for each $0 < k < n$ is precisely the definition of elements of multiplicative order n.

We now prove some simple properties that are mostly stated in [Bin, p. 2]

Proposition 3.1.4 (Primitive n-th Roots of Unity Properties)
Let $\mathbb{K}$ be a field and let $n \in \mathbb{Z}^+$. If $\omega_n \in \mathbb{K}$ is a primitive n-th root of unity then

1. *the powers $1_{\mathbb{K}}, \omega_n, \ldots, \omega_n^{n-1}$ are all distinct;*
2. *the set $\{1_K, \omega_n, \ldots, \omega_n^{n-1}\}$ is the set of all n-th roots of unity in $\mathbb{K}$ and is a cyclic multiplicative subgroup $\langle \omega_n \rangle < \mathbb{K}^*$, of order n;*

3. *the inverse ω_n^{-1} is still a primitive n-th root of unity.*

Proof. We prove each point

1. Assume by contradiction that $\omega_n^i = \omega_n^j$ for $0 \leq j < i < n$, then $\omega_n^{i-j} = 1_{\mathbb{K}}$ with $0 < i - j < n$ and that contradicts the definition of primitive n-th root of unity, because the order of ω_n is n.

2. The n distinct powers ω_n^k are all n-th roots of unity because

$$(\omega_n^k)^n = (\omega_n^n)^k = 1_{\mathbb{K}}^k = 1_{\mathbb{K}} \quad \text{for each } k \in \{0, \dots, n-1\}.$$

   Given that the polynomial $X^n - 1_{\mathbb{K}}$ has at most n roots in $\mathbb{K}$, those form the complete set of n-th roots of unity in $\mathbb{K}$.

   Now we prove that the set $\{1_{\mathbb{K}}, \omega_n, \dots, \omega_n^{n-1}\}$ is a subgroup of $\mathbb{K}^*$.

   We need to verify the subgroup properties:

   - **Closure**: For $i, j \in \{0, 1, \dots, n-1\}$ we have that $\omega_n^i \cdot \omega_n^j = \omega_n^{i+j \mod n}$, that is still an element of the set.
   - **Identity Element**: $\omega_n^0 = 1_{\mathbb{K}}$ is in the set.
   - **Inverse**: For each ω_n^k, the inverse is ω_n^{n-k} that is in the set.

   Given that the set is generated by ω_n, this is exactly the cyclic subgroup $\langle \omega_n \rangle < \mathbb{K}^*$ of order n.

3. Using the previous points ω_n^{-1} is again a n-th root of unity. Suppose by contradiction that it is not primitive, then it exists $0 < k < n$ such that $(\omega_n^{-1})^k = 1_{\mathbb{K}}$ that is $\omega_n^{-k} = 1_{\mathbb{K}}$.

   We then have $\omega_n^k = 1_{\mathbb{K}}$ that is absurd because ω_n is primitive.

□

Now we analyze the conditions that have to be met to guarantee the existence of a primitive n-th root of unity, for this we need the definition of characteristic [LN97, p. 16].

Definition 3.1.5 (Field Characteristic)
The characteristic *of a field $\mathbb{K}$, denoted by $char(\mathbb{K})$, is the smallest $p \in \mathbb{Z}^+$ such that*

$$p \cdot 1_{\mathbb{K}} = \underbrace{1_{\mathbb{K}} + \cdots + 1_{\mathbb{K}}}_{p \text{ times}} = 0_{\mathbb{K}},$$

where $0_{\mathbb{K}}$ is the additive identity element.

If such p doesn't exist, we have $char(\mathbb{K}) = 0$.

Remark 3.1.6 (Infinite Fields Characteristic)
Rational numbers $\mathbb{Q}$, real numbers $\mathbb{R}$ and complex numbers $\mathbb{C}$ have 0 characteristic, given that the sum $\underbrace{1 + 1 + \cdots + 1}_{n \text{ times}} = n$ is never 0.

Following again [LN97, p. 16] we prove

Proposition 3.1.7 (Prime Characteristic)
If $char(\mathbb{K}) = p > 0$, then p is a prime number.

Proof. Suppose by contradiction that p is not prime, then there are two numbers $a, b \in \mathbb{N}$ with $1 < a \leq b < p$ such that $p = ab$. Given that the field characteristic is p we have that $p \cdot 1_{\mathbb{K}} = 0_{\mathbb{K}}$, therefore $(a \cdot 1_{\mathbb{K}})(b \cdot 1_{\mathbb{K}}) = 0_{\mathbb{K}}$. This means that $a \cdot 1_{\mathbb{K}} = 0_{\mathbb{K}}$ or $b \cdot 1_{\mathbb{K}} = 0_{\mathbb{K}}$, and this is absurd given the minimality of p. □

This gives the following corollary, also in [LN97, p. 16].

Corollary 3.1.8 (Finite Field Characteristic)
Let $\mathbb{F}_q$ be a finite field with $q = p^k$ where p is a prime and $k \in \mathbb{Z}^+$. Then char$(\mathbb{F}_q) = p$.

Proof. Let $\text{char}(\mathbb{F}_q) = r$ and consider $m \cdot 1_{\mathbb{F}_q}$ for $m = 1, 2, \ldots, q+1$.

Given that $|\mathbb{F}_q| = q$, by the pigeonhole principle there are two indices $i, j \in \mathbb{N}$ with $1 \leq j < i \leq q+1$ such that $i \cdot 1_{\mathbb{F}_q} = j \cdot 1_{\mathbb{F}_q}$.

Therefore $(i-j) \cdot 1_{\mathbb{F}_q} = 0_{\mathbb{F}_q}$ with $i - j > 0$ that implies $r > 0$ and from Proposition 3.1.7 we know that r must be prime.

Now consider the additive subgroup generated by $1_{\mathbb{F}_q}$, it has order r and for the Lagrange Theorem for groups we have $r \mid p^k$. It follows that $r = p$. □

We analyze the existence of the primitive n-th root of unity in different types of fields.

Proposition 3.1.9 (Complex Numbers)
Let $\mathbb{K} = \mathbb{C}$ and $n \in \mathbb{Z}^+$, then the element $\omega_n = e^{\frac{2\pi i}{n}}$ is a primitive n-th root of unity.

Proof. We have that $\omega_n^n = \left(e^{\frac{2\pi i}{n}}\right)^n = e^{2\pi i} = 1$, so it is a n-th root of unity. To prove that it is primitive, suppose by contradiction that exists $0 < k < n$ such that $\omega_n^k = 1$. This means that $e^{\frac{2\pi i k}{n}} = e^{2\pi i}$ and so $\dfrac{k}{n} \in \mathbb{Z}$, that is absurd because $0 < \dfrac{k}{n} < 1$.

Therefore $\omega_n = e^{\frac{2\pi i}{n}}$ is a primitive n-th root of unity. □

Now for finite fields we prove the Lemma found in [vzGG99, p. 217].

Proposition 3.1.10 (Finite Fields)
Let $\mathbb{K} = \mathbb{F}_q$ be a finite field and let $n \in \mathbb{Z}^+$. The finite field $\mathbb{F}_q$ contains a primitive n-th root of unity if and only if n divides $q - 1$.

Proof. Recall that the multiplicative subgroup $\mathbb{F}_q^*$ is cyclic [LN97, p. 50] and has order $q - 1$. We prove the two implications

$\Longrightarrow$ Suppose that $\omega_n \in \mathbb{F}_q$ is a primitive n-th root of unity. We have that $\omega_n \in \mathbb{F}_q^*$ and it is an element of order n, by the Lagrange theorem for groups the order of an element must divide the order of the group, so we have $n \mid (q-1)$.

$\Longleftarrow$ Given that $\mathbb{F}_q^*$ is a cyclic subgroup of order $q - 1$, it contains an element of order d for each divisor d of $q - 1$. Therefore, if $n \mid (q-1)$, there exists an element of order n in $\mathbb{F}_q^*$, that is a primitive n-th root of unity.

□

The field characteristic is fundamental to guarantee the existence of the inverse of the element $n_{\mathbb{K}} = n \cdot 1_{\mathbb{K}}$, this will be crucial to define the Inverse Discrete Fourier Transform (IDFT).

Proposition 3.1.11 (Inverse)
Let $\mathbb{K}$ be a field and $n \in \mathbb{Z}^+$. The element $n_{\mathbb{K}} = n \cdot 1_{\mathbb{K}}$ is invertible in $\mathbb{K}$ if and only if char$(\mathbb{K})$ does not divide n.

Proof. If $\text{char}(\mathbb{K}) = 0$, as seen for $\mathbb{Q}, \mathbb{R}$ and $\mathbb{C}$, we have $n_{\mathbb{K}} = n$ that is not zero and then invertible.

If $\text{char}(\mathbb{K}) = p > 0$, as seen for finite fields, we have $n_{\mathbb{K}} = (n \pmod p) \cdot 1_{\mathbb{K}}$ that is not zero and so invertible if and only if p does not divide n. □

Following the reasoning in [LN97, pp. 63-64] we now see that this condition is also necessary for the existence of a primitive n-th root of unity.

Proposition 3.1.12 (Necessary Condition)
Let $\mathbb{K}$ be a field and let $n \in \mathbb{Z}^+$. If it exists a primitive n-th root of unity $\omega_n \in \mathbb{K}$ then $\text{char}(\mathbb{K}) \nmid n$.

Proof. Suppose by contradiction that a primitive n-th root of unity $\omega_n \in \mathbb{K}$ exists but that $\text{char}(\mathbb{K}) \mid n$. Then we have $\text{char}(\mathbb{K}) = p > 0$ and $n = mp^k$ for some $0 < m, p^k < n$.

Using the binomial theorem we obtain

$$(X^m - 1_{\mathbb{K}})^{p^k} = \sum_{i=0}^{p^k} \binom{p^k}{i} X^{m(p^k - i)} (-1_{\mathbb{K}})^{p^k}.$$

Each term in the binomial coeffcient, excluding the first and the last, is a multiple of p^k. Given that $\text{char}(\mathbb{K}) = p$, we then have

$$(X^m - 1_{\mathbb{K}})^{p^k} = X^n - 1_{\mathbb{K}}$$

and that is absurd because ω_n would be a m-th root of unity with $m < n$. □

As a consequence of what we have proven, assuming the existence of a primitive n-th root of unity $\omega_n \in \mathbb{K}$ will automatically guarantee $\text{char}(\mathbb{K}) \nmid n$ and then, given the Proposition 3.1.11, that the element $n_{\mathbb{K}}$ is invertible in $\mathbb{K}$.

3.1.2 DFT Definition over $\mathbb{K}$

Given the algebraic prerequisites, we can now define the Discrete Fourier Transform (DFT) and its inverse (IDFT) respectively as solutions of the evaluation and interpolation problems over a set of nodes called Fourier nodes, we will follow [Bin] and [BBCM92, p. 417].

Definition 3.1.13 (Fourier Nodes)
Let $n \in \mathbb{Z}^+$ and let $\mathbb{K}$ be a field such that there exists a primitive n-th root of unity $\omega_n \in \mathbb{K}$.

We call Fourier nodes *the elements of the cyclic evaluation domain*

$$\langle \omega_n \rangle = \{1_{\mathbb{K}}, \omega_n, \dots, \omega_n^{n-1}\} < \mathbb{K}^*.$$

Given a vector of evaluations $\mathbf{y} = (y_0, y_1, \dots, y_{n-1})^{\mathrm{T}} \in \mathbb{K}^n$, the problem of interpolation over the Fourier nodes is to determine the vector of coefficients

$$\mathbf{a} = (a_0, a_1, \dots, a_{n-1})^{\mathrm{T}} \in \mathbb{K}^n$$

of the unique polynomial of degree at most $n - 1$ (see Theorem 2.2.4)

$$P(X) = a_0 + a_1 X + \cdots + a_{n-1} X^{n-1} \in \mathbb{K}[X]$$

such that the interpolation conditions are satisfied

$$P(\omega_n^i) = y_i \quad \text{for each } i \in \{0, 1, \dots, n-1\}.$$

Those conditions can be written as

$$y_i = \sum_{j=0}^{n-1} a_j(\omega_n^i)^j = \sum_{j=0}^{n-1} a_j\omega_n^{ij} \quad \text{for each } i \in \{0,1,\dots,n-1\}.$$

We can represent this system in matrix form using the Fourier matrix.

Definition 3.1.14 (Fourier Matrix)
Let $n \in \mathbb{Z}^+$ and let $\mathbb{K}$ be a field such that there exists a primitive n-th root of unity $\omega_n \in \mathbb{K}$. The Fourier matrix *(also* DFT matrix*), denoted by V_{ω_n}, is the matrix of elements*

$$(V_{\omega_n})_{i,j} := \omega_n^{ij \pmod n} \quad \textit{for } i,j \in \{0,1,\dots,n-1\}.$$

Remark 3.1.15 (Visualizing the Fourier Matrix)
Making the elements explicit we have

$$V_{\omega_n} = \begin{pmatrix} 1 & 1 & 1 & \cdots & 1 \\ 1 & \omega_n & \omega_n^2 & \cdots & \omega_n^{n-1} \\ 1 & \omega_n^2 & \omega_n^4 & \cdots & \omega_n^{2(n-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega_n^{n-1} & \omega_n^{2(n-1)} & \cdots & \omega_n^{(n-1)^2} \end{pmatrix} = \begin{pmatrix} 1 & 1 & 1 & \cdots & 1 \\ 1 & \omega_n & \omega_n^2 & \cdots & \omega_n^{n-1} \\ 1 & \omega_n^2 & \omega_n^4 & \cdots & \omega_n^{n-2} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega_n^{n-1} & \omega_n^{n-2} & \cdots & \omega_n \end{pmatrix}.$$

Therefore the matrix V_{ω_n} is a symmetric Vandermonde matrix.

Formally, V_{ω_n} is a Vandermonde matrix because $(V_{\omega_n})_{i,j} = (\omega_n^i)^j$ and is symmetric because $\omega_n^{ij} = \omega_n^{ji}$ for each $i,j \in \{0,1,\dots,n-1\}$.

The system of linear equations can be then written as

$$\mathbf{y} = V_{\omega_n}\mathbf{a}.$$

Definition 3.1.16 (Discrete Fourier Transform (DFT))
The linear transformation that maps the vector of coefficients $\mathbf{a} \in \mathbb{K}^n$ to the vector of evaluations $\mathbf{y} \in \mathbb{K}^n$ is called Discrete Fourier Transform (DFT)

$$DFT_n(\mathbf{a}) := V_{\omega_n}\mathbf{a} = \mathbf{y}.$$

This map corresponds to the evaluation of $P(X)$ over the Fourier nodes.

Proposition 3.1.17 (V_{ω_n} Invertibility)
The matrix V_{ω_n} is invertible and its inverse is $n_{\mathbb{K}}^{-1}V_{\omega_n^{-1}}$.

Proof. Given the candidate we verify that the product is the identity, we have

$$(V_{\omega_n} \cdot (n_{\mathbb{K}}^{-1}V_{\omega_n^{-1}}))_{i,j} = n_{\mathbb{K}}^{-1}(V_{\omega_n} \cdot V_{\omega_n^{-1}})_{i,j} = n_{\mathbb{K}}^{-1}\sum_{k=0}^{n-1}\omega_n^{ik}\omega_n^{-kj} = n_{\mathbb{K}}^{-1}\sum_{k=0}^{n-1}\omega_n^{(i-j)k} = \begin{cases} 1_{\mathbb{K}} \text{ if } i = j \\ 0_{\mathbb{K}} \text{ if } i \neq j \end{cases}$$

where we used $\sum_{k=0}^{n-1}\omega_n^{rk} = 0$ for $r \neq 0 \mod n$ because considering

$$1 - x^n = (1-x)\sum_{k=0}^{n-1} x^k$$

if we take $x = \omega_n^r$, given that $r \neq 0 \mod n$ we get $1 - x \neq 0$ and $1 - x^n = 0$ so it must be $\sum_{k=0}^{n-1}\omega_n^{rk} = \sum_{k=0}^{n-1} x^k = 0$. □

Taking the inverse we obtain the linear system

$$\mathbf{a} = n_{\mathbb{K}}^{-1} V_{\omega_n^{-1}} \mathbf{y}.$$

Definition 3.1.18 (Inverse Discrete Fourier Transform (IDFT))
The linear transformation that maps the vector of evaluations $\mathbf{y} \in \mathbb{K}^n$ *to the vector of coefficients* $\mathbf{a} \in K^n$ *is called* Inverse Discrete Fourier Transform (IDFT)*:*

$$IDFT_n(\mathbf{y}) := n_{\mathbb{K}}^{-1} V_{\omega_n^{-1}} \mathbf{y} = \mathbf{a}.$$

This map corresponds to the interpolation over the Fourier nodes of the evaluations $\mathbf{y}$*.*

Remark 3.1.19 (Terminology: DFT/IDFT)
In literature multiple conventions are adopted for indicating the DFT and its inverse, in this thesis we adopted the convention used by [VL92, vzGG99] where the DFT is the transform associated to the evaluation. Other authors [Bin, BBCM92] use the opposite.

Remark 3.1.20 (Terminology: NTT)
When $\mathbb{K} = \mathbb{F}_q$*, the DFT is called* Number-Theoretic Transform (NTT) *[Nus82, SSM+23]. As we have seen in Proposition 3.1.10, in this case the existence of a primitive n-th root of unity is equivalent to* $n \mid q - 1$*.*

Remark 3.1.21 (Terminology: Time Domain and Frequency Domain)
By analogy with the classic use of the DFT in signal analysis, where the transform maps a signal on the time domain to a representation on the frequency domain, it is common to adopt the following terminology

- *the index* j*, associated with the coefficients* $\mathbf{a}$*, is called* temporal index*;*
- *the index* i*, associated with the evaluations* $\mathbf{y}$*, is called* frequency index*.*

3.2 Fast Fourier Transform (FFT)

The DFT computation can be executed using algorithms that have quasilinear time complexity $O(n \log_2 n)$ that use the *divide et impera* principle, collectively called *Fast Fourier Transforms (FFTs)*. The term *fast* is used to distinguish this approach and the one that uses the matrix-vector multiplication, that would have complexity $O(n^2)$.

3.2.1 Kronecker Product

The first step is introducing the Kronecker product between matrices that is defined in [VL92, p. 7].

Definition 3.2.1 (Kronecker Product)
Let $A \in \mathbb{K}^{p \times q}$ *and* $B \in \mathbb{K}^{m \times n}$ *the* Kronecker product *is the block matrix* $A \otimes B \in \mathbb{K}^{pm \times qn}$ *with blocks of dimension* $m \times n$ *defined as*

$$A \otimes B := \begin{pmatrix} A_{0,0}B & \cdots & A_{0,q-1}B \\ \vdots & \ddots & \vdots \\ A_{p-1,0}B & \cdots & A_{p-1,q-1}B \end{pmatrix}$$

Remark 3.2.2 (Elements of the Kronecker Product)
Using the block structure we have

$$(A \otimes B)_{i,j} = A_{\lfloor i/m \rfloor, \lfloor j/n \rfloor} B_{i \pmod m, j \pmod n}$$

Remark 3.2.3 (Block Diagonal Matrices)
We can write a block diagonal matrix of dimension $pn \times qn$ in which the blocks on the diagonal are all equal to B using the Kronecker Product

$$I_n \otimes B = \begin{pmatrix} B & \mathbf{0} & \cdots & \mathbf{0} \\ \mathbf{0} & B & \cdots & \mathbf{0} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{0} & \mathbf{0} & \cdots & B \end{pmatrix}$$

We can now prove some simple properties of Kronecker products

Proposition 3.2.4 (Mixed Product)
Let A, B, C, D be matrices defined with dimensions such that the products AC and BD are well-defined. In particular, let $A \in \mathbb{K}^{p\times q}$, $B \in \mathbb{K}^{m\times n}$, $C \in \mathbb{K}^{q\times r}$ and $D \in \mathbb{K}^{n\times o}$.

The Kronecker product has the mixed product property

$$(A \otimes B)(C \otimes D) = AC \otimes BD$$

Proof. We just perform the product

$$\begin{aligned}(A \otimes B)(C \otimes D) &= \begin{pmatrix} A_{0,0}B & \cdots & A_{0,q-1}B \\ \vdots & \ddots & \vdots \\ A_{p-1,0}B & \cdots & A_{p-1,q-1}B \end{pmatrix} \begin{pmatrix} C_{0,0}D & \cdots & C_{0,r-1}D \\ \vdots & \ddots & \vdots \\ C_{q-1,0}D & \cdots & C_{q-1,r-1}D \end{pmatrix} = \\ &= \begin{pmatrix} \sum_{i=0}^{q-1} A_{0,i}C_{i,0}BD & \cdots & \sum_{i=0}^{q-1} A_{0,i}C_{i,r-1}BD \\ \vdots & \ddots & \vdots \\ \sum_{i=0}^{q-1} A_{p-1,i}C_{i,0}BD & \cdots & \sum_{i=0}^{q-1} A_{p-1,i}C_{i,r-1}BD \end{pmatrix} = AC \otimes BD\end{aligned}$$

□

Corollary 3.2.5 (Distributive Property with the Identity)
Let I be the identity matrix and let $A_1, A_2, \ldots, A_k$ be matrices such that the product $A_1A_2\cdots A_k$ is well-defined. Then the following equality holds

$$I \otimes (A_1A_2\cdots A_k) = (I \otimes A_1)(I \otimes A_2)\cdots(I \otimes A_k)$$

Proof. The proof proceeds by induction on k

- *Base Case ($k = 2$):* Using Proposition 3.2.4 we have

$$(I \otimes A_1)(I \otimes A_2) = I^2 \otimes A_1A_2 = I \otimes A_1A_2$$

- *Inductive Step:* Suppose that the conclusion is valid for $k-1$, then we have

$$I \otimes (A_1A_2\cdots A_{k-1}) = (I \otimes A_1)(I \otimes A_2)\cdots(I \otimes A_{k-1}).$$

  Using the base case and then the inductive hypotesis we have

$$\begin{aligned}I \otimes (A_1A_2\cdots A_k) &= I \otimes ((A_1A_2\cdots A_{k-1})A_k) \\ &= [I \otimes (A_1A_2\cdots A_{k-1})](I \otimes A_k) \\ &= (I \otimes A_1)(I \otimes A_2)\cdots(I \otimes A_k)\end{aligned}$$

□

Proposition 3.2.6 (Associative Property)
Let $A \in \mathbb{K}^{p\times q}$, $B \in \mathbb{K}^{m\times n}$ and $C \in \mathbb{K}^{r\times s}$. The Kronecker product is associative

$$(A \otimes B) \otimes C = A \otimes (B \otimes C)$$

Proof. We write the i,j-th element for both the matrices, for the first one we have

$$\begin{aligned}((A \otimes B) \otimes C)_{i,j} &= (A \otimes B)_{\lfloor i/r\rfloor,\lfloor j/s\rfloor} C_{i \pmod r, j \pmod s} \\ &= A_{\lfloor \lfloor i/r\rfloor/m\rfloor,\lfloor \lfloor j/s\rfloor/n\rfloor} B_{\lfloor i/r\rfloor \pmod m,\lfloor j/s\rfloor \pmod n} C_{i \pmod r, j \pmod s}. \\ &= A_{\lfloor i/rm\rfloor,\lfloor j/sn\rfloor} B_{\lfloor i/r\rfloor \pmod m,\lfloor j/s\rfloor \pmod n} C_{i \pmod r, j \pmod s}\end{aligned}$$

For the second one, given that $B \otimes C \in \mathbb{K}^{mr\times ns}$, we have

$$\begin{aligned}(A \otimes (B \otimes C))_{i,j} &= A_{\lfloor i/mr\rfloor,\lfloor j/ns\rfloor} (B \otimes C)_{i \pmod{mr}, j \pmod{ns}} \\ &= A_{\lfloor i/mr\rfloor,\lfloor j/ns\rfloor} B_{\lfloor (i \pmod{mr})/r\rfloor,\lfloor (j \pmod{ns})/s\rfloor} C_{(i \pmod{mr}) \pmod r,(j \pmod{ns}) \pmod s}. \\ &= A_{\lfloor i/rm\rfloor,\lfloor j/sn\rfloor} B_{\lfloor i/r\rfloor \pmod m,\lfloor j/s\rfloor \pmod n} C_{i \pmod r, j \pmod s}\end{aligned}$$

□

Proposition 3.2.7 (Permutation)
If $P \in \mathbb{K}^{p\times p}$ and $Q \in \mathbb{K}^{n\times n}$ are permutation matrices then their Kronecker product $P \otimes Q$ is a permutation matrix.

Proof. A permutation matrix is an identity matrix with permuted rows or columns, we choose to permute the columns. Then if P and Q represent respectively the permutation π and the permutation ρ, we have $P_{i,j} = \delta_{i,\pi(j)}$ and $Q_{i,j} = \delta_{i,\rho(j)}$.

Therefore

$$\begin{aligned}(P \otimes Q)_{i,j} = P_{\lfloor i/n\rfloor,\lfloor j/n\rfloor} Q_{i \pmod n, j \pmod n} &= \delta_{\lfloor i/n\rfloor,\pi(\lfloor j/n\rfloor)} \delta_{i \pmod n,\rho(j \pmod n)} \\ &= \begin{cases} 1 & \text{if } \begin{array}{c} \lfloor i/n\rfloor = \pi(\lfloor j/n\rfloor) \\ i \pmod n = \rho(j \pmod n) \end{array} \\ 0 & \text{otherwise} \end{cases}.\end{aligned}$$

It suffices to show that, once a column index j is fixed, there exists only one index i that satisfies both conditions.

This is true because fixing j is equivalent to fixing the values of $\lfloor i/n\rfloor$ and $i \pmod n$ that are respectively quotient and remainder of the euclidean division of i by n, that implies the uniqueness of $i = \lfloor i/n\rfloor n + i \pmod n$. □

3.2.2 Framework Mixed Radix (Cooley-Tukey)

Let $n \in \mathbb{Z}^+$ be a composite number and let $n = n_1 n_2 \cdots n_l$ be one of its factorizations.

To prove the general mechanism, we start by considering the factorizations of n in two factors, $n = n_1 n_2$ with $1 < n_1, n_2 < n$.

Given the fact that the set of Fourier nodes is a cyclic group $\langle \omega_n \rangle < \mathbb{K}^*$, the FFT can be interpreted using group theory [MR04]. A direct consequence is that the DFT matrix can be factorized in a product of structured and sparse matrices that depend on smaller DFT matrices [VL92, p. 76].

Theorem 3.2.8 (Radix Splitting)
Let $n = n_1 n_2 \in \mathbb{Z}^+$ and $\mathbb{K}$ be a field such that there exists a primitive n-th root of unity $\omega_n \in \mathbb{K}$. The DFT matrix V_{ω_n} can be factorized in the following way

$$V_{\omega_n} = (V_{\omega_{n_1}} \otimes I_{n_2}) D_{n_1,n_2} (I_{n_1} \otimes V_{\omega_{n_2}}) P_{n_1,n}$$

where D_{n_1,n_2} is a diagonal matrix with elements that are called twiddle factors *and $P_{n_1,n}$ is a permutation matrix known as* index-reversal.

Proof. We start with

$$\mathbf{y} = V_{\omega_n} \mathbf{a}$$

and analyze the elements

$$y_i = \sum_{j=0}^{n-1} a_j \omega_n^{ij} \quad \text{for each } i \in \{0, 1, \ldots, n-1\}.$$

The idea is to not iterate through the time index j using its natural order, but to use instead the order given by the partition of the cyclic group $\langle \omega_n \rangle$ into n_1 cosets.

Given that $n = n_1 n_2$, there is a unique subgroup $\langle \omega_n \rangle$ of order n_2, that is

$$\langle \omega_n^{n_1} \rangle = \{1_K, \omega_n^{n_1}, \omega_n^{2n_1}, \ldots, \omega_n^{(n_2-1)n_1}\} = \{\omega_n^{j_2 n_1} \text{ for } j_2 \in \{0, \ldots, n_2 - 1\}\}.$$

Therefore we have the following coset partition

$$\langle \omega_n \rangle = \bigsqcup_{j_1=0}^{n_1-1} \omega_n^{j_1} \langle \omega_n^{n_1} \rangle = \{\omega_n^{j_1} \omega_n^{j_2 n1} \text{ for } j_1 \in \{0, \ldots, n_1 - 1\} \text{ and } j_2 \in \{0, \ldots, n_2 - 1\}\}$$

that implies

$$\langle \omega_n \rangle = \{\omega_n^j \text{ for } j \in \{0, \ldots, n-1\}\} = \{\omega_n^{j_1 + j_2 n_1} \text{ for } j_1 \in \{0, \ldots, n_1 - 1\} \text{ and } j_2 \in \{0, \ldots, n_2 - 1\}\}$$

where, by equating the indices $j = j_1 + j_2 n_1$, we have that j_1 and j_2 are respectively remainder and quotient of the division of j by n_1, that is $j_1 = j \mod n_1$ and $j_2 = \lfloor j / n_1 \rfloor$.

The coset partition suggests to sum first by coset and then by the element inside the coset, that is

$$y_i = \sum_{j=0}^{n-1} a_j \omega_n^{ij} = \sum_{j_1=0}^{n_1-1} \sum_{j_2=0}^{n_2-1} a_{j_1 + j_2 n_1} \omega_n^{i(j_1 + j_2 n_1)} \quad \text{for each } i \in \{0, 1, \ldots, n-1\}.$$

Isolating the terms that depend only on j_1 we have

$$y_i = \sum_{j_1=0}^{n_1-1} \omega_n^{ij_1} \left(\sum_{j_2=0}^{n_2-1} a_{j_1 + j_2 n_1} (\omega_n^{n_1})^{ij_2} \right) \quad \text{for each } i \in \{0, 1, \ldots, n-1\}.$$

Given that $\omega_n^{n_1}$ has order n_2 we have that it is a primitive n_2-root of unity so we write $\omega_n^{n_1} = \omega_{n_2}$ that implies

$$y_i = \sum_{j_1=0}^{n_1-1} \omega_n^{ij_1} \left(\sum_{j_2=0}^{n_2-1} a_{j_1 + j_2 n_1} \omega_{n_2}^{ij_2} \right) \quad \text{for each } i \in \{0, 1, \ldots, n-1\}.$$

Applying the same logic to the frequency index i, this time with a subgroup of order n_1, we get

$$\langle \omega_n \rangle = \{\omega_n^{i_1 n_2 + i_2} \text{ for } i_1 \in \{0, \dots, n_1 - 1\} \text{ and } i_2 \in \{0, \dots, n_2 - 1\}\}$$

therefore

$$y_{i_1 n_2 + i_2} = \sum_{j_1=0}^{n_1-1} (\omega_n^{n_2})^{i_1 j_1} \omega_n^{i_2 j_1} \left(\sum_{j_2=0}^{n_2-1} a_{j_1 + j_2 n_1} \omega_{n_2}^{(i_1 n_2 + i_2) j_2} \right) \quad \text{for } i_1 \in \{0, \dots, n_1 - 1\} \text{ and } i_2 \in \{0, \dots, n_2 - 1\},$$

and using again the multiplicative order characterization we get

$$y_{i_1 n_2 + i_2} = \underbrace{\sum_{j_1=0}^{n_1-1} \omega_{n_1}^{i_1 j_1} \left[\underbrace{\omega_n^{i_2 j_1}}_{\text{Twiddle Factor}} \cdot \underbrace{\left(\sum_{j_2=0}^{n_2-1} a_{j_1 + j_2 n_1} \omega_{n_2}^{i_2 j_2} \right)}_{\text{inner DFT with } V_{\omega_{n_2}}} \right]}_{\text{outer DFT with } V_{\omega_{n_1}}}.$$

We can now derive the matrix form starting from the innermost operation.

We start with the *divide* part of the algorithm. If we want to use the vector $\mathbf{a}$ we first need to reorder its elements. That's because the sum takes place on the vector $\mathbf{a}' = P_{n_1,n}\mathbf{a}$ that is $\mathbf{a}$ reordered using the coset order

$$\begin{aligned} \mathbf{a}' &= \left([a'_0, a'_1, \dots, a'_{n_2-1}], [a'_{n_2}, a'_{n_2+1}, \dots, a'_{2n_2-1}], \dots, [a'_{(n_1-1)n_2}, \dots, a'_{n_1 n_2 - 1}] \right)^{\mathrm{T}} \\ &= \left([a_0, a_{n_1}, \dots, a_{(n_2-1)n_1}], [a_1, a_{1+n_1}, \dots, a_{1+(n_2-1)n_1}], \dots, [a_{n_1-1}, \dots, a_{n_1-1+(n_2-1)n_1}] \right)^{\mathrm{T}}. \end{aligned}$$

So the permutation rule is that the j-th element of $\mathbf{a}$ is sent in the position corresponding to the index $(j \pmod{n_1}) n_2 + \lfloor j/n_1 \rfloor$ in $\mathbf{a}'$.

This map is called *index-reversal permutation*, because it reverses the role of j_1 and j_2 in the index representation.

A permutation matrix can then be obtained permuting the columns of an identity matrix, we use

$$(P_{n_1,n})_{i,j} = \delta_{i,(j \pmod{n_1}) n_2 + \lfloor j/n_1 \rfloor}.$$

We can now proceed with the *impera* part where we solve the subproblems, each block of lenght n_2 of $\mathbf{a}'$ must be multiplied by the DFT matrix $V_{\omega_{n_2}}$.

This is the product with a $n \times n$ diagonal block matrix, where each block on the diagonal is $V_{\omega_{n_2}}$. Using the Kronecker product, this matrix can be written as $I_{n_1} \otimes V_{\omega_{n_2}}$ and has the following structure

$$I_{n_1} \otimes V_{\omega_{n_2}} = \begin{pmatrix} V_{\omega_{n_2}} & \mathbf{0} & \cdots & \mathbf{0} \\ \mathbf{0} & V_{\omega_{n_2}} & \cdots & \mathbf{0} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{0} & \mathbf{0} & \cdots & V_{\omega_{n_2}} \end{pmatrix}.$$

The next step is multiplying by the twiddle factors, that is multiplying by a $n \times n$ diagonal block matrix with the following $n_2 \times n_2$ diagonal blocks

$$D_{n_1,n_2} = \begin{pmatrix} D_0 & \mathbf{0} & \cdots & \mathbf{0} \\ \mathbf{0} & D_1 & \cdots & \mathbf{0} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{0} & \mathbf{0} & \cdots & D_{n_1-1} \end{pmatrix} \quad \text{where} \quad D_{j_1} = \operatorname{diag}(1_{\mathbb{K}}, \omega_n^{j_1}, \dots, \omega_n^{(n_2-1)\cdot j_1}).$$

After multiplying by the twiddle factors we still have a vector divided in blocks of size n_2 and in permuted order. However, the outer DFT needs to be applied on the elements in the correct order.

So for each $i_2 \in \{0, \dots, n_2 - 1\}$, that is the positon index inside a block, we need to apply a DFT of dimension $n_1 \times n_1$ on the elements that are in position i_2 of each block.

Using again the Kronecker product, this operation corresponds to the matrix $V_{\omega_{n_1}} \otimes I_{n_2}$, that has the following block structure

$$V_{\omega_{n_1}} \otimes I_{n_2} = \begin{pmatrix} (V_{\omega_{n_1}})_{0,0} I_{n_2} & \cdots & (V_{\omega_{n_1}})_{0,n_1-1} I_{n_2} \\ \vdots & \ddots & \vdots \\ (V_{\omega_{n_1}})_{n_1-1,0} I_{n_2} & \cdots & (V_{\omega_{n_1}})_{n_1-1,n_1-1} I_{n_2} \end{pmatrix}$$

where each block is a $n_2 \times n_2$ diagonal matrix because it is a multiple of I_{n_2}.

Combining those structured and sparse matrices we obtain the V_{ω_n} factorization. □

If n has factorization $n = n_1 n_2 \cdots n_l$, we can recursively apply Theorem 3.2.8 and obtain a recursive algorithm.

Remark 3.2.9 (Decimation in Time (DIT) and Decimation in Frequency (DIF))
We derived an algorithm of the Cooley-Tukey *family. Because we have chosen to first decompose the temporal index j, this variant is called* Decimation in Time (DIT)*. In this version we do a permutation on the input and obtain the output in natural order.*

If we first decompose the frequency index i, we obtain another variant called Decimation in Frequency (DIF)*. In this version we take the input in natural order and do a permutation on the output [Bin, p. 8],[VL92, p. 64].*

To obtain an iterative algorithm, we prove that there exists a complete factorization for the DFT matrix V_{ω_n} as seen in [VL92, pp. 95-96].

Corollary 3.2.10 (Complete DFT Matrix Factorization)
Let $n = n_1 n_2 \cdots n_l$ be a factorization of $n \in \mathbb{Z}^+$ and let $\mathbb{K}$ be a field such that there exists a primitive n-th root of unity $\omega_n \in \mathbb{K}$. The DFT matrix V_{ω_n} can be factorized as

$$V_{\omega_n} = A_l A_{l-1} \cdots A_1 P_n,$$

where P_n is a permutation matrix and

$$A_i = \begin{cases} I_{n_1 \cdots n_{l-1}} \otimes V_{\omega_{n_l}} & \text{if } i = 1 \\ I_{n_1 \cdots n_{l-i}} \otimes \left[(V_{\omega_{n_{l-i+1}}} \otimes I_{n_{l-i+2} \cdots n_l}) \cdot D_{n_{l-i+1},(n_{l-i+2} \cdots n_l)} \right] & \text{if } 2 \le i \le l-1 \\ (V_{\omega_{n_1}} \otimes I_{n_2 \cdots n_l}) \cdot D_{n_1,(n_2 \cdots n_l)} & \text{if } i = l \end{cases}$$

Proof. The proof is by induction on the number of factors l

- *Base Case ($l = 2$):* is the Theorem 3.2.8. This is because for $l = 2$ we have

$$A_i = \begin{cases} I_{n_1} \otimes V_{\omega_{n_2}} & \text{if } i = 1 \\ (V_{\omega_{n_1}} \otimes I_{n_2}) \cdot D_{n_1,n_2} & \text{if } i = 2 \end{cases}$$

- *Inductive Step:* Suppose that for $n' = n_2 n_3 \cdots n_l$ we have $V_{\omega_{n'}} = A'_{l-1} A'_{l-2} \cdots A'_1 P_{n'}$ with (it suffices to change the indices to start from n_2)

$$A'_i = \begin{cases} I_{n_2 \cdots n_{l-1}} \otimes V_{\omega_{n_l}} & \text{if } i = 1 \\ I_{n_2 \cdots n_{l-i}} \otimes \left[(V_{\omega_{n_{l-i+1}}} \otimes I_{n_{l-i+2} \cdots n_l}) \cdot D_{n_{l-i+1},(n_{l-i+2} \cdots n_l)} \right] & \text{if } 2 \leq i \leq l-2 \\ (V_{\omega_{n_2}} \otimes I_{n_3 \cdots n_l}) \cdot D_{n_2,(n_3 \cdots n_l)} & \text{if } i = l-1 \end{cases}$$

and apply the base case to $n = n_1 n'$ then

$$\begin{aligned} V_{\omega_n} &= (V_{\omega_{n_1}} \otimes I_{n'}) D_{n_1,n'} (I_{n_1} \otimes V_{\omega_{n'}}) P_{n_1,n} \\ &= (V_{\omega_{n_1}} \otimes I_{n'}) D_{n_1,n'} (I_{n_1} \otimes [A'_{l-1} A'_{l-2} \cdots A'_1 P_{n'}]) P_{n_1,n} \quad \text{(using the Corollary 3.2.5)} \\ &= (V_{\omega_{n_1}} \otimes I_{n'}) D_{n_1,n'} (I_{n_1} \otimes A'_{l-1})(I_{n_1} \otimes A'_{l-2}) \cdots (I_{n_1} \otimes A'_1)(I_{n_1} \otimes P_{n'}) P_{n_1,n} \end{aligned}$$

We can now take $P_n = (I_{n_1} \otimes P_{n'}) P_{n_1,n}$ that for Proposition 3.2.7 is again a permutation matrix and we point out that if we take

$$A_i = \begin{cases} I_{n_1} \otimes A'_i & \text{if } i \leq l-1 \\ (V_{\omega_{n_1}} \otimes I_{n'}) D_{n_1,n'} & \text{if } i = l \end{cases}$$

$$= \begin{cases} I_{n_1} \otimes \left[I_{n_2 \cdots n_{l-1}} \otimes V_{\omega_{n_l}} \right] & \text{if } i = 1 \\ I_{n_1} \otimes \left[I_{n_2 \cdots n_{l-i}} \otimes \left[(V_{\omega_{n_{l-i+1}}} \otimes I_{n_{l-i+2} \cdots n_l}) \cdot D_{n_{l-i+1},(n_{l-i+2} \cdots n_l)} \right] \right] & \text{if } 2 \leq i \leq l-2 \\ I_{n_1} \otimes \left[(V_{\omega_{n_2}} \otimes I_{n_3 \cdots n_l}) \cdot D_{n_2,(n_3 \cdots n_l)} \right] & \text{if } i = l-1 \\ (V_{\omega_{n_1}} \otimes I_{n'}) D_{n_1,n'} & \text{if } i = l \end{cases}$$

and using the associative property from Proposition 3.2.6 we get

$$A_i = \begin{cases} I_{n_1 \cdots n_{l-1}} \otimes V_{\omega_{n_l}} & \text{if } i = 1 \\ I_{n_1 \cdots n_{l-i}} \otimes \left[(V_{\omega_{n_{l-i+1}}} \otimes I_{n_{l-i+2} \cdots n_l}) \cdot D_{n_{l-i+1},(n_{l-i+2} \cdots n_l)} \right] & \text{if } 2 \leq i \leq l-2 \\ I_{n_1} \otimes \left[(V_{\omega_{n_2}} \otimes I_{n_3 \cdots n_l}) \cdot D_{n_2,(n_3 \cdots n_l)} \right] & \text{if } i = l-1 \\ (V_{\omega_{n_1}} \otimes I_{n'}) D_{n_1,n'} & \text{if } i = l \end{cases}$$

that is the conclusion. □

Remark 3.2.11 (Computing the IDFT)
The results we have proved do not depend on the particular choice of the primitive n-th root of unity so they are valid to compute the IDFT aswell.

3.2.3 Time Complexity

We can now derive the time complexity following the reasoning of [Bin, pp. 6-7]

Proposition 3.2.12 (Mixed Radix Time Complexity)
Let $n \in \mathbb{Z}^+$ and let $\mathbb{K}$ be a field such that there exists a primitive n-th root of unity $\omega_n \in \mathbb{K}$. If $n = n_1 n_2 \cdots n_l$, the time complexity to compute the DFT using the FFT Mixed Radix algorithm is

$$c(n) = O\left(n \sum_{i=1}^{l} n_i\right)$$

Proof. We use the recursive algorithm

$$y_{i_1 \frac{n}{n_1} + i_2} = \underbrace{\sum_{j_1=0}^{n_1-1} \omega_{n_1}^{i_1 j_1} \left[\underbrace{\omega_n^{i_2 j_1}}_{\text{Twiddle Factor}} \cdot \underbrace{\left(\sum_{j_2=0}^{\frac{n}{n_1}-1} a_{j_1 + j_2 n_1} \omega_{n/n_1}^{i_2 j_2} \right)}_{\text{inner DFT with } V_{\omega_{n/n_1}}} \right]}_{\text{outer DFT with } V_{\omega_{n_1}}}.$$

We have to compute

- n_1 DFTs of order n/n_1;
- after each of the n_1 inner DFTs we multiply by the twiddle factor and we do that for each of the n/n_1 indices i_2, obtaining a total of n multiplications;
- n/n_1 DFTs of order n_1.

If we denote by $c(n)$ the total computational cost and with M the multiplication computational cost, we obtain the following recursion relation

$$c(n) = n_1 c\left(\frac{n}{n_1}\right) + nM + \frac{n}{n_1} c\left(n_1\right).$$

Now, dividing by n, we obtain

$$\frac{c(n)}{n} = \frac{n_1}{n} c\left(\frac{n}{n_1}\right) + M + \frac{c\left(n_1\right)}{n_1}$$

and then

$$\begin{aligned}
\frac{c(n)}{n} &= \frac{c(n_1)}{n_1} + \frac{c(n_2 \cdots n_l)}{n_2 \cdots n_l} + M \\
&= \frac{c(n_1)}{n_1} + \left(\frac{c(n_2)}{n_2} + \frac{c(n_3 \cdots n_l)}{n_3 \cdots n_l} + M \right) + M \\
&= \frac{c(n_1)}{n_1} + \frac{c(n_2)}{n_2} + \frac{c(n_3 \cdots n_l)}{n_3 \cdots n_l} + 2M \\
&= \cdots \\
&= \sum_{i=1}^{l} \frac{c(n_i)}{n_i} + (l-1)M.
\end{aligned}$$

Substituting the base DFT cost $c(n_i) = O(n_i^2)$ we obtain

$$c(n) = n\sum_{i=1}^{l} O(n_i) + n(l-1)M = O\left(n\sum_{i=1}^{l} n_i\right)$$

□

In the particular case in which the factorization of n is a power of an integer r the algorithm is called Radix-r.

Corollary 3.2.13 (Radix-r Time Complexity)
Let $n \in \mathbb{Z}^+$ and let $\mathbb{K}$ be a field such that there exists a primitive n-th root of unity $\omega_n \in \mathbb{K}$.

If $n = r^k$, the computational cost for computing the DFT using the FFT Mixed Radix algorithm is

$$c(n) = O\left(nr\log_r n\right)$$

Proof. Using Proposition 3.2.12 and the fact that $k = \log_r n$ we obtain

$$c(n) = O\left(n\sum_{i=1}^{k} r\right) = O\left(nkr\right) = O\left(nr\log_r n\right)$$

□

The minimum possible complexity is reached in the Radix-2 framework with $n = 2^k$, in this framework we have time complexity $O(2n\log_2 n) = O(n\log_2 n)$.

3.3 FFTs in STARK protocols

To maximize efficiency, STARK protocols use the Radix-2 framework to interpolate the trace polynomials. This choice imposes to extend the lenght $T+1$ of the execution trace by *padding* it with dummy rows until we reach the nearest power of two $n = 2^k$ such that $n \geq T$.

3.3.1 Finite Field Choice

After padding, the execution trace has lenght $n = 2^k$ and that has direct consequences on the finite field choice used in the arithmetization. In fact, the following conditions must hold:

1. **Adequate Cardinality:** To interpolate an execution trace of T steps, that as been padded to lenght $n = 2^k > T$, the cardinality $|\mathbb{F}|$ must be greater than n to contain the trace domain H. In real implementations $|\mathbb{F}| \gg n$ is chosen for security and efficiency as we will see for example in Chapter 4.
2. **Primitive n-th Root of Unity:** To use an FFT of dimension $n = 2^k$, the field $\mathbb{F}$ must contain a primitive n-th root of unity. By Proposition 3.1.10 we have that n must divide $|\mathbb{F}| - 1$.

Those requirements allow us to directly exclude multiple types of finite fields.

A first consequence is the incompatibility with fields of characteristic 2.

Corollary 3.3.1 (Incompatibility with Fields of Characteristic 2)
Let $\mathbb{K}$ be a field with char$(\mathbb{K}) = 2$ and let $n = 2^k$ with $k \geq 1$. Then $\mathbb{K}$ does not contain any primitive n-th root of unity.

Proof. If $\mathbb{K} = \mathbb{F}_{2^m}$, the necessary condition for the existence of the primitive n-th root of unity is that $n \mid (2^m - 1)$. Given that $n = 2^k$ is a power of two and $2^m - 1$ is odd, this condition can't be satisfied for $k \geq 1$. □

Although it is theoretically possible to use extension fields $\mathbb{F}_{q^m}$ where $q \neq 2$ is a prime, as observed in Remark 2.3.2 we prefer to use the same prime field $\mathbb{F}_q$ with $q \neq 2$ for modeling the computation and for the arithmetization process, to avoid field extensions.

Therefore the choice is restricted to prime fields $\mathbb{F}_q$ that are FFT-Friendly.

Definition 3.3.2 (FFT-Friendly Field)
We call FFT-friendly *a prime field $\mathbb{F}_q$ where $q - 1$ can be factorized as*

$$q - 1 = c \cdot 2^s,$$

where $s \in \mathbb{Z}^+$ and c is a odd cofactor.

Remark 3.3.3 (Greatest Supported FFT)
The factor 2^s defines the maximum dimension for an FFT Radix-2 in the field. The condition to execute an FFT of dimension $n = 2^k$ is then that the exponent k is less or equal than the exponent s supported by the field, so we must have $k \leq s$.

To show how this property is fundamental in practical implementations we take the fields introduced in Table 2.2.15 and highlight their FFT-friendliness.

Field Name	$q-1$ **Decomposition**	**Greatest** $n=2^s$	**Implementation**
Baby Bear	$15 \cdot 2^{27}$	2^{27}	RISC Zero [BGt23] Plonky3 [The22]
KoalaBear	$127 \cdot 2^{24}$	2^{24}	Plonky3 [The22]
Rescue	$(2^{27}+5) \cdot 2^{34}$	2^{34}	ethSTARK [Tea21, p. 5]
f62	$(2^{23}-111) \cdot 2^{39}$	2^{39}	Winterfell [Fac21]
Goldilocks	$(2^{32}-1) \cdot 2^{32}$	2^{32}	Plonky3 [The22] Winterfell [Fac21]
f128	$(2^{88}-45) \cdot 2^{40}$	2^{40}	Winterfell [Fac21]
Rescue-Prime	$407 \cdot 2^{119}$	2^{119}	Anatomy of a STARK [Sze]
STARK	$(2^{59}+17) \cdot 2^{192}$	2^{192}	Starknet [Sta]

Table 3.1: Finite fields used across STARK projects and their FFT-friendliness

Remark 3.3.4 (Circle FFTs)
We excluded all fields that are not FFT-Friendly including the Mersenne prime field M31 with $q = 2^{31}-1$ [p. 3][HLP24]. Recent developments introduced an FFT variant called Circle FFT *[HLP24, p. 12] that utilizes the points on the circle curve as group structure, allowing efficient DFT computation even on fields of cardinality q such that $q+1$ is divisible for a great power of 2.*

3.3.2 Padding The Execution Trace

We now show how to pad the execution trace to obtain a new AIR and that this is a valid extension, in the sense of Definition 2.4.9.

The idea is that the new constraint polynomials must constrain the same logic. To ensure that, after the execution trace padding, we need to modify the constraint polynomials evaluation domains.

Given an AIR

$$\mathcal{AIR}_1 = (\mathbb{F}, T, w, H, \sigma(X), \mathcal{P}, \mathcal{P}_{\boldsymbol{\pi}}, \mathcal{B})$$

if we choose $n \geq T+1$ as the new execution trace lenght and

$$H' = \{h'_0, \dots, h'_T, h'_{T+1}, \dots, h'_{n-1}\}$$

as the new trace domain, we consider the injective function

$$\begin{aligned} f\colon H &\to H' \\ h_t &\mapsto h'_t \end{aligned}$$

and define the new constraint polynomials as follows

- if $\mathcal{P} = \{(P_1, H_{P_1}), \dots, (P_s, H_{P_s})\}$ then $\mathcal{P}' = \{(P_1, f(H_{P_1})), \dots, (P_s, f(H_{P_s}))\}$;
- if $\mathcal{P}_{\boldsymbol{\pi}} = \{(P_{\pi_1}, H_{P_{\pi_1}}), \dots, (P_{\pi_r}, H_{P_{\pi_r}})\}$ then $\mathcal{P}'_{\boldsymbol{\pi}} = \{(P_{\pi_1}, f(H_{P_{\pi_1}})), \dots, (P_{\pi_r}, f(H_{P_{\pi_r}}))\}$;

- if $\mathcal{B} = \{(B_1, H_{B_1}), \dots, (B_m, H_{B_m})\}$ then $\mathcal{B}' = \{(B_1, f(H_{B_1})), \dots, (B_m, f(H_{B_m}))\}$.

The new AIR is

$$\mathcal{AIR}_2 = (\mathbb{F}, n-1, w, H', \sigma'(X), \mathcal{P}', \mathcal{P}'_{\boldsymbol{\pi}}, \mathcal{B}'),$$

where $\sigma'(X)$ is the shift polynomial associated with the new trace domain H'.

Proposition 3.3.5 (Padding Extension Validity)
The AIR $\mathcal{AIR}_2$ obtained through padding is a valid extension of the original $\mathcal{AIR}_1$, in the sense of Definition 2.4.9. That is

$$\mathcal{AIR}_1 \preceq \mathcal{AIR}_2.$$

Proof. We consider a candidate discrete witness $(x, p) \in \mathbb{F}^{(T+1)\times w} \times H^{(T+1)\times r}$ and we build the extension function Ext.

To bring the execution trace length to n, we fix a padding matrix $P_{pad,x} \in \mathbb{F}^{(n-(T+1))\times w}$ and take

$$x' = \begin{pmatrix} x \\ P_{pad,x} \end{pmatrix} \in \mathbb{F}^{n\times w},$$

that in matrix form is

$$\begin{pmatrix} x_1(0) & \cdots & x_w(0) \\ \vdots & \ddots & \vdots \\ x_1(T) & \cdots & x_w(T) \end{pmatrix} \quad \mapsto \quad \left(\begin{array}{ccc} x_1(0) & \cdots & x_w(0) \\ \vdots & \ddots & \vdots \\ x_1(T) & \cdots & x_w(T) \\ \hdashline (P_{pad,x})_{1,1} & \cdots & (P_{pad,x})_{1,w} \\ \vdots & \ddots & \vdots \\ (P_{pad,x})_{n-(T+1),1} & \cdots & (P_{pad,x})_{n-(T+1),w} \end{array}\right)$$

For the permutation trace we consider

$$\begin{aligned} F\colon H^{(T+1)\times r} &\to H'^{(T+1)\times r} \\ \begin{pmatrix} p_{1,1} & \cdots & p_{1,r} \\ \vdots & \ddots & \vdots \\ p_{T+1,1} & \cdots & p_{T+1,r} \end{pmatrix} &\mapsto \begin{pmatrix} f(p_{1,1}) & \cdots & f(p_{1,r}) \\ \vdots & \ddots & \vdots \\ f(p_{T+1,1}) & \cdots & f(p_{T+1,r}) \end{pmatrix}. \end{aligned}$$

and in a similar way we add the padding matrix $P_{pad,p} \in H'^{(n-(T+1))\times r}$ such that

$$P_{pad,p} = \begin{pmatrix} h'_{T+1} & \cdots & h'_{T+1} \\ \vdots & \ddots & \vdots \\ h'_{n-1} & \cdots & h'_{n-1} \end{pmatrix}$$

that is

$$p' = \begin{pmatrix} F(p) \\ P_{pad,p} \end{pmatrix} \in H'^{n\times r}.$$

This way the polynomials $\pi'_k : H' \to H'$ that interpolate the columns over the trace domain H' are again a bijection.

The extension function is then

$$\begin{aligned} \text{Ext}\colon \mathbb{F}^{(T+1)\times w} \times H^{(T+1)\times r} &\to \mathbb{F}^{n\times w} \times H'^{n\times r} \\ (x, p) &\mapsto (x', p') = \left(\begin{pmatrix} x \\ P_{pad,x} \end{pmatrix}, \begin{pmatrix} F(p) \\ P_{pad,p} \end{pmatrix}\right). \end{aligned}$$

We prove that the properties of a valid extension hold:

- **Information Preservation:** $x'_{[0:T;1:w]} = x$ by construction.
- **Validity Equivalence:** We show that $x \in \mathcal{T}_{\mathcal{AIR}_1} \iff \text{Ext}(x) \in \mathcal{T}_{\mathcal{AIR}_2}$ proving the two implications

  $\Longrightarrow$ let $(x, p) \in \mathcal{T}_{\mathcal{AIR}_1}$ and let $(\mathbf{W}', \boldsymbol{\pi}')$ be the vector of polynomials obtained by interpolating the columns of $(x', p') = \text{Ext}(x, p)$ over H'.
  By construction we have for each $t \in \{0, \dots, T\}$ that

  $$W'(f(h_t)) = W'(h'_t) = x'(t) = x(t) = W(h_t)$$

  and because for each $t \in \{0, \dots, T\}$ we have $\pi'_k(h'_t) = f(\pi_k(h_t))$ we obtain

  $$W'(\pi'_k(h'_t)) = W'(f(\pi_k(h_t))) = W(\pi_k(h_t)).$$

  Therefore the new AIR constraints are satisfied

  $P_i(W'(h'_t), W'(\sigma'(h'_t)))) = P_i(W(h_t), W(\sigma(h_t))) = 0$ for each $h'_t \in f(H_{P_i})$;
  $P_{\pi_i}(W'(h'_t), W'(\pi'_i(h'_t))) = P_{\pi_i}(W(h_t), W(\pi_i(h_t))) = 0$ for each $h'_t \in f(H_{P_{\pi_i}})$;
  $B_i(W'(h'_t)) = B_i(W(h_t)) = 0$ for each $h'_t \in f(H_{B_i})$.

  that implies $(\mathbf{W}', \boldsymbol{\pi}') \in \mathcal{W}_{\mathcal{AIR}_2}$ and so $(x', p') \in \mathcal{T}_{\mathcal{AIR}_2}$.

  $\Longleftarrow$ it suffices to prove the contrapositive, let $(x, p) \notin \mathcal{T}_{\mathcal{AIR}_1}$ and let $(\mathbf{W}', \boldsymbol{\pi}')$ be the vector of polynomials obtained by interpolating the columns of the new discrete witness $(x', p') = \text{Ext}(x, p)$ over H'.
  Because $(x, p) \notin \mathcal{T}_{\mathcal{AIR}_1}$, at least one of the constraint specifications in $\mathcal{AIR}_1$ is not satisfied, without loss of generality we suppose that

  $$P_i(W(h_t), W(\sigma(h_t))) \neq 0$$

  for some $h_t \in H_{P_i}$, again we have by contruction that for each $t \in \{0, \dots, T\}$ we have

  $$W'(f(h_t)) = W'(h'_t) = x'(t) = x(t) = W(h_t)$$

  so we get $P_i(W'(h'_t), W'(\sigma'(h'_t)))) = P_i(W(h_t), W(\sigma(h_t))) = 0$ for each node $h'_t = f(h_t) \in f(H_{P_i})$.
  Therefore $(\mathbf{W}', \boldsymbol{\pi}') \notin \mathcal{W}_{\mathcal{AIR}_2}$ and this implies $(x', p') \notin \mathcal{T}_{\mathcal{AIR}_2}$.

□

Remark 3.3.6 (Zero-Knowledge)
The padding is randomly chosen by the Prover, this allows to mask the witness because the new trace polynomials will now depend on random values.

In the following chapters we will suppose that the trace is already padded to have $T = 2^k - 1$.

Chapter 4

The Degree Problem

In this chapter we will analyze the main problems of computational efficiency that are linked to the degree of check polynomials.

As a first step we will analyze the case of non-local constraint polynomials, using permutation arguments. To do that, we first need to introduce a vector commitment scheme.

4.1 Merkle Trees

To guarantee the data integrity for the data used by the Prover without revealing the data itself, we use a commitment scheme based on hash functions and Merkle trees.

We define hash functions and their desired properties following [MvOV96, pp. 321-324] and [TW20, p. 226-227]

Definition 4.1.1 (Hash Function)
Let $A = \{a \in \{0,1\}^j \mid j \in \mathbb{N}\}$ and $B = \{0,1\}^k$ for a fixed $k \in \mathbb{N}$, a function $\mathcal{H}\colon A \to B$ is a hash function *if for each $a \in A$ the image $\mathcal{H}(a)$ is computable in polynomial time in the length of a.*

Remark 4.1.2 (Input Concatenation)
Given two elements $a_1, a_2 \in A$ such that $a_1 \in \{0,1\}^{j_1}$ and $a_2 \in \{0,1\}^{j_2}$ for some $j_1, j_2 \in \mathbb{N}$, the pair $(a_1, a_2) \in \{0,1\}^{j_1+j_2}$ is still an element of A, this allows us to compute the hash function even on input concatenations.

We now list the three desired properties for a hash function.

Definition 4.1.3 (Pre-Image Resistant)
A hash function $\mathcal{H}\colon A \to B$ is Pre-Image Resistant *if for each $b \in \mathcal{H}(A)$ it is computationally infeasible to find $a \in A$ such that $\mathcal{H}(a) = b$.*

Definition 4.1.4 (Second Pre-Image Resistant)
A hash function $\mathcal{H}\colon A \to B$ is Second Pre-Image Resistant *if for each $a \in A$ it is computationally infeasible to find $a' \in A$ such that $a \neq a'$ and $\mathcal{H}(a) = \mathcal{H}(a')$.*

Definition 4.1.5 (Collision Resistant)
A hash function $\mathcal{H}\colon A \to B$ is Collision Resistant *if it is computationally infeasible to find $a, a' \in A$ such that $a \neq a'$ and $\mathcal{H}(a) = \mathcal{H}(a')$.*

We now give the definition of Merkle tree [KL14, p. 183] but using level and position indexing to highlight the recursive nature of the definition.

Definition 4.1.6 (Merkle Tree)
Let $(a_1, \ldots, a_{2^k}) \in A^{2^k}$ with $k \in \mathbb{N}$ and let $\mathcal{H}\colon A \to B$ be a hash function.

A Merkle tree *is a perfect binary tree where the node at level $l \in \{0, \cdots, k\}$ and position $p \in \{1, \cdots, 2^{k-l}\}$ is defined as*

$$m_{l,p} := \begin{cases} \mathcal{H}(a_p) & \text{if } l = 0 \\ \mathcal{H}(m_{l-1,2p-1}, m_{l-1,2p}) & \text{if } l > 0 \end{cases}.$$

The element $m_{k,1}$ is called Merkle root.

Remark 4.1.7 (Cryptographic Notation)
In cryptographic literature [KL14, p. 183], another notation is used that immediately links the node with the data used to compute it, that is $m_{i\ldots j}$ where

$$\begin{aligned} i &= (p-1) \cdot 2^l + 1 \\ j &= p \cdot 2^l \end{aligned}.$$

This map between indices is easily derived because in a perfect binary tree each node at level l has 2^l leaves, if that node occupies position p there are $p-1$ other nodes before it and that means that the first leaf associated with the node is preceded by $(p-1) \cdot 2^l$ leaves, this gives $i = (p-1) \cdot 2^l + 1$.

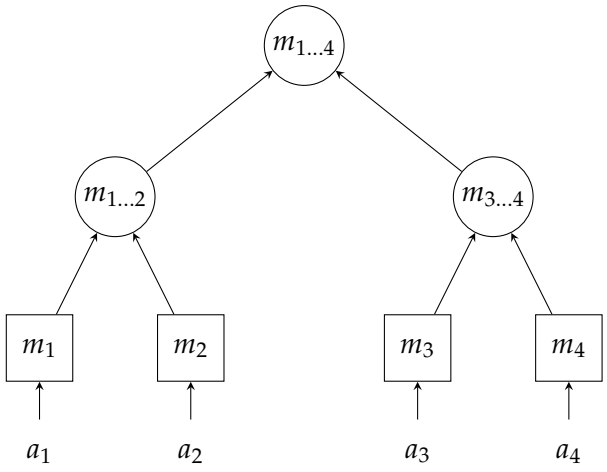


Figure 4.1: Merkle Tree structure.

Lets state an important Theorem found in [KL14, p. 184]

Theorem 4.1.8 (Collision Resistant Merkle Tree)
Let $\mathcal{H}$ be a collision resistant hash function, then the function

$$\begin{aligned} \mathcal{MT}_{k,\mathcal{H}} \colon A^{2^k} &\to B \\ (a_1, \ldots, a_{2^k}) &\mapsto m_{k,1} \end{aligned}$$

that computes the Merkle root of the Merkle tree built using the hash function $\mathcal{H}$ is collision resistant.

Definition 4.1.9 (Merkle Path)
Given a leaf a_i of the Merkle tree built using the data $(a_1, \ldots, a_{2^k}) \in A^{2^k}$ with $k \in \mathbb{N}$ and using the hash function $\mathcal{H} \colon A \to B$, the Merkle Path *is the set of $k-1$ sibling nodes along the path from the leaf to the root.*

Suppose that a Prover wants to commit to the whole data set $(a_1, \ldots, a_{2^k}) \in A^{2^k}$ without making the data public, he computes the Merkle root $m_{k,1}$ and makes it public.

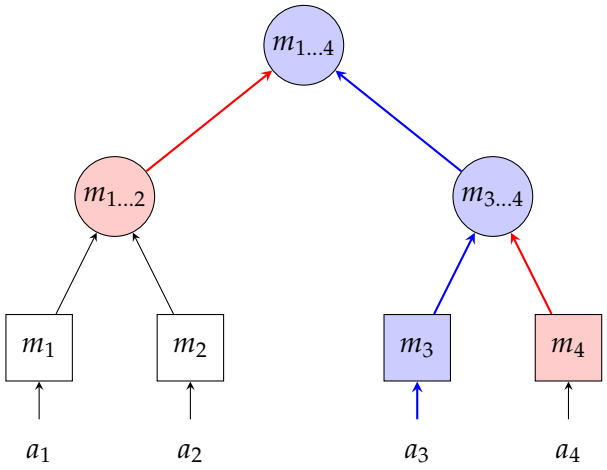


Figure 4.2: Direct path (blue) and Merkle path (red) for the data a_3.

If the Verifier wants to verify that the Prover has the data a_i he asks for the corresponding Merkle path and, if the path is correct, he can compute the root using $O(\log_2 2^k) = O(k)$ evaluations of the hash function.

Thanks to Theorem 4.1.8, if the hash function is collision resistant, the Prover can't change the value of the leaf a_i with a second value a_i' and find a Merkle path that allows to compute the same root, because this would imply finding a collision and that is computationally infeasible.

Remark 4.1.10 (Arbitrary Input Lenght)
When the input lenght is not a power of two, to obtain a perfect binary tree, we use again a padding strategy.

To minimize the number of hashes to compute, a strategy could be to check at each level if the number of nodes is even and, if that's not the case, to make a copy of the last node.

This is not safe because, as seen in [Ant23, p. 254], this implementation breaks the collision resistance property and that caused a critical vulnerability in the Bitcoin Core code, known as CVE-2012-2459 [Nat12].

In STARK protocols this in not a problem because the execution trace is already padded to the desired lenght, as we have seen in Section 3.3.2.

4.1.1 Committing to Multiple Polynomials

In the following sections the Prover will need to commit to multiple data sets of lenght 2^k in a single commitment. In particular, as seen in [AMGM24, p. 20], it is possible to commit to a vector of polynomials $\mathbf{f}$ evaluating them over an extended domain D and computing a single hash for all the evaluations over the same node, obtaining the leaves

$$m_{0,i} = \mathcal{H}(f_1(d_i), \dots, f_w(d_i)) \quad \text{for } d_i \in D.$$

Remark 4.1.11 (Low Degree Extension)
Each time we will talk about commitments, those will take place using evaluations over an extended domain D such that $D \cap H = \emptyset$ and $|D| = \beta(T+1)$ where β is a power of two. We will call the set of evaluations Low Degree Extension. *The motivations behind this particular choice of D will be made explicit in Section 5.3.1.*

Initially, we will assume the Prover is honest during the commitment phases, meaning they commit to the polynomials required by the protocol. We will conduct a full analysis in Section 5.3.3 after obtaining the soundness estimates for this base case.

4.2 Non-Local constraints

As seen in Example 2.4.7, the use of non-local constraints reduces the execution trace width and the number of constraints. However, this can lead to check polynomials

$$\text{Check}_{P_{\pi_k}}(X) = P_{\pi_k}(\mathbf{W}(X), \mathbf{W}(\pi_k(X)))$$

of high degree, because, being that $\deg \mathbf{W} \leq T$ and $\deg \boldsymbol{\pi} \leq T$, we have

$$\deg \text{Check}_{P_{\pi_k}}(X) \leq \deg(P_{\pi_k}) \cdot T^2.$$

To tackle this problem we avoid the direct use of the permutation inside the check polynomials, this is done extending the trace with auxiliary columns that will contain the permuted columns and cryptographic permutation arguments that allow to probabilistically verify that the new columns are indeed a permutation of the original ones.

The soundness of those arguments is based upon the Schwartz-Zippel lemma [MR95, pp. 165-166].

Lemma 4.2.1 (Schwartz-Zippel)
Let $\mathbb{F}$ be a finite field and let $Q \in \mathbb{F}[X_1, \ldots, X_m]$ be a multivariate polynomial that is not identically zero with $\deg Q \leq d$. Then Q has at most $d|\mathbb{F}|^{m-1}$ roots, that is

$$\Pr_{x \in \mathbb{F}^m}[Q(x) = 0] \leq \frac{d|\mathbb{F}|^{m-1}}{|\mathbb{F}|^m} = \frac{d}{|\mathbb{F}|}.$$

Proof. The proof is by induction on m

- *Base Case ($m = 1$)* : An univariate polynomial with $\deg Q \leq d$ has at most d roots, as also seen in the proof of Theorem 2.2.4.
- *Inductive Step* : Suppose that the conclusion is true for $m - 1$. If we take $i = \deg_{X_m} Q$ we get

  $$Q(X_1, \ldots, X_m) = \sum_{k=0}^{i} Q_k(X_1, \ldots, X_{m-1}) X_m^k,$$

  where $Q_k \in \mathbb{F}[X_1, \ldots, X_{m-1}]$. Because $\deg Q_i \leq \deg Q - i = d - i$ by inductive hypotesis we have

  $$\Pr_{\mathbf{a} \in \mathbb{F}^{m-1}}[Q_i(a_1, \ldots, a_{m-1}) = 0] \leq \frac{d-i}{|\mathbb{F}|}.$$

  For each $\mathbf{a} = (a_1, \ldots, a_{m-1}) \in \mathbb{F}^{m-1}$ consider the following polynomial

  $$Q_{\mathbf{a}}(X_m) = Q(a_1, \ldots, a_{m-1}, X_m) = \sum_{k=0}^{i} Q_k(a_1, \ldots, a_{m-1}) X_m^k.$$

  Suppose that $Q_i(a_1, \ldots, a_{m-1}) \neq 0$, then $Q_{\mathbf{a}}(X_m) \not\equiv 0$.

  Therefore being that $\deg Q_{\mathbf{a}} = i \leq d$ and using the base case we get

  $$\Pr_{a_m \in \mathbb{F}}[Q_{\mathbf{a}}(a_m) = 0 \,|Q_i(a_1, \ldots, a_{m-1}) \neq 0\,] \leq \frac{i}{|\mathbb{F}|}$$

that is

$$\Pr_{(a_1,\ldots,a_m)\in\mathbb{F}^m}[Q(a_1,\ldots,a_m)=0\,|Q_i(a_1,\ldots,a_{m-1})\neq 0\,]\leq\frac{i}{|\mathbb{F}|}$$

If we denote by A the event $Q(a_1,\ldots,a_m)=0$ for $(a_1,\ldots,a_m)\in\mathbb{F}^m$ and by B the event $Q_i(a_1,\ldots,a_{m-1})=0$ for $(a_1,\ldots,a_{m-1})\in\mathbb{F}^{m-1}$ we obtain that

$$\begin{aligned}\Pr[B]&\leq\frac{d-i}{|\mathbb{F}|}\\ \Pr[A|B^C]&\leq\frac{i}{|\mathbb{F}|}\end{aligned}.$$

Given that

$$\Pr[A]=\Pr[A\cap B]+\Pr[A\cap B^C]$$

and using the conditioned probability formula

$$\begin{aligned}\Pr[A]&=\Pr[A|B]\Pr[B]+\Pr[A|B^c]\Pr[B^c]\\ &\leq\Pr[B]+\Pr[A|B^c]\leq\frac{d-i}{|\mathbb{F}|}+\frac{i}{|\mathbb{F}|}=\frac{d}{|\mathbb{F}|}\end{aligned}$$

□

Definition 4.2.2 (Multiset Equality)
Given $n\in\mathbb{Z}^+$ and two vectors $\mathbf{f},\mathbf{g}\in\mathbb{F}^n$, we will say that $\mathbf{f}$ and $\mathbf{g}$ are equal as multisets and write $\mathbf{f}\doteq\mathbf{g}$ if there exists a permutation π of the index set $\{1,\ldots,n\}$ such that $f_i=g_{\pi(i)}$ for each $i\in\{1,\ldots,n\}$.

We have the following Lemma [AMGM24, p. 17]

Lemma 4.2.3 (Soundness of Multiset Equality)
Given $n\in\mathbb{Z}^+$ and two vectors $\mathbf{f},\mathbf{g}\in\mathbb{F}^n$, then if $\mathbf{f}\neq\mathbf{g}$ we have

$$\Pr_{\gamma\in\mathbb{F}}\left[\prod_{i=1}^{n}(f_i+\gamma)=\prod_{i=1}^{n}(g_i+\gamma)\right]\leq\frac{n-1}{|\mathbb{F}|}.$$

Proof. Given that $\mathbf{f}\neq\mathbf{g}$ we have $f_i\neq g_j$ for some $i,j\in\{1,\ldots,n\}$. Therefore if we define

$$\begin{aligned}F(X)&:=\prod_{i=1}^{n}(f_i+X)\\ G(X)&:=\prod_{i=1}^{n}(g_i+X)\end{aligned}$$

they are built using two different sets of roots and then $F\neq G$ and so $F-G\not\equiv 0$.

Now $\deg F=\deg G=n$ and given that both are monic polynomials, the term of degree n in $F-G\not\equiv 0$ vanishes.

Thus we have $\deg(F-G)=n-1$ and using the Schwartz-Zippel lemma we have

$$\Pr_{\gamma\in\mathbb{F}}[F(\gamma)-G(\gamma)=0]\leq\frac{n-1}{|\mathbb{F}|}$$

that is the conclusion. □

Remark 4.2.4 (Field Choice)
From Lemma 4.2.3 we have that if the Prover is dishonest and $f \neq g$ then

$$\Pr_{\gamma \in \mathbb{F}} \left[\prod_{i=1}^{n} (f_i + \gamma) \neq \prod_{i=1}^{n} (g_i + \gamma) \right] \geq 1 - \frac{n-1}{|\mathbb{F}|}.$$

In STARK protocols we have $|\mathbb{F}| \gg n$ to make terms like $\frac{n-1}{|\mathbb{F}|}$ negligible.

In this way the Verifier can uniformly choose a random $\gamma \in \mathbb{F}$ and if the equality holds he can be convinced with high probability that $f \doteq g$.

Now we analyze the cryptographic permutation argument, but before we make some simplifying assumptions.

Remark 4.2.5 (Notation)
To ease the index manipulation, without loss of generality, we suppose to decompose only one constraint polynomial P_{π_k} and to add auxiliary columns beginning from the index $w+1$.

To obtain the general result, it suffices to add an offset to the indices, in particular if $v_{\pi_k} \leq w$ is the number of columns involved in the polynomial P_{π_k}, when we modify the polynomial $P_{\pi_{k+1}}$ we need to add the offset $\sum_{m=1}^{k}(v_{\pi_m}+1)$ to the auxiliary columns indices.

In a similar way, we will assume that all the polynomials have the same evaluation domain H.

Suppose that the permutation constraint polynomial P_{π_k} uses the i-th trace column and define

$$\mathbf{f} = (f_0, \ldots, f_T)^{\mathrm{T}} = (W_i(h_0), \ldots, W_i(h_T))^{\mathrm{T}} \in \mathbb{F}^{T+1}$$
$$\mathbf{g} = (g_0, \ldots, g_T)^{\mathrm{T}} = (W_i(\pi_k(h_T)), W_i(\pi_k(h_0)), \ldots, W_i(\pi_k(h_{T-1})))^{\mathrm{T}} \in \mathbb{F}^{T+1}.$$

The first auxiliary column $x_{w+1}(t)$ will be defined such that $W_{w+1}(h_t) = g_t$ and this allows to exchange the variable Y_i with the variable Y_{w+1} in P_{π_k}, obtaining a permutation constraint polynomial P'_{π_k} that is satisfied for $\pi'_k = \sigma$ because we imposed

$$W_{w+1}(\sigma(h_t)) = W_i(\pi_k(h_t)),$$

so we formally have a local constraint polynomial.

To verify that the auxiliary column is a permutation of the original we consider the running product

$$\prod_{i=0}^{t-1} \frac{(f_i + \gamma)}{(g_i + \gamma)},$$

and introduce a second auxiliary column $x_{w+2}(t)$ such that

$$W_{w+2}(h_t) = Z(h_t) = \begin{cases} 1 & \text{if } t = 0 \\ \prod_{i=0}^{t-1} \frac{(f_i + \gamma)}{(g_i + \gamma)} & \text{if } t \in \{1, \ldots, T\} \end{cases}.$$

It suffices to add the following constraint specifications

- given that $Z(h_0) - 1 = 0$ we obtain $(X_{w+2} - 1, \{h_0\})$;
- given that
$$Z(h_t) = Z(h_{t-1}) \cdot \frac{(f_{t-1} + \gamma)}{(g_{t-1} + \gamma)}$$
we obtain
$$(Y_{w+2}(X_{w+1} + \gamma) - X_{w+2}(X_i + \gamma), H \setminus \{h_T\});$$

- finally to impose that

$$\prod_{i=0}^{T} \frac{(f_i + \gamma)}{(g_i + \gamma)} = 1$$

that is

$$\frac{(f_T + \gamma)}{(g_T + \gamma)} \prod_{i=0}^{T-1} \frac{(f_i + \gamma)}{(g_i + \gamma)} = 1$$

and so

$$\frac{(f_T + \gamma)}{(g_T + \gamma)} Z(h_T) = 1$$

we add the following specification

$$(X_{w+2} \cdot (X_i + \gamma) - (X_{w+1} + \gamma), \{h_T\}).$$

In practice H is always chosen as a cyclic subgroup so we have $\sigma(h_T) = h_0$, this means that we can limit to the specifications

$$\{(X_{w+2} - 1, \{h_0\}), (Y_{w+2}(X_{w+1} + \gamma) - X_{w+2}(X_i + \gamma), H)\}.$$

The argument that uses the running product can be extended to the case where there are more columns involved in P_{π_k}.

If $(\mathbf{f}_1, \ldots, \mathbf{f}_N)$ is the vector of the original columns and $(\mathbf{g}_1, \ldots, \mathbf{g}_N)$ is the vector of the permuted ones, for each vector we aggregate the values on the same row with a random linear combination and obtain only two columns that can be verified using the multiset equality argument.

For the random linear combination we have the following lemma [AMGM24, p. 23]

Lemma 4.2.6 (Soundness of Random Linear Combination)
Given $N \in \mathbb{Z}^+$ and $\mathbf{f}, \mathbf{g} \in \mathbb{F}^N$, then if $\mathbf{f} \neq \mathbf{g}$ we have

$$\Pr_{\alpha \in \mathbb{F}} \left[\sum_{i=1}^{N} \alpha^{i-1} f_i = \sum_{i=1}^{N} \alpha^{i-1} g_i \right] \leq \frac{(N-1)}{|\mathbb{F}|}.$$

Proof. In a similar way as we did in Lemma 4.2.3, because $\mathbf{f} \neq \mathbf{g}$ we have $f_i \neq g_j$ for some $i, j \in \{1, \ldots, N\}$. Then if we define

$$F'(X) := \sum_{i=1}^{N} X^{i-1} f_i$$
$$G'(X) := \sum_{i=1}^{N} X^{i-1} g_i$$

at least one of the coefficients is different and then $F' \neq G'$. Now $\deg F, \deg G \leq N - 1$ and using the Schwartz-Zippel lemma on $F' - G' \not\equiv 0$ we have

$$\Pr_{\alpha \in \mathbb{F}} [F'(\alpha) - G'(\alpha) = 0] \leq \frac{N-1}{|\mathbb{F}|}$$

that is the conclusion. □

For the total probability we can directly apply the Schwartz-Zippel lemma to the bivariate polynomial to obtain a tighter error bound than the one shown in [AMGM24, p. 24]

Lemma 4.2.7 (Soundness of Vector Multiset Equality)
Given $n, N \in \mathbb{Z}^+$ with $N \geq 2$ and given $\mathbf{f}_i, \mathbf{g}_i \in \mathbb{F}^n$ for $i \in \{1, \ldots, N\}$, then if

$$(\mathbf{f}_1, \ldots, \mathbf{f}_N) \neq (\mathbf{g}_1, \ldots, \mathbf{g}_N)$$

we have

$$\Pr_{(\alpha,\gamma)\in\mathbb{F}^2}\left[\prod_{j=1}^{n}(F'_j(\alpha)+\gamma) = \prod_{j=1}^{n}(G'_j(\alpha)+\gamma)\right] \leq \frac{n(N-1)}{|\mathbb{F}|}$$

where

$$F'_j(X) := \sum_{i=1}^{N} X^{i-1} f_{i,j}, \quad G'_j(X) := \sum_{i=1}^{N} X^{i-1} g_{i,j} \quad \text{for } j \in \{1, \ldots, n\}.$$

Proof. Considering the bivariate polynomial

$$Q(X,Y) = \prod_{j=1}^{n}(\sum_{i=1}^{N} X^{i-1} f_{i,j} + Y) - \prod_{j=1}^{n}(\sum_{i=1}^{N} X^{i-1} g_{i,j} + Y),$$

we have $\deg Q \leq n(N-1)$ and given that $(\mathbf{f}_1, \ldots, \mathbf{f}_N) \neq (\mathbf{g}_1, \ldots, \mathbf{g}_N)$ it must be $Q \not\equiv 0$, using Schwartz-Zippel we obtain

$$\Pr_{(\alpha,\gamma)\in\mathbb{F}^2}[Q(X,Y) = 0] \leq \frac{n(N-1)}{|\mathbb{F}|}$$

that is the conclusion. □

In a similar way as we have seen for the constraint polynomial P_{π_k} that involved only one column, if $v_{\pi_k} \leq w$ is the number of columns involved in P_{π_k} we can use the index $v \in \{1, \ldots, v_{\pi_k}\}$ and with $i_v \in \{i_1, \ldots, i_{v_{\pi_k}}\}$ we can identify the indices of the original columns and write

$$\mathbf{f}_v = (f_{v,0}, \ldots, f_{v,T})^{\mathrm{T}} = (W_{i_v}(h_0), \ldots, W_{i_v}(h_T))^{\mathrm{T}} \in \mathbb{F}^{T+1}$$
$$\mathbf{g}_v = (g_{v,0}, \ldots, g_{v,T})^{\mathrm{T}} = (W_{i_v}(\pi_k(h_T)), W_{i_v}(\pi_k(h_0)), \ldots, W_{i_v}(\pi_k(h_{T-1})))^{\mathrm{T}} \in \mathbb{F}^{T+1}.$$

Now for each $v \in \{1, \ldots, v_{\pi_k}\}$ we define the auxiliary columns using $W_{w+v}(h_t) = g_{v,t}$ to obtain columns that are in permuted and shifted order, that allows us to substitute the variable Y_{i_v} with the variable Y_{w+v} in the permutation constraint polynomial P_{π_k}, obtaining a permutation constraint polynomial P'_{π_k} that is satisfied for $\pi'_k = \sigma$ and then we formally have a local constraint polynomial.

For the vector permutation argument we have

$$W_{w+v_{\pi_k}+1}(h_t) = Z_{\pi_k}(h_t) = \begin{cases} 1 & \text{if } t = 0 \\ \prod_{j=0}^{t-1} \frac{(\sum_{v=1}^{v_{\pi_k}} \alpha^{v-1} f_{v,j} + \gamma)}{(\sum_{v=1}^{v_{\pi_k}} \alpha^{v-1} g_{v,j} + \gamma)} & \text{if } t \in \{1, \ldots, T\} \end{cases}.$$

Suppose H cyclic, so it suffices to add this set of constraint specifications for each P_{π_k}:

- given that $Z_{\pi_k}(h_0) - 1 = 0$ we obtain the specification $(X_{w+v_{\pi_k}+1} - 1, \{h_0\})$;
- given that

$$Z_{\pi_k}(h_t) = Z_{\pi_k}(h_{t-1}) \cdot \frac{(\sum_{v=1}^{v_{\pi_k}} \alpha^{v-1} f_{v,t-1} + \gamma)}{(\sum_{v=1}^{v_{\pi_k}} \alpha^{v-1} g_{v,t-1} + \gamma)}$$

we obtain the specification

$$\left(Y_{w+v_{\pi_k}+1} \cdot (\sum_{v=1}^{v_{\pi_k}} \alpha^{v-1} X_{w+v} + \gamma) - X_{w+v_{\pi_k}+1} \cdot (\sum_{v=1}^{v_{\pi_k}} \alpha^{v-1} X_{i_v} + \gamma), H\right).$$

Therefore we have seen that given an AIR

$$\mathcal{AIR}_1 = (\mathbb{F}, T, w, H, \sigma(X), \mathcal{P}, \mathcal{P}_{\boldsymbol{\pi}}, \mathcal{B})$$

by repeating the process for each permutation constraint polynomial we can obtain the AIR

$$\mathcal{AIR}_2 = (\mathbb{F}, T, w', H, \sigma(X), \mathcal{P}', \mathcal{P}'_{\boldsymbol{\pi}}, \mathcal{B}')$$

such that, if r is the number of permutation constraint polynomials in $\mathcal{P}_{\boldsymbol{\pi}}$, the new non-local constraints are satisfied with permutation $\pi'_k = \sigma$ for each $k \leq r$ and, if $v_{\pi_k} \leq w$ is the number of columns involved in P_{π_k}, we have

$$w' = w + \underbrace{\sum_{k=1}^{r} v_{\pi_k}}_{\text{permuted columns}} + \underbrace{r}_{\text{permutation arguments}} \leq w + r(w+1).$$

Proposition 4.2.8 (Probabilistically Sound Extension)
Let $\mathcal{AIR}_2$ be the AIR obtained from $\mathcal{AIR}_1$ using the permutation arguments then we have

$$\mathcal{AIR}_1 \preceq_{\Pr}^{\epsilon_{ARG}} \mathcal{AIR}_2$$

with $\epsilon_{ARG} = \sum_{k=1}^{r} \epsilon_k$ *where*

$$\epsilon_k = \begin{cases} \dfrac{T}{|\mathbb{F}|} & \text{if } v_{\pi_k} = 1 \\ \dfrac{(T+1)(v_{\pi_k}-1)}{|\mathbb{F}|} & \text{if } v_{\pi_k} \geq 2 \end{cases}$$

Proof. The extension function for the general case is

$$\begin{aligned} \text{Ext} : &\mathbb{F}^{(T+1)\times w} \times H^{(T+1)\times r} \times (\mathbb{F}^2)^r \to \mathbb{F}^{(T+1)\times w'} \times H^{(T+1)\times r} \\ &((x,p),(\alpha_1,\gamma_1),\ldots,(\alpha_r,\gamma_r)) \mapsto (x',p') \end{aligned}$$

where, if $(\mathbf{W}, \boldsymbol{\pi})$ is the vector of polynomials obtained by interpolating the columns of (x,p) over H, we have

$$x' = \left(x, A_1, \ldots, A_r\right)$$

where for each non-local constraint polynomial P_{π_k} that uses v_{π_k} columns and their indices $\{i_{\pi_k,1}, \ldots, i_{\pi_k,v_{\pi_k}}\}$ we have

$$A_k = \begin{pmatrix} W_{i_{\pi_k,1}}(\pi_k(h_T)) & \cdots & W_{i_{\pi_k,v_{\pi_k}}}(\pi_k(h_T)) & Z_{\pi_k}(h_0) \\ W_{i_{\pi_k,1}}(\pi_k(h_0)) & \cdots & W_{i_{\pi_k,v_{\pi_k}}}(\pi_k(h_0)) & Z_{\pi_k}(h_1) \\ \vdots & \ddots & \vdots & \\ W_{i_{\pi_k,1}}(\pi_k(h_{T-1})) & \cdots & W_{i_{\pi_k,v_{\pi_k}}}(\pi_k(h_{T-1})) & Z_{\pi_k}(h_T) \end{pmatrix}$$

where Z_{π_k} depends on the random challenges $(\alpha_k, \gamma_k) \in \mathbb{F}^2$ and imposing $p'_{k,t} = \sigma(h_t)$ we have

$$p' = \begin{pmatrix} h_1 & \cdots & h_1 \\ \vdots & \ddots & \vdots \\ h_T & \cdots & h_T \\ h_0 & \cdots & h_0 \end{pmatrix}$$

so $\pi'_k = \sigma$.

Given that $\preceq^{\epsilon}_{\Pr}$ has the transitivity property we can prove a single step and then use transitivity.

Consider the constraint polynomial P_{π_k} and we take

$$x' = (x, A_k)$$

and the same p' as before. We need to show the three properties:

- **Information Preservation:** $x'_{[0:T;1:w]} = x$ by construction;
- **Completeness:** if $(x, p) \in \mathcal{T}_{\mathcal{AIR}_1}$ then all the original specifications are satisfied.

  In the new AIR interpolating p' we obtain the vector $\boldsymbol{\pi}' = (\sigma, \ldots, \sigma)$.

  The new non-local constraints are therefore satisfied because, by construction, the auxiliary columns contain the exact permuted and shifted versions of the original ones. Consequently, the running product column will also satisfy the new constraints for the permutation argument.
- **Soundness:** as shown in Lemma 4.2.7 if $(x, p) \notin \mathcal{T}_{\mathcal{AIR}_1}$ then if $v_k \geq 2$

  $$\Pr_{(\alpha_k,\gamma_k)\in\mathbb{F}^2}\left[\prod_{j=0}^{T}(\sum_{v=1}^{v_{\pi_k}} \alpha_k^{v-1} f_{v,j} + \gamma_k) - \prod_{j=0}^{T}(\sum_{v=1}^{v_{\pi_k}} \alpha_k^{v-1} g_{v,j} + \gamma_k)\right] \leq \frac{(T+1)(v_{\pi_k}-1)}{|\mathbb{F}|}$$

  that is

  $$\Pr_{(\alpha_k,\gamma_k)\in\mathbb{F}^2}\left[(x', p') \in \mathcal{T}_{\mathcal{AIR}_2}\right] \leq \frac{(T+1)(v_{\pi_k}-1)}{|\mathbb{F}|}.$$

  That is $\epsilon_k = \dfrac{(T+1)(v_{\pi_k}-1)}{|\mathbb{F}|}$ if $v_{\pi_k} \geq 2$.

  If $v_{\pi_k} = 1$, the running product reduces to the one that depends only on the random γ, so we are in the case of Lemma 4.2.3 and we have $\epsilon_k = \dfrac{T}{|\mathbb{F}|}$.

Using transitivity we obtain that in the general case, applying the extension to all the r non-local constraint polynomials

$$\Pr_{((\alpha_1,\gamma_1),\ldots,(\alpha_r,\gamma_r))\in(\mathbb{F}^2)^r}\left[(x', p') \in \mathcal{T}_{\mathcal{AIR}_2}\right] \leq \sum_{k=1}^{r} \epsilon_k.$$

□

We have seen how we can force $\pi'_k = \sigma$ using a probabilistically sound extension, so when H is cyclic we have $\deg \pi' = \deg \sigma = 1$ and then the new check polynomials have degree

$$\deg \mathrm{Check}_{P'_{\pi_k}}(X) \leq \deg(P'_{\pi_k}) \cdot T.$$

The protocol has different steps:

1. the Prover finds $(x, p) \in \mathcal{T}_{\mathcal{AIR}_1}$ and computes all the columns that do not depend on the random challenges (α_k, γ_k).

   He then computes the trace polynomials $\mathbf{W}$ on this extended trace and commits to them;

2. the Verifier sends the random challenges $(\alpha_k, \gamma_k) \in \mathbb{F}^2$ for each r;

3. the Prover now computes the whole x' adding the running product columns Z_{π_k} and commits to $\mathbf{Z}$;

4. the protocol can now use $\mathcal{AIR}_2$ because $\mathcal{AIR}_1 \preceq^{\epsilon_{ARG}}_{\mathrm{Pr}} \mathcal{AIR}_2$.

Remark 4.2.9 (Notation)
For the sake of simplicity, we will use $\mathbf{W}$ *to refer to the pair* $(\mathbf{W}, \mathbf{Z})$ *and when we will discuss the commitment to* $\mathbf{W}$*, we will be referring to the commitments made to* $\mathbf{W}$ *and* $\mathbf{Z}$ *separately.*

We will reintroduce the explicit distinction between the two components only in Section 4.4.3 and in Chapter 6.

4.3 Local Constraints

We have formally obtained all local constraint polynomials so we need to analyze the degree in this scenario. We have $\deg \mathbf{W} \leq T$ and if H is a cyclic subgroup we have $\deg \sigma = 1$ that implies

$$\deg \mathrm{Check}_{P_i}(X) \leq \deg(P_i) \cdot T$$

and given that $P_i \in \mathbb{F}_q[\mathbf{X}, \mathbf{Y}]$ is a polynomial in $2w$ variables we obtain

$$\deg \mathrm{Check}_{P_i}(X) \leq 2w(q-1) \cdot T.$$

This again is a prohibitive upper bound, so we analyze valid extensions that allow to reduce the degree to $\deg(P_i) \leq d_{max}$ where d_{max} is a predetermined little constant.

4.3.1 FSM-based Constraints

In the design of AIRs we usually convert the logic of a state transition function, so we begin analyzing the optimization of constraint polynomials of the form

$$P_k(\mathbf{X}, \mathbf{Y}) = Y_k - D_k(\mathbf{X}),$$

where $\mathbf{D}$ is the canonical representative of the transition function δ. If δ has a complex logic, we will have high degree constraints.

Remark 4.3.1 (Simplifying Notation)
Since we are restricting to FSM-based constraint polynomials, they all have evaluation domain $H \setminus \{h_T\}$*. So, to ease notation, we will only write the polynomials instead of the complete specifications.*

We will unroll the evaluation of $D_k(\mathbf{X})$ using a set of operations I and saving the intermediate results in auxiliary columns.

One can initially limit the unrolling to the use of addition and multiplication, obtaining an unrolling similar to the one used in R1CS (Rank One Constraint Systems) [Tha22, p. 130], [Ram25, pp. 25-27].

Example 4.3.2 (Unrolling Using $I = \{+, \cdot\}$)
The naive approach consists in obtaining $d_{max} = 2$ using the set of elementary operations

$$I = \{+, \cdot\},$$

this gives a number of auxiliary columns equal to the number of elementary operations required to evaluate D_k.

In particular, if the l-th step of the evaluation of $D_k(\mathbf{X})$ uses the operation $X_a \circ X_b$ where $\circ \in I$ and $a, b < w + l$, we add the polynomial constraint

$$P'_{k,l}(\mathbf{X}', \mathbf{Y}') := X_{w+l} - (X_a \circ X_b)$$

that depends only on the variables $\mathbf{X}'$.

So if the complete evaluation of $D_k(\mathbf{X})$ uses N_k operations from I, we will use $N_k + 1$ substitutive constraint polynomials $\{P'_{k,1}, \ldots, P'_{k,N_k}, P'_{k,N_k+1}\}$ that have degree at most 2 and N_k auxiliary columns to memorize the intermediate results.

The last constraint polynomial P'_{k,N_k+1} is the one that links Y_k to the evaluation of $D_k(\mathbf{X})$ that is memorized in column X_{w+N_k}, that is the degree 1 polynomial

$$P'_{k,N_k+1}(\mathbf{X}', \mathbf{Y}') := Y_k - X_{w+N_k}.$$

This naive approach has a limitation. If $D_k(\mathbf{X})$ is a multivariate dense polynomial of degree $d = d_1 + \cdots + d_w$ where $d_i = \deg_{X_i} D_k$ then the complexity of the evalation in a point $(a_1, \ldots, a_w) \in \mathbb{F}_q^w$ must be at least the complexity of the evaluation of the univariate polynomial $D_k(a_1, \ldots, a_{w-1}, X_w)$ where the evaluation on the first $w - 1$ components already took place.

Given that the Horner method is optimal in evaluating univariate polynomials [BBCM92, p. 16], we have that $N_k \geq \Omega(d_w)$ and then, given that $d_w \leq q - 1$, unrolling a dense polynomial using only $I = \{+, \cdot\}$ can require an amount of auxiliary columns that renders this approach infeasible.

The non-dense but high degree case is also problematic as we can see with the inversion operation.

Example 4.3.3 (Inversion Unrolling Using $I = \{+, \cdot\}$)
Consider the constraint polynomial $Y_k - X_i^{-1}$ where $Y_k, X_i \in \mathbb{F}_q$ and $k, i \leq w$ and assume that $X_i \neq 0$. As a consequence of the Lagrange theorem for groups we have that $X_i^{-1} = X_i^{q-2}$ in $\mathbb{F}_q$ so the constraint polynomial can be written using the polynomial representative of the inversion as

$$P_k(\mathbf{X}, \mathbf{Y}) = Y_k - X_i^{q-2} \in \mathbb{F}_q[\mathbf{X}, \mathbf{Y}].$$

We can use the binary method for exponentiation *[Knu98, p. 462] that we show in Algorithm 1.*
The value of R is updated when E is odd using a multiplication and the value of B is updated at each iteration using another multiplication. We do not count the halving operation because it is performed with a right bit shift and in the same way the check of $E \pmod 2 \neq 0$ is performed by reading the least significant bit.

The total number of iterations is given by the total number of times e can be halved, that is $\lfloor \log_2 e \rfloor + 1$. So the maximum number of multiplications is $2\lfloor \log_2 e \rfloor$.

Given that in our case $e = q - 2$, we have $N_k = O(\log_2 q)$ auxiliary columns. This can be problematic given that q can be pretty big as seen in Table 2.2.15.

$R \leftarrow 1$
$B \leftarrow X$ ▷ *Base*
$E \leftarrow e$ ▷ *Exponent*
while $E > 0$ **do**
  if $E \pmod 2 \neq 0$ **then**
    ▷ *If E is odd*
  $R \leftarrow R \cdot B$ ▷ *Update the result*
  end if
  $B \leftarrow B \cdot B$ ▷ *Square B*
  $E \leftarrow \lfloor E/2 \rfloor$ ▷ *Halve E*
end while
return R ▷ *Result in R corresponds to* X^e

Algorithm 1: Binary Method for Exponentiation X^e

There exists a more efficient way to constrain the inversion, we will analyze the simpler case INV instead of INV0 that is treated in [Sze].

Example 4.3.4 (Unrolling Inversion)
Let $X_i \neq 0$ and consider the constraint polynomial $Y_k - X_i^{-1}$, instead of directly unrolling

$$P_k(\mathbf{X}, \mathbf{Y}) = Y_k - X_i^{q-2}$$

we introduce an auxiliary variable X_{w+1} and use the following constraint polynomial of degree 2

$$P'_{k,1}(\mathbf{X}', \mathbf{Y}') := X_i \cdot X_{w+1} - 1.$$

The constraint is

$$P'_{k,1}(\mathbf{W}'(h_t), \mathbf{W}'(\sigma(h_t))) = 0$$

and that guarantees that if $W_i(h_t) \neq 0$ then $W_{w+1}(h_t) = (W_i(h_t))^{-1}$.

The column X_{w+1} will contain the computed value for the inverse so we add the constraint polynomial

$$P'_{k,2}(\mathbf{X}', \mathbf{Y}') := Y_k - X_{w+1}.$$

The complete set of substitute constraint polynomials for $P_k(\mathbf{X}, \mathbf{Y}) = Y_k - X_i^{q-2}$ is

$$\mathcal{P}'_k = \{X_i \cdot X_{w+1} - 1, \quad Y_k - X_{w+1}\}.$$

This approach uses only one auxiliary column and so is more efficient than the naive approach that uses $O(\log_2 q)$ auxiliary columns.

The previous example for INV suggests that some operations that have high degree can indeed be constrained using a set of constraint polynomials that have maximum degree d_{max}, we group them in a definition.

Definition 4.3.5 (Low-Degree Constrainable Operation)
Given a function $f : \mathbb{F}^{n_{in}} \to \mathbb{F}^{n_{out}}$. We will call f low-degree constrainable *if there exist*

- *a number $w_{aux,f} \geq 0$ of auxiliary variables* $\mathbf{Z}$

- *a set of c_f constraint polynomials $\{C_1, \dots, C_{c_f}\}$, where each $C_j \in \mathbb{F}[\mathbf{X}, \mathbf{Z}, \mathbf{Y}]$ has maximum degree $d_{\max}$*

such that for each input $\mathbf{x} \in \mathbb{F}^{n_{in}}$ and output $\mathbf{y} \in \mathbb{F}^{n_{out}}$, the following hold

$$\mathbf{y} = f(\mathbf{x}) \iff \exists \mathbf{z} \in \mathbb{F}^{w_{aux,f}} \text{ such that } C_j(\mathbf{x}, \mathbf{z}, \mathbf{y}) = 0 \text{ for each } j = 1, \dots, c_f.$$

That suggests to expand the set I to include low-degree constrainable operations.

Example 4.3.6 (Unrolling $P_k(\mathbf{X}, \mathbf{Y}) = Y_k - X_i^{q-n}$ using $I = \{+, \cdot, \text{INV}\}$)
Let $P_k(\mathbf{X}, \mathbf{Y}) = Y_k - X_i^{q-n}$ where $n \in \mathbb{N}$ and $2 \leq n \ll q$ and assume $X_i \neq 0$.

We remember that

$$X_i^{q-n} = X_i^{q-1-(n-1)} = X_i^{q-1} \cdot X_i^{-(n-1)} = 1 \cdot X_i^{-(n-1)} = (X_i^{-1})^{n-1}$$

so we consider the constraint polynomial $P_k(\mathbf{X}, \mathbf{Y}) = Y_k - (X_i^{-1})^{n-1}$.

Using the operations contained in I we implement the constraint polynomial set as it follows:

1. ***Inverse:*** *We introduce the auxiliary column X_{w+1} that contains X_i^{-1} and the constraint polynomial*
$$P'_{k,1}(\mathbf{X}', \mathbf{Y}') = X_i \cdot X_{w+1} - 1.$$

2. ***Power:*** *we use the binary method to compute $(X_{w+1})^{n-1}$, so we need $N_{pow} = O(\log_2 n)$ auxiliary variables and constraint polynomials.*

   The last variable $X_{w+1+N_{pow}}$ will contain $(X_{w+1})^{n-1}$.

3. ***Output:*** *at last we link the result in $X_{w+1+N_{pow}}$ to Y_k with the constraint polynomial*
$$P'_{k,c_k}(\mathbf{X}', \mathbf{Y}') = Y_k - X_{w+1+N_{pow}}$$
   where $c_k = 1 + N_{pow} + 1$.

If we denote by $w_{aux,k}$ and $c_{aux,k}$ respectively the number of auxiliary columns and the number of constraint polynomials resulting from the unrolling of P_k, we have that the complete set

$$\mathcal{P}'_k = \{P'_{k,1}, \dots, P'_{k,c_{aux,k}}\}$$

has $c_{aux,k} = O(\log_2 n)$ constraint polynomials of maximum degree $d_{max} = 2$ and those polynomials need $w_{aux,k} = O(\log_2 n)$ auxiliary variables.

Given that $n \ll q$ we have $w_{aux,k} = O(\log_2 n) \ll O(\log_2 q)$ that is more efficient than the naive approach.

We can gather those concepts in a formal proposition and prove that the approach is a valid AIR extension.

Proposition 4.3.7 (Unrolling Extension Validity)
Let $\mathcal{AIR}_A = (\mathbb{F}, T, w, H, \sigma(X), \mathcal{C}_A)$ be an AIR and suppose that the set of its constraint polynomials $\mathcal{C}_A$ contains the polynomial $P_k(\mathbf{X}, \mathbf{Y}) = Y_k - D_k(\mathbf{X})$.

Assume that I is a set of low-degree constrainable operations with maximum degree $d_{\max}$ and that the evaluation of $D_k(\mathbf{X})$ can be expressed as a sequence $(o_1, \dots, o_{N_k})$ of $N_k \geq 0$ operations $o_j \in I$.

We denote by w_{o_j} the number of auxiliary columns and with c_{o_j} the number of constraint polynomials needed to constrain o_j with maximum degree d_{max}.

We build a new AIR $\mathcal{AIR}_C$ starting from $\mathcal{AIR}_A$ as it follows

1. *the new width is* $w_C = w + w_{aux,k}$ *where* $w_{aux,k} = \sum_{j=1}^{N_k} w_{o_j}$.

2. *the original polynomial* P_k *is removed from* $\mathcal{C}_A$ *and replaced by* $c_{aux,k} = \sum_{j=1}^{N_k} c_{o_j} + 1$ *polynomials* $\{ P'_{k,1}, \ldots, P'_{k,c_{aux,k}} \}$ *each of maximum degree* $d_{\max}$.

   Those new polynomials collectively implement the sequence $(o_1, \ldots, o_{N_k})$ *and guarantee that the final result is equal to* Y_k. *We denote the resulting set by* $\mathcal{C}_C$.

Then the obtained $\mathcal{AIR}_C$ *is a valid extension of* $\mathcal{AIR}_A$, *that is*

$$\mathcal{AIR}_A \preceq \mathcal{AIR}_C.$$

Proof. The proof is by induction on the number N_k of operations in I needed to evaluate $D_k(\mathbf{X})$.

- *Base Case ($N_k = 0$)*: If $N_k = 0$ we can use the original constraint polynomial so $w_{\text{aux},k} = 0$ and $c_k = 1$ That is $\mathcal{AIR}_C = \mathcal{AIR}_A$ and by reflexivity $\mathcal{AIR}_A \preceq \mathcal{AIR}_C$.

- *Inductive Step:* Assume that the conclusion holds for each function that uses $N_k - 1$ operations from I.

  We need to prove that the conclusion holds for $D_k(\mathbf{X})$ that uses N_k operations $(o_1, \ldots, o_{N_k})$.

  We consider the decomposition $D_k(\mathbf{X}) = \bar{D}_k(\mathbf{X}, o_1(\mathbf{X}))$ where $\bar{D}_k$ is the composition of the remaining operations $(o_2, \ldots, o_{N_k})$, that is $\bar{D}_k(\mathbf{X}') = (o_{N_k} \circ \cdots \circ o_2)(\mathbf{X}')$ where $\mathbf{X}' = (\mathbf{X}, o_1(\mathbf{X}))$.

  We can replace the original constraint polynomial using

  $$P'_{k,1}(\mathbf{X}', \mathbf{Y}') = X_{w+1} - o_1(\mathbf{X})$$
  $$P'_{k,2}(\mathbf{X}', \mathbf{Y}') = Y_k - \bar{D}_k(\mathbf{X}')$$

  and obtain an intermediate $\mathcal{AIR}_B$.

  Note that the polynomial $P'_{k,1}(\mathbf{X}', \mathbf{Y}') = X_{w+1} - o_1(\mathbf{X})$ is the naive constraint polynomial for o_1 that isn't using intermediate variables yet.

  We have $\mathcal{AIR}_A \preceq \mathcal{AIR}_B$ because considering

  $$\text{Ext}_{A,B} : (x, p) \mapsto (x', p')$$

  where $x' = (x, o_1(x))$ and $p' = p$, indicating with abuse of notation that $o_1(x)$ is the column obtained by applying o_1 to the rows of x.

  Both properties hold

  - **Information Preservation:** $x'_{[0:T;1:w]} = x$ by construction;
  - **Validity Equivalence:** $(x, p) \in \mathcal{T}_{\mathcal{AIR}_A} \iff (x', p') \in \mathcal{T}_{\mathcal{AIR}_B}$ because the new constraints are logically equivalent to the old ones.

  We now unroll o_1 and $\bar{D}_k$ obtaining $\mathcal{AIR}_C$ such that $\mathcal{AIR}_B \preceq \mathcal{AIR}_C$, that's because o_1 is low-degree constrainable (so the unrolling is logically equivalent to the original constraint giving a valid extension) and unrolling $\bar{D}_k$ is a valid extension by induction hypotesis.

  Given that $\mathcal{AIR}_A \preceq \mathcal{AIR}_B$ we conclude by transitivity.

□

Given a constraint polynomial derived from $\mathbf{D}\colon \mathbb{F}^w \to \mathbb{F}^w$ expressed as a vector

$$\mathbf{P}(\mathbf{X}, \mathbf{Y}) = \mathbf{Y} - \mathbf{D}(\mathbf{X})$$

we can use Proposition 4.3.7 on the components obtaining

$$w_{tot} = \sum_{k=1}^{w} w_{aux,k}$$

auxiliary columns and

$$c_{tot} = \sum_{k=1}^{w} c_{aux,k}$$

constraint polynomials.

Remark 4.3.8 (Scaling Problem)
The number of steps N_k used in the unrolling of D_k using the operations contained in I is not determined by a simple formula but, as in the traditional computational complexity analysis, optimizing N_k involves case by case choices.

This manual approach does not scale well for creating general-purpose and easily reusable proof systems.

4.3.2 General Local Constraints

The unrolling logic can be easily adapted to generic $2w$-variate polynomials $P_i(\mathbf{X}, \mathbf{Y})$. We give a simple example similar to the one found in [AMGM24][p. 30].

Example 4.3.9 (Degree Reduction)
Suppose that an AIR $\mathcal{AIR}_1$ with $w = 3$ contains the constraint polynomial

$$P_k(\mathbf{X}, \mathbf{Y}) = X_1 \cdot X_2 \cdot Y_2 \cdot Y_3 - X_3^3,$$

to lower the degree to $d_{max} = 3$ we can replace $P_k(\mathbf{X}, \mathbf{Y})$ with the set $\mathcal{P}'_k$ containing

$$\begin{aligned} P'_{k,1}(\mathbf{X}', \mathbf{Y}') &= X_4 \cdot Y_2 \cdot Y_3 - X_3^3 \\ P'_{k,2}(\mathbf{X}', \mathbf{Y}') &= X_1 \cdot X_2 - X_4 \end{aligned}$$

where $\mathbf{X}', \mathbf{Y}'$ are the variables in the extended execution trace

$$\mathbf{X}' = (X_1, \ldots, X_4)$$

$$\mathbf{Y}' = (Y_1, \ldots, Y_4).$$

We obtained the new AIR $\mathcal{AIR}_2$ with $w = 4$.

The one described is a valid extension where the extension function is

$$\begin{aligned} \text{Ext}\colon \mathbb{F}^{(T+1)\times w} \times H^{(T+1)\times r} &\to \mathbb{F}^{(T+1)\times (w+1)} \times H^{(T+1)\times r} \\ (x, p) &\longmapsto (x_1, x_2, x_3, x_1 \odot x_2, p) \end{aligned}$$

where $(x_1 \odot x_2)_j = x_{1,j} \cdot x_{2,j}$ is the Hadamard product and is easy to verify that the conditions for the extension to be valid are met.

In the general case we use the same notation where

$$w_{tot} = \sum_{k=1}^{w} w_{aux,k}$$

is the number of auxiliary columns and

$$c_{tot} = \sum_{k=1}^{w} c_{aux,k}$$

is the number of constraint polynomials.

4.3.3 State Constraints

Regarding state constraint specifications $\mathcal{B} = \{(B_1, H_{B_1}), \ldots, (B_m, H_{B_m})\}$ we apply the same exact logic in unrolling $B_l(\mathbf{X}) \in \mathbb{F}[\mathbf{X}]$ because, depending only on $\mathbf{X}$, it is analogous to the FSM-based case.

In a similar way we will denote by w_{aux,B_l} the number of auxiliary columns and by c_{aux,B_l} the number of constraint polynomials used in the unrolling of B_l obtaining

$$w_{tot,\mathcal{B}} = \sum_{l=1}^{m} w_{aux,B_l}$$

auxiliary columns and

$$c_{tot,\mathcal{B}} = \sum_{l=1}^{m} c_{aux,B_l}$$

constraint polynomials.

4.4 Instruction Set Architectures (ISA) and Virtual Machines (VM)

To avoid designing a custom AIR on a case by case basis, the standard practice, in projects like Starknet or Risc Zero, is to adopt an approach based on an *Instruction Set Architecture (ISA)*.

Instead of starting with an AIR with generic constraint polynomials and trying to reduce the degree using valid extensions, this approach uses a fixed AIR structure. The same system can then prove the execution of any program written for that ISA without changes to the underlying AIR.

To adopt this strategy we proceed as follows:

1. a *Virtual Machine (VM)* architecture is chosen together with its instruction set
$$\mathcal{I} = \{\text{instr}_1, \text{instr}_2, \ldots, \text{instr}_N\}$$
and their relative constraint polynomials;
2. non-local constraint polynomials are modified using permutation arguments, adding $2r$ constraint polynomials $\mathcal{C}_{perm}$;
3. from each instruction instr_i, using the unrolling for degree reduction, we derive the set
$$\mathcal{C}_i = \{(C_{i,1}, H_{\mathcal{C}_i}), \ldots, (C_{i,c_{tot,i}}, H_{\mathcal{C}_i})\}$$
of $c_{tot,i}$ constraint polynomials of maximum degree d_{max} that verify the correct execution of the instruction.
Since only one instruction can be executed at a time, the corresponding evaluation domains for each instruction will be pairwise disjoint
$$H = \bigsqcup_{i=1}^{N} H_{\mathcal{C}_i};$$
4. if necessary, using the unrolling for degree reduction, we get the set
$$\mathcal{C}_{state} = \{(C_{state,1}, H_{C_{state,1}}), \ldots, (C_{state,c_{tot,\mathcal{B}}}, H_{C_{state,c_{tot,\mathcal{B}}}})\}$$
of $c_{tot,\mathcal{B}}$ state constraint polynomials of maximum degree d_{max}.
5. the program is encoded using a sequence of instructions from the ISA and the evaluation domains for the polynomial constraints are set;
6. the final AIR then verifies that, at each step t, one of the ISA instructions was selected and correctly executed, updating $x(t)$ using the rules of that instruction.

Given such an AIR $\mathcal{AIR}_1 = (\mathbb{F}, T, w, H, \sigma(X), \mathcal{C})$ we divide the set $\mathcal{C}$ of constraint polynomials into instruction, permutation arguments and state constraint polynomials obtaining
$$\mathcal{C} = \{\mathcal{C}_1, \ldots, \mathcal{C}_N\} \cup \mathcal{C}_{perm} \cup \mathcal{C}_{state}.$$
We have that the specifications in $\{\mathcal{C}_1, \ldots, \mathcal{C}_N\}$ depend on the specific program, while $\mathcal{C}_{perm}$ and $\mathcal{C}_{state}$ depend only on the chosen VM architecture.

Even if the use of specific evaluation domains for each constraint polynomial already allows us to write an AIR that verifies the computation, each computation would require a different set of evaluation domains.

To obtain a general-purpose proof system with fixed domains, our goal is to unify all the evaluation domains into the single domain H.

4.4.1 Preprocessed AIR

To ease the encoding of instructions without using specific evaluation domains, we introduce a probabilistically sound extension that utilizes public and preprocessed columns to encode which instruction is active at time t and activate the corresponding constraints.

Remark 4.4.1 (Literature)
In literature as seen in [Gab] and [MF23, p. 6] an AIR that uses preprocessed columns is called Preprocessed AIR (PAIR) and, in particular, an AIR that uses preprocessed columns and permutation arguments as in our case is called Randomized AIR with Preprocessing (RAP) [Gab].

The preprocessing technique that we discuss is the one found in [MF23].

In [MF23, p. 6] the paper starts from the ideal case in which all the constraint polynomials have pairwise disjoint evaluation domains. However, in our case the constraint polynomials in the set $\mathcal{C}_i$ associated to the instruction instr_i have all the same evaluation domain $H_{\mathcal{C}_i}$.

So the first step is aggregating all the elements in the set $\mathcal{C}_i$ in one specification $(\psi_i, H_{\mathcal{C}_i})$ that will verify the logic of the entire instruction, this can be done using a probabilistically sound extension.

After receiving from the Verifier a random parameter ζ drawn uniformly from $\mathbb{F}$, for each $\text{instr}_i \in \mathcal{I}$, we build the aggregated constraint polynomial $(\psi_i, H_{\mathcal{C}_i})$ computing ψ_i as a random linear combination of all the constraint polynomials in $\mathcal{C}_i$, that is

$$\psi_i(\mathbf{X}, \mathbf{Y}) = \sum_{j=1}^{c_{tot,i}} \zeta^{j-1} C_{i,j}(\mathbf{X}, \mathbf{Y}).$$

We then obtain an intermediate AIR $\mathcal{AIR}_2$ in which each instruction instr_i is now constrained by a single ψ_i.

Proposition 4.4.2 (Aggregation is a Probabilistically Sound Extension)
Let $\mathcal{AIR}_1$ be an AIR with the instruction constraint specification sets $\mathcal{C}_i$ and consider the AIR $\mathcal{AIR}_2$ obtained by replacing the specifications in $\mathcal{C}_i$ with the specification $(\psi_i, H_{\mathcal{C}_i})$ for each $i \in \{1, \dots, N\}$.

Then $\mathcal{AIR}_1 \preceq^{\epsilon}_{\text{Pr}} \mathcal{AIR}_2$ with $\epsilon = \dfrac{\sum_{i=1}^{N}(c_{tot,i} - 1)}{|\mathbb{F}|}$.

Proof. Consider the aggregation $(\psi_i, H_{\mathcal{C}_i})$ for the set $\mathcal{C}_i$ for some $i \in \{1, \dots, N\}$ and consider the intermediate AIR $\mathcal{AIR}_{\psi_i}$.

We have that $\mathcal{AIR}_1 \preceq^{\epsilon_i}_{\text{Pr}} \mathcal{AIR}_{\psi_i}$ with $\epsilon_i = \dfrac{c_{tot,i} - 1}{|\mathbb{F}|}$.

It suffices to consider the extension function that leaves the trace unchanged

$$\text{Ext}\colon ((x, p), \zeta) \mapsto (x, p)$$

and we verify the conditions

- **Information Preservation:** by construction $x'_{[0:T;1:w]} = x$;
- **Completeness:** $(x, p) \in \mathcal{T}_{\mathcal{AIR}_1} \implies (x, p) \in \mathcal{T}_{\mathcal{AIR}_{\psi_i}}$, this is because

$$\psi_i(\mathbf{W}(h_t), \mathbf{W}(\sigma(h_t))) = \sum_{j=1}^{c_{tot,i}} \zeta^{j-1} C_{i,j}(\mathbf{W}(h_t), \mathbf{W}(\sigma(h_t))) \quad \text{for each } h_t \in H_{\mathcal{C}_i}$$

and given

$$C_{i,j}(\mathbf{W}(h_t), \mathbf{W}(\sigma(h_t))) = 0 \quad \text{for each } h_t \in H_{\mathcal{C}_i}, j \in \{1, \ldots, c_{tot,i}\}$$

we have

$$\psi_i(\mathbf{W}(h_t), \mathbf{W}(\sigma(h_t))) = 0 \quad \text{for each } h_t \in H_{\mathcal{C}_i}.$$

- **Soundness:** if $(x, p) \notin \mathcal{T}_{\mathcal{AIR}_1}$ then we have

$$C_{i,j}(\mathbf{W}(h_t), \mathbf{W}(\sigma(h_t))) \neq 0 \quad \text{for some } h_t \in H_{\mathcal{C}_i}, j \in \{1, \ldots, c_{tot,i}\}$$

so the vector

$$\mathbf{v} = (C_{i,1}(\mathbf{W}(h_t), \mathbf{W}(\sigma(h_t))), \ldots, C_{i,c_{tot,i}}(\mathbf{W}(h_t), \mathbf{W}(\sigma(h_t))))$$

is not the null vector. The aggregated ψ_i, requires that the linear combination of this vector is zero and that is equivalent to specializing Lemma 4.2.6 to confront $\mathbf{v}$ with the null vector. Given that $\mathbf{v} \neq \mathbf{0}$, the lemma guarantees that the probability that the equality is satisfied for a random choice of ζ is

$$\Pr_{\zeta \in \mathbb{F}}[\text{Ext}((x, p), \zeta) \in \mathcal{T}_{\mathcal{AIR}_{\psi_i}}] \leq \epsilon_i,$$

where $\epsilon_i = \dfrac{c_{tot,i} - 1}{|\mathbb{F}|}$.

Repeating the extension for each $\mathcal{C}_i$ we get by transitivity that $\mathcal{AIR}_1 \preceq^{\epsilon}_{\text{Pr}} \mathcal{AIR}_2$ where, as seen in Proposition 2.4.13 proof, we get

$$\epsilon = \sum_{i=1}^{N} \epsilon_i = \frac{\sum_{i=1}^{N}(c_{tot,i} - 1)}{|\mathbb{F}|}.$$

□

Remark 4.4.3 (Notation Differences with [MF23])
We denoted by ψ_i the constraint polynomial while in [MF23, p. 4] they denote by ψ_i the relative check polynomial.

After that aggregation we obtain the set

$$\mathcal{C}_{aggr} = \{(\psi_1, H_{\mathcal{C}_1}), \ldots, (\psi_N, H_{\mathcal{C}_N})\}$$

of aggregated constraint specifications associated to the ISA instructions, where the evaluation domains $H_{\mathcal{C}_i}$ are pairwise disjoint.

We can now add a preprocessed and public column $x_{w+1}(t)$ that acts as the following selector

$$W_{w+1}(h_t) = i \quad \text{if } h_t \in H_{\mathcal{C}_i}$$

by taking

$$x_{w+1}(t) = \sum_{i=1}^{N} i \sum_{z \in H_{\mathcal{C}_i}} \ell_t(z)$$

where $\ell_t(z)$ is the Lagrange basis polynomial

$$\ell_t(h_s) = \delta_{s,t} = \begin{cases} 1 & \text{if } s = t \\ 0 & \text{if } s \neq t \end{cases}.$$

We now define the vanishing polynomial of a set

Definition 4.4.4 (Vanishing Polynomial)
Given a set $I \subset \mathbb{F}$, the vanishing polynomial *of I, denoted by $Z_I(X) \in \mathbb{F}[X]$, is the monic polynomial*

$$Z_I(X) := \prod_{i \in I}(X - i).$$

Using the vanishing polynomial as a switch we can now build the aggregated constraint polynomial

$$\psi(\mathbf{X}', \mathbf{Y}') = \sum_{i=1}^{N} \psi_i(\mathbf{X}, \mathbf{Y}) \cdot Z_{\{1,\dots,N\}\setminus i}(X_{w+1})$$

that activates only the constraint polynomial that corresponds to the index contained in the auxiliary column and has degree

$$\deg \psi \leq \max_{i \in \{1,\dots,N\}} \deg \psi_i + (N-1) \leq \max_{\substack{i \in \{1,\dots,N\},\\ j \in \{1,\dots,c_{tot,i}\}}} \deg C_{i,j} + (N-1) \leq d_{max} + N - 1$$

so the check polynomial has degree

$$\deg \mathrm{Check}_\psi \leq (d_{max} + N - 1) \cdot T.$$

Remark 4.4.5 (Further Optimizations)
There are more advanced techniques to mitigate the degree increase introduced by the vanishing polynomial $Z_{\{1,\dots,N\}\setminus i}$ used in the selector logic.

As seen in [MF23, p. 11], instead of using a single column that acts as a selector with N distinct values, it is possible to encode the instruction index using multiple columns to use a smaller representation basis but this does not guarantee a degree improvement because the improvement is a function of the number of constraints and the basis chosen.

For this reason we will use the model that uses a single selector column.

Replacing the constraint polynomials $(\psi_i, H_{\mathcal{C}_i})$ with the aggregated (ψ, H), that has evaluation domain H, we get $\mathcal{AIR}_3 = (\mathbb{F}, T, w+1, H, \sigma(X), \mathcal{C}_{\mathcal{AIR}_3})$ where

$$\mathcal{C}_{\mathcal{AIR}_3} = \{(\psi, H)\} \cup \mathcal{C}_{perm} \cup \mathcal{C}_{state}$$

contains $c = 1 + 2 \cdot r + c_{tot,\mathcal{B}}$ constraint polynomials.

Proposition 4.4.6 (Preprocessing is a Valid Extension)
Let $\mathcal{AIR}_2$ be the AIR containing the aggregated constraint polynomials $\{(\psi_i, H_{\mathcal{C}_i})\}_{i=1}^N$ and consider the final AIR $\mathcal{AIR}_3$ with the constraint ψ.

We have

$$\mathcal{AIR}_2 \preceq \mathcal{AIR}_3$$

Proof. Let $x_{w+1}(t) = \sum_{i=1}^{N} i \sum_{z \in H_{\mathcal{C}_i}} \ell_t(z)$ and consider the extension function

$$\mathrm{Ext} : (x, p) \mapsto (x', p)$$

where $x' = (x, x_{w+1})$ then we have

- **Information Preservation:** by construction $x'_{[0:T;1:w]} = x$;
- **Validity Equivalence:** $(x, p) \in \mathcal{T}_{\mathcal{AIR}_2} \iff \mathrm{Ext}(x, p) \in \mathcal{T}_{\mathcal{AIR}_3}$, because

$$\psi(\mathbf{W}'(h_t), \mathbf{W}'(\sigma(h_t))) = \sum_{i=1}^{N} \psi_i(\mathbf{W}(h_t), \mathbf{W}(\sigma(h_t))) \cdot Z_{\{1,\dots,N\}\setminus i}(W_{w+1}(h_t)) = 0$$

  if and only if $\psi_i(\mathbf{W}(h_t), \mathbf{W}(\sigma(h_t))) = 0$ for $W_{w+1}(h_t) = i$.

□

Using transitivity we get $\mathcal{AIR}_1 \preceq_{\mathrm{Pr}}^{\epsilon_{ISA}} \mathcal{AIR}_3$ with $\epsilon_{ISA} = \frac{\sum_{i=1}^{N}(c_{tot,i}-1)}{|\mathbb{F}|}$.

4.4.2 Boolean Selectors

Let $\mathcal{AIR} = (\mathbb{F}, T, w, H, \sigma(X), \mathcal{C})$ be an AIR associated to an ISA with constraint specifications

$$\mathcal{C} = \{(\psi, H)\} \cup \mathcal{C}_{perm} \cup \mathcal{C}_{state}$$

obtained as in the previous section. Our goal is to unify the evaluation domains also for

$$\mathcal{C}' = \mathcal{C}_{perm} \cup \mathcal{C}_{state}.$$

We denote by $c_{tot,\mathcal{C}'}$ the total number of constraint specifications in $\mathcal{C}'$ and for each $(\mathcal{C}'_i, H_{\mathcal{C}'_i})$ we add a preprocessed auxiliary column that acts as a boolean selector. We modify the constraint polynomials as follows:

- we modify the original constraint polynomials multiplying each one by the variable of the corresponding boolean selector column
$$(X_{w+i} \cdot \mathcal{C}'_i, H);$$
- we then ensure that the auxiliary columns contain only boolean values by adding the specifications
$$(X_{w+i}(X_{w+i} - 1), H) \text{ for } i \in \{1, \dots, c_{tot,\mathcal{C}'}\}.$$

We get a new AIR $\mathcal{AIR}_{sel} = (\mathbb{F}, T, w + c_{tot,\mathcal{C}'}, H, \sigma(X), \mathcal{C}_{sel})$ where

$$\mathcal{C}_{sel} = \{(\psi, H)\} \cup \{(X_{w+i} \cdot \mathcal{C}'_i, H), (X_{w+i}(X_{w+i} - 1), H) \mid i \in \{1, \dots, c_{tot,\mathcal{C}'}\}\}.$$

The new constraints are logically equivalent to the old ones so it is easy to prove that $\mathcal{AIR} \preceq \mathcal{AIR}_{sel}$.

4.4.3 Complete RAP Trace for an ISA

We can now have a complete vision on the model we built, we reintroduce the explicit distinction between the various types of trace polynomials getting rid of the simplifying assumptions that we adopted since Remark 4.2.9.

If we denote by sel_{ISA} the trace polynomial for the column of the ISA selector, by $sel_1, \dots, sel_{c_{tot,\mathcal{C}'}}$ the trace polynomials for the columns containing the boolean selectors and with w_{perm} the number of permuted columns necessary for the permutation arguments we obtain for the entire extended trace

$$\Bigg(\underbrace{\overbrace{W_1, \dots, W_{w+w_{perm}+w_{tot}+w_{tot,\mathcal{B}}}^{\text{Base Trace + Permuted Columns + Auxiliary Columns}} \mid \overbrace{Z_1, \dots, Z_r}^{\text{Permutation Arguments}}}_{\text{Private Witness (to commit in two steps)}} \mid \underbrace{\overbrace{sel_{ISA}, sel_1, \dots, sel_{c_{tot,\mathcal{C}'}}}^{\text{Selectors}}}_{\text{Public Columns}} \Bigg).$$

So the trace polynomials can be divided in

$$\begin{aligned} \mathbf{W} &= (W_1, \dots, W_{w+w_{perm}+w_{tot}+w_{tot,\mathcal{B}}}) \\ \mathbf{Z} &= (Z_1, \dots, Z_r) \\ \mathbf{sel} &= (sel_{ISA}, sel_1, \dots, sel_{c_{tot,\mathcal{C}'}}) \end{aligned}.$$

Recalling that the check polynomials depend on the whole extended trace, if we denote by $\mathcal{C}$ the final set of constraints, the check polynomials are such that

$$Check_{C_i}(X) = C_i((\mathbf{W}(X), \mathbf{Z}(X), \mathbf{sel}(X)), (\mathbf{W}(\sigma(X)), \mathbf{Z}(\sigma(X)), \mathbf{sel}(\sigma(X)))).$$

To simplify notation and ease the exposition, we will again use $\mathbf{W}$ to refer to the whole private witness $(\mathbf{W}, \mathbf{Z})$ and when we will discuss the commitment to $\mathbf{W}$, we will be referring to the commitments made to $\mathbf{W}$ and $\mathbf{Z}$ separately.

Chapter 5

Encoding And Proximity Proof

This chapter is the final link between the algebraic representation and the cryptographic proof.

Starting from the check polynomials we will aggregate them doing a random linear combination and then we will compute a quotient polynomial, we will then decompose this quotient to obtain a set of low degree polynomials called *trace quotient polynomials*.

The protocol then has two phases that togheter guarantee the computation correctness:

- the first phase will end with a consistency check that uses the *DEEP (Domain Extending for Eliminating Pretenders)* technique to verify, at a single random point z, the consistency between the trace polynomials and the trace quotient polynomials;
- the second phase consists in using the *FRI (Fast Reed-Solomon Interactive Oracle Proof of Proximity)* protocol to guarantee the authenticity of the evaluations submitted by the Prover and to check that the polynomials involved are of low degree.

5.1 The Quotient Polynomial

Let $\mathcal{AIR} = (\mathbb{F}, T, w, H, \sigma(X), \mathcal{C})$ be an AIR associated to an ISA that uses all the optimizations we discussed in the previous chapters, given that we have all local constraint polynomials the constraint associated to (C_i, H) for $i \in \{1, \dots, c\}$ is that the check polynomial

$$Check_{C_i}(X) = C_i(\mathbf{W}(X), \mathbf{W}(\sigma(X)))$$

vanishes over H.

This can be translated into a divisibility condition.

Proposition 5.1.1 (Vanishing Criterion)
Let $H \subset \mathbb{F}$ and let $P(X) \in \mathbb{F}[X]$, then $P(h) = 0$ for each $h \in H$ if and only if $Z_H(X) \mid P(X)$.

Proof. We prove the two implications

$\Longrightarrow$ Divide P for Z_H then

$$P = Q \cdot Z_H + R \quad \text{with } \deg R < |H|,$$

then evaluating in $h \in H$ we obtain $R(h) = 0$ for each $h \in H$ and then $R \equiv 0$.

$\Longleftarrow$ If $Z_H(X) \mid P(X)$ then $P = Q \cdot Z_H + R$ and evaluating in $h \in H$ we have the conclusion.

□

Following [AMGM24, p. 21], we can define the polynomials

$$q_i(X) := \frac{Check_{C_i}(X)}{Z_H(X)}$$

Recalling that

$$\deg Check_{C_i} = \deg(C_i) \cdot \deg(\mathbf{W}) \leq (d_{max} + N - 1) \cdot T,$$

proving that $Check_{C_i}(X)$ vanishes over H is equivalent to proving that the quotient q_i is a polynomial of degree

$$\deg q_i \leq (d_{max} + N - 1) \cdot T - |H| = (d_{max} + N - 1) \cdot T - (T + 1) = (d_{max} + N - 2) \cdot T - 1$$

for each $i \in \{1, \ldots, c\}$.

To reduce all to only one verification, the Prover commits the trace polynomials $\mathbf{W}$ on the extended domain D and, after receiving from the Verifier a random α uniformly drawn from $\mathbb{F}$, he does a random linear combination.

Definition 5.1.2 (Quotient Polynomial)
We call quotient polynomial *the polynomial*

$$Q(X) := \sum_{i=1}^{c} \alpha^{i-1} q_i(X)$$

obtained through a random linear combination of the quotients $q_i(X) := \frac{Check_{C_i}(X)}{Z_H(X)}$.

Remark 5.1.3 (Degree Adjustment)
We have chosen to avoid the degree adjustment, that is described in [Tea21, p. 16] and [AMGM24, p. 14], to follow the more modern approach of [AMGM24, p. 21]. This forced us to unify all the evaluation domains to H using boolean selectors. Using the degree adjustment, one can keep the original set of constraints

$$\{(\psi, H)\} \cup \mathcal{C}_{perm} \cup \mathcal{C}_{state}.$$

This adds a soundness error that we derive in the following proposition.

Proposition 5.1.4 (Completeness and Soundness of Quotient Polynomial)
Given $\mathcal{AIR} = (\mathbb{F}, T, w, H, \sigma(X), \mathcal{C})$, *let* $(x, p) \in \mathbb{F}^{(T+1)\times w}$ *be a candidate discrete witness and let* $Q(X)$ *be the quotient polynomial derived from the candidate* $\mathbf{W}$ *that interpolates the columns of* x *over* H. *Then we have*

- ***Completeness :***

$$(x, p) \in \mathcal{T}_{\mathcal{AIR}} \implies Q(X) \in \mathbb{F}[X] \text{ and } \deg Q \leq (d_{max} + N - 2) \cdot T - 1$$

- ***Soundness:*** *If* $(x, p) \notin \mathcal{T}_{\mathcal{AIR}}$ *then*

$$\Pr_{\alpha \in \mathbb{F}} [\{Q(X) \in \mathbb{F}[X]\} \cap \{\deg Q \leq (d_{max} + N - 2) \cdot T - 1\}] \leq \frac{c - 1}{|\mathbb{F}|}$$

Proof. Let $d = (d_{max} + N - 2) \cdot T - 1$ and we prove the two properties

- **Completeness :** if $(x, p) \in \mathcal{T}_{\mathcal{AIR}}$, then the polynomial $Check_{C_i}(X)$ vanishes over the trace domain H and so $Z_H(X) \mid Check_{C_i}(X)$ that implies that q_i is a polynomial with

$$\deg q_i = \deg Check_{C_i} - \deg(Z_H) \leq (d_{max} + N - 2) \cdot T - 1$$

for each $i \in \{1, \ldots, c\}$ that is the conclusion.

- **Soundness:** We denote by A the event $Q(X) \in \mathbb{F}[X]$ and with B the event $\deg Q \leq d$ then using the conditional probability formula we get

$$\Pr_{\alpha \in \mathbb{F}}[B \cap A] = \Pr_{\alpha \in \mathbb{F}}[B|A] \Pr_{\alpha \in \mathbb{F}}[A] \leq \Pr_{\alpha \in \mathbb{F}}[A].$$

So it suffices to bound the probability $\Pr_{\alpha \in \mathbb{F}}[A]$.
If $(x, p) \notin \mathcal{T}_{\mathcal{AIR}}$, there exists at least one index $j_0 \in \{1, \ldots, c\}$ and a point $h_0 \in H_{C_{j_0}}$ such that $Check_{C_{j_0}}(h_0) \neq 0$. This implies that the term $q_{j_0}(X) = \frac{Check_{C_{j_0}}(X)}{Z_H(X)}$ is a rational function with a factor $\frac{1}{X - h_0}$.
Being that $\deg Z_H(X) \leq \deg \text{Check}_{C_i}(X)$ we have

$$\text{Check}_{C_i}(X) = \text{Quotient}_i(X) \cdot Z_H(X) + \text{Remainder}_i(X)$$

where $\deg(\text{Remainder}_i) < \deg(Z_H)$. Therefore we can write $q_i(X)$ as

$$q_i(X) = \text{Quotient}_i(X) + \frac{\text{Remainder}_i(X)}{Z_H(X)}$$

Now we can apply the partial fraction decomposition [vzGG99, p. 119] to

$$\frac{\text{Remainder}_i(X)}{Z_H(X)} = \sum_{h \in H} \frac{k_{i,h}}{X - h}$$

To find a coefficient $k_{i,\bar{h}}$ we multiply both sides by $(X - \bar{h})$

$$\frac{\text{Remainder}_i(X)}{Z_H(X)/(X - \bar{h})} = k_{i,\bar{h}} + (X - \bar{h}) \cdot (\text{other fractions})$$

Using the formal derivative $Z'_H(X, \bar{h}) = \frac{Z_H(X) - Z_H(\bar{h})}{X - \bar{h}}$ and given that $Z_H(\bar{h}) = 0$ we can evaluate in $X = \bar{h}$ to isolate $k_{i,\bar{h}}$

$$k_{i,\bar{h}} = \frac{\text{Remainder}_i(\bar{h})}{Z'_H(\bar{h}, \bar{h})}$$

From the initial polynomial division we can evaluate in $\bar{h}$ and being $Z_H(\bar{h}) = 0$, we obtain

$$\text{Check}_{C_i}(h) = \text{Remainder}_i(\bar{h})$$

and so

$$k_{i,\bar{h}} = \frac{\text{Check}_{C_i}(\bar{h})}{Z'_H(\bar{h}, \bar{h})}.$$

So we have

$$Q(X) = \sum_{i=1}^{c} \alpha^{i-1} q_i(X) = \sum_{i=1}^{c} \alpha^{i-1} \left(\text{Quotient}_i(X) + \sum_{h \in H} \frac{k_{i,h}}{X - h} \right)$$

and to have that $Q(X)$ is a polynomial, it must be

$$\sum_{i=1}^{c} \alpha^{i-1} \sum_{h \in H} \frac{k_{i,h}}{X - h} = 0$$

so reordering the sum

$$\sum_{h\in H}\frac{1}{X-h}\sum_{i=1}^{c}\alpha^{i-1}\cdot k_{i,h}=0$$

that vanishes if and only if for each $h\in H$ we have

$$\sum_{i=1}^{c}\alpha^{i-1}\cdot k_{i,h}=0.$$

In particular, if we define

$$F_h(X):=\sum_{i=1}^{c}X^{i-1}\cdot k_{i,h}$$

it has $\deg F_h\leq c-1$. Using the Schwartz-Zippel lemma we obtain that the probability that a random α drawn uniformly from $\mathbb{F}$ is a root of $F_{h_0}(\alpha)$ is

$$\Pr_{\alpha\in\mathbb{F}}[F_{h_0}(\alpha)=0]\leq\frac{\deg(F_{h_0})}{|\mathbb{F}|}\leq\frac{c-1}{|\mathbb{F}|}.$$

Now the event A can happen only if $F_{h_0}(\alpha)=0$ and so we have

$$\Pr_{\alpha\in\mathbb{F}}[A]=\Pr_{\alpha\in\mathbb{F}}[A\cap\{F_{h_0}(\alpha)=0\}]\leq\Pr_{\alpha\in\mathbb{F}}[F_{h_0}(\alpha)=0]\leq\frac{c-1}{|\mathbb{F}|}.$$

□

Remark 5.1.5 (Complete Analysis)
Proposition 5.1.4 guarantees that if a Prover uses a witness $\mathbf{W}(X)$ *that does not satisfy the AIR constraints, the resulting* $Q(X)$ *will not be a low degree polynomial and this will be detected with high probability.*

However that analysis assumes, as seen in Remark 4.1.11, that the prover uses the same witness $\mathbf{W}(X)$ *for the initial commitment and to build the quotient.*

A full analysis will be presented in Section 5.3.3.

Now the Prover, instead of directly convincing the Verifier that $\mathcal{W}_{\mathcal{AIR}}\neq\emptyset$, can convince him that $Q(X)$ is a polynomial of degree less than d.

To reach that goal we need Reed Solomon codes.

5.2 Reed Solomon Codes

In Chapter 3 we introduced the discrete Fourier transform, we can use the DFT to directly define Reed Solomon codes in the particular case where the evaluation domain is the set of Fourier nodes, we follow [Giu06, p. 180].

Let $n,k\in\mathbb{Z}^+$ with $k\leq n$ and consider the polynomials with coefficients in $\mathbb{F}_q$ of degree less than k, we denote this set as

$$R_k[\mathbb{F}_q]:=\{m(X)\in\mathbb{F}_q[X]\mid\deg(m)<k\}.$$

Each polynomial $m(X)\in R_k[\mathbb{F}_q]$ can be represented using the vector of its coefficients padded to lenght n that we call *messages*

$$\mathbf{m}=(m_0,m_1,\ldots,m_{k-1},0,\ldots,0)^{\mathrm{T}}\in\mathbb{F}_q^n.$$

We can then define the set of all messages

Definition 5.2.1 (Message Space)
Let $\mathbb{F}_q$ be a finite field and let $n, k \in \mathbb{Z}^+$ with $k \leq n$.
The message space *$\mathcal{M}_k$ is the vector subspace*

$$\mathcal{M}_k := \{\mathbf{m} = (m_0, m_1, \ldots, m_{k-1}, 0, \ldots, 0)^{\mathrm{T}} \in \mathbb{F}_q^n\}.$$

We can now define the encoding map

Definition 5.2.2 (Reed Solomon Encoding)
Let $\mathbb{F}_q$ be a finite field and let $n, k \in \mathbb{Z}^+$ with $k \leq n$ and such that there exists a primitive n-th root of unity $\omega_n \in \mathbb{F}_q$. The Reed-Solomon encoding *is the restriction of the DFT to the message space*

$$\begin{aligned} DFT_n|_{\mathcal{M}_k} \colon \mathcal{M}_k &\to \mathbb{F}_q^n \\ \mathbf{m} &\mapsto V_{\omega_n}\mathbf{m} \end{aligned}.$$

So we can define Reed-Solomon codes

Definition 5.2.3 (Reed-Solomon Code)
Let $\mathbb{F}_q$be a finite field and let $n, k \in \mathbb{Z}^+$ with $k \leq n$ and such that there exists a primitive n-th root of unity $\omega_n \in \mathbb{F}_q$.
A Reed-Solomon code $\mathrm{RS}(n, k)$ *of lenght n and dimension k is the image of the encoding map, that is the following vector subspace of $\mathbb{F}_q^n$*

$$\mathrm{RS}(n, k) = \left\{ V_{\omega_n} \cdot \mathbf{m} \mid \mathbf{m} = (m_0, \ldots, m_{k-1}, 0, \ldots, 0)^{\mathrm{T}} \in \mathbb{F}_q^n \right\}.$$

The elements $\mathbf{c} \in \mathrm{RS}(n, k)$ are called codewords.

Given that each codeword $\mathbf{c} \in \mathrm{RS}(n, k)$ is the image of a polynomial $m(X)$ of degree $\deg m < k$ we have that by Lagrange interpolation $m(X)$ is uniquely determined by k evaluations, so the other $n - k$ components $\mathbf{c}$ contain redundant information.
So we can formally define redundancy [Giu06, p. 40]

Definition 5.2.4 (Redundancy)
Let $\mathrm{RS}(n, k)$ be a Reed-Solomon code, we call redundancy *the quantity*

$$r := n - k.$$

We now define the Hamming distance [Giu06, p. 36]

Definition 5.2.5 (Hamming Distance)
Let $\mathbb{F}_q$ be a finite field and let $n \in \mathbb{Z}^+$, given $\mathbf{p}, \mathbf{q} \in \mathbb{F}_q^n$ the Hamming distance *$d_H(\mathbf{p}, \mathbf{q})$ is the number of components in which $\mathbf{p}$ and $\mathbf{q}$ differ*

$$d_H(\mathbf{p}, \mathbf{q}) := |\{i \mid p_i \neq q_i\}|.$$

Proposition 5.2.6 (Well-defined d_H)
The Hamming distance d_H is a distance.

Proof. Let $\mathbf{p}, \mathbf{q}, \mathbf{r} \in \mathbb{F}_q^n$, we prove that the distance properties hold:

- $d_H(\mathbf{p}, \mathbf{q}) \geq 0$ because we are counting elements;
- $d_H(\mathbf{p}, \mathbf{q}) = 0$ if and only if $\mathbf{p} = \mathbf{q}$, because by definition $d_H(\mathbf{p}, \mathbf{q}) = 0$ if and only if all the components are equal;

- $d_H(\mathbf{p}, \mathbf{q}) = d_H(\mathbf{q}, \mathbf{p})$ because the number of elements that differ does not change;
- $d_H(\mathbf{p}, \mathbf{r}) \leq d_H(\mathbf{p}, \mathbf{q}) + d_H(\mathbf{q}, \mathbf{r})$ because we have

  $$d_H(\mathbf{p}, \mathbf{r}) = |\{i \mid p_i \neq r_i\}|$$

  and

  $$d_H(\mathbf{p}, \mathbf{q}) + d_H(\mathbf{q}, \mathbf{r}) = |\{i \mid p_i \neq q_i\}| + |\{i \mid q_i \neq r_i\}| \geq |\{i \mid p_i \neq q_i\} \cup \{i \mid q_i \neq r_i\}|.$$

  So it suffices to prove that

  $$\{i \mid p_i \neq r_i\} \subseteq \{i \mid p_i \neq q_i \text{ or } q_i \neq r_i\}$$

  but this is true because if $p_i \neq r_i$, then it is not possible that $p_i = q_i$ and $q_i = r_i$ at the same time and so we have $p_i \neq q_i$ or $q_i \neq r_i$.

□

We now define the minimum distance for a subset [Giu06, p. 41]

Definition 5.2.7 (Minimum Distance)
Let $\mathbb{F}_q$ be a finite field and let $n \in \mathbb{Z}^+$, given a subset $S \subseteq \mathbb{F}_q^n$ the minimum distance *of S is the minimum Hamming distance between two elements of S*

$$d_H(S) := \min_{\substack{\mathbf{p},\mathbf{q} \in S \\ \mathbf{p} \neq \mathbf{q}}} d_H(\mathbf{p}, \mathbf{q}).$$

We now prove an important property [Giu06, p. 45]

Proposition 5.2.8 (Singleton Bound)
If $S \subseteq \mathbb{F}_q^n$ has cardinality $|S| = q^k$ we have that

$$d_H(S) \leq n - (k - 1).$$

Proof. It suffices to prove that $|S| \leq |\mathbb{F}_q|^{n-(d_H(S)-1)}$, that is $q^k \leq q^{n-d_H(S)+1}$ so taking the base q logarithm we obtain the conclusion.

Consider the set S' built using the elements $\mathbf{s} \in S$ where we removed the last $d_H(S) - 1$ coordinates, by construction we have $|S| = |S'| = q^k$.

Each $\mathbf{s}' \in S'$ has lenght $n - (d_H(S) - 1)$ and the elements in S' are all distinct.

That's because if we suppose by contadiction that there are two $\mathbf{s}'_1, \mathbf{s}'_2 \in S'$ such that $\mathbf{s}'_1 = \mathbf{s}'_2$, this means that they coincide on $n - (d_H(S) - 1)$ positions and then there are $\mathbf{s}_1, \mathbf{s}_2 \in S$ that coincide on the first $n - (d_H(S) - 1)$ positions, that is $d_H(\mathbf{s}_1, \mathbf{s}_2) = d_H(S) - 1$. This is absurd given the definition of minimum distance $d_H(S)$.

We obtained that $|S'| \leq q^{n-(d_H(S)-1)}$ that implies the conclusion. □

Definition 5.2.9 (Maximum Distance Separable)
Let $\mathbb{F}_q$ be a finite field and let $n, k \in \mathbb{Z}^+$ with $k \leq n$, given a subset $S \subseteq \mathbb{F}_q^n$ of dimension $|S| = q^k$ we say that it is Maximum Distance Separable (MDS) *if*

$$d_H(S) = n - (k - 1).$$

We now prove that Reed-Solomon codes maximize the minimum distance [Giu06, p. 178].

Proposition 5.2.10 (RS codes are MDS)
Let $\mathbb{F}_q$ be a finite field and let $n, k \in \mathbb{Z}^+$ with $k \leq n$ and such that there exists a primitive n-th root of unity $\omega_n \in \mathbb{F}_q$. We have

$$d_H(\mathrm{RS}(n,k)) = n - (k-1).$$

Proof. By Proposition 5.2.8 it suffices to prove

$$d_H(\mathrm{RS}(n,k)) \geq n - (k-1).$$

Let $m_1(X), m_2(X) \in R_k[\mathbb{F}_q]$ be two distinct elements and consider

$$r(X) = m_1(X) - m_2(X) \in R_k[\mathbb{F}_q],$$

given that $r \not\equiv 0$ and $\deg r(X) < k$ we obtain that it has maximum $k-1$ roots.

If we consider messages $\mathbf{m_1}$ and $\mathbf{m_2}$, then the respective codewords $\mathbf{c_1} = V_{\omega_n}\mathbf{m_1}$ and $\mathbf{c_2} = V_{\omega_n}\mathbf{m_2}$ coincide on at most $k-1$ positions, therefore

$$d_H(\mathrm{RS}(n,k)) = \min_{\substack{\mathbf{c_1},\mathbf{c_2} \in \mathrm{RS}(n,k) \\ \mathbf{c_1} \neq \mathbf{c_2}}} d_H(\mathbf{c_1}, \mathbf{c_2}) \geq n - (k-1).$$

□

Remark 5.2.11 (Different Polynomials)
The MDS property of Reed-Solomon codes is what allows us to efficiently distinguish between different polynomials because even the smallest difference between two vectors of coefficients, that is Hamming distance 1*, is amplified in a distance of at least $n - k + 1$ between the relative codewords.*

We need to prove a corollary [Giu06, pp. 43-44] but we first need Hamming balls

Definition 5.2.12 (Hamming Ball)
Let $\mathbb{F}_q$ be a finite field and let $n \in \mathbb{Z}^+$, given $\mathbf{c} \in \mathbb{F}_q^n$ the Hamming ball *$B_H(\mathbf{c}, r)$ of radius r and center $\mathbf{c} \in \mathbb{F}_q^n$ is the set*

$$B_H(\mathbf{c}, r) := \{\mathbf{x} \in \mathbb{F}_q^n \mid d_H(\mathbf{x}, \mathbf{c}) \leq r\}.$$

Corollary 5.2.13 (Unique Decoding)
Given a Reed Solomon code $\mathrm{RS}(n,k)$, a vector $\mathbf{x} \in \mathbb{F}_q^n$ and the radius $t = \left\lfloor \frac{d_H(\mathrm{RS}(n,k))-1}{2} \right\rfloor$, if there is codeword $\mathbf{c} \in \mathrm{RS}(n,k)$ such that $\mathbf{x} \in B_H(\mathbf{c}, t)$, that codeword $\mathbf{c}$ is unique.

Proof. Let $\mathbf{c}_1, \mathbf{c}_2 \in \mathrm{RS}(n,k)$ be such that $\mathbf{c}_1 \neq \mathbf{c}_2$ and suppose by contradiction that we have $\mathbf{x} \in B_H(\mathbf{c}_1, t)$ and $\mathbf{x} \in B_H(\mathbf{c}_2, t)$. By the triangular inequality

$$d_H(\mathbf{c}_1, \mathbf{c}_2) \leq d_H(\mathbf{x}, \mathbf{c}_1) + d_H(\mathbf{x}, \mathbf{c}_2) \leq 2t = 2\left\lfloor \frac{d_H(\mathrm{RS}(n,k)) - 1}{2} \right\rfloor \leq d_H(\mathrm{RS}(n,k)) - 1$$

that is absurd given the MDS property. □

So we define

Definition 5.2.14 (Unique Decoding Radius)
We call unique decoding radius *for the code $\mathrm{RS}(n,k)$ the radius $t = \left\lfloor \frac{d_H(\mathrm{RS}(n,k))-1}{2} \right\rfloor$.*

What we proved can also be seen in terms of fractional distance [Giu06, p. 42], [Hab22, p. 3]

Definition 5.2.15 (Fractional Hamming Distance)
Let $\mathbb{F}_q$ be a finite field and let $n \in \mathbb{Z}^+$, given $\mathbf{p}, \mathbf{q} \in \mathbb{F}_q^n$ the fractional Hamming distance *$\delta_H(\mathbf{p}, \mathbf{q})$ is the ratio between the Hamming distance and the vector lenght*

$$\delta_H(\mathbf{p}, \mathbf{q}) := \frac{d_H(\mathbf{p}, \mathbf{q})}{n}.$$

we can then define the fractional unique decoding radius

Definition 5.2.16 (Fractional Unique Decoding Radius)
We call fractional unique decoding radius *for the code* $\mathrm{RS}(n, k)$ *the radius*

$$\theta_0 := \frac{t}{n} = \frac{\left\lfloor \frac{d_H(\mathrm{RS}(n,k))-1}{2} \right\rfloor}{n}.$$

We can then define the fractional distance between a vector and a set

Definition 5.2.17 (Fractional Distance Vector-Set)
Let $\mathbb{F}_q$ be a finite field and let $n \in \mathbb{Z}^+$, given a subset $S \subseteq \mathbb{F}_q^n$ and $\mathbf{p} \in \mathbb{F}_q^n$ the fractional distance of $\mathbf{p}$ from S *is the minimum fractional Hamming distance between the two*

$$\delta_H(\mathbf{p}, S) := \min_{\mathbf{q} \in S} \delta_H(\mathbf{p}, \mathbf{q}).$$

We can now define proximity

Definition 5.2.18 (θ-near)
Let $\mathbb{F}_q$ be a finite field and let $n \in \mathbb{Z}^+$, given a subset $S \subseteq \mathbb{F}_q^n$ and $\mathbf{p} \in \mathbb{F}_q^n$ we say that $\mathbf{p}$ is θ-near *to S if*

$$\delta_H(\mathbf{p}, S) \leq \theta.$$

We say that it is θ-far *otherwise.*

We now state a proximity criterion to determine if a vector is an evaluation of a low-degree polynomial.

Proposition 5.2.19 (Proximity Criterion)
Let $\mathrm{RS}(n, k)$ be a Reed Solomon code, let θ_0 be its fractional unique decoding radius and let $\mathbf{r} \in \mathbb{F}_q^n$. If

$$\delta_H(\mathbf{r}, \mathrm{RS}(n, k)) \leq \theta_0$$

then there exists a unique polynomial $m(X) \in R_k[\mathbb{F}_q]$ where the associated codeword $\mathbf{c} \in \mathrm{RS}(n, k)$ satisfies $\delta_H(\mathbf{r}, \mathbf{c}) \leq \theta_0$.

Proof. Given that $\mathbf{r}$ is θ_0-close to the code $\mathrm{RS}(n, k)$ by definition there exists a codeword $\mathbf{c} \in \mathrm{RS}(n, k)$ such that $\delta_H(\mathbf{r}, \mathbf{c}) \leq \theta_0$ so being $t = n\theta_o$ we have $d_H(\mathbf{r}, \mathbf{c}) \leq t$ and using Corollary 5.2.13 we have that such a $\mathbf{c}$ is unique.

Given that the DFT is invertible we have that there is a unique $\mathbf{m} = V_{\omega_n}^{-1}\mathbf{c}$ and given that $\mathbf{c} \in \mathrm{RS}(n, k)$ we must have

$$\mathbf{m} = (m_0, m_1, \ldots, m_{k-1}, 0, \ldots, 0)^{\mathrm{T}} \in \mathbb{F}_q^n.$$

So the unique polynomial

$$m(X) = \sum_{i=0}^{k-1} m_i X^i \in R_k[\mathbb{F}_q].$$

□

The criterion just proven allows the Verifier to relax the verification by testing for θ_0-proximity instead of directly checking the membership in the code.

5.3 Towards the Proof of Proximity

The Prover's objective is to convince the Verifier that the quotient polynomial $Q(X)$ has degree $\deg Q \leq d$. He could choose $n \geq d+1$ such that $n \mid (q-1)$ and commit to a vector $\mathbf{c} \in \mathbb{F}_q^n$ of evaluations of $Q(X)$, but that would be inefficient because

$$\deg Q \leq (d_{max} + N - 2) \cdot T - 1.$$

To reduce the number of necessary operations, we split $Q(X)$ using the quotient trace polynomials [AMGM24, p. 14].

Definition 5.3.1 (Quotient Trace Polynomials)
Let $S = d_{max} + N - 2$, we call quotient trace polynomials *the polynomials $Q_j(X)$ of degree $\deg Q_j(X) \leq \left\lfloor \frac{ST-1}{S} \right\rfloor = \left\lfloor T - \frac{1}{S} \right\rfloor = T - 1$ such that*

$$Q(X) = \sum_{j=1}^{S} X^{j-1} Q_j(X^S).$$

We will denote by $\mathbf{Q}$ the quotient trace polynomials vector.

So instead of directly convincing the Verifier that $\deg Q \leq d$, the Prover can convince him that $\deg Q_j \leq T - 1$ for each $j \in \{1, \ldots, S\}$.

We prove that this strategy has a lower time complexity.

Proposition 5.3.2 (Time Complexity Comparison)
Let $S > 4$ and let $n \in \mathbb{N}$ be the minimum power of two such that $n \geq ST$. The evaluation of the trace quotient polynomials $Q_j(X)$ using $S > 4$ Radix-2 FFTs, each with domain of cardinality $T + 1$, has lower time complexity than evaluating $Q(X)$ using only one Radix-2 FFT over a domain of n points.

Proof. We analyze the two methods.

- **Without Splitting:** We have that $\deg Q \leq S \cdot T - 1$ where $S = (d_{max} + N - 2)$ and if we denote by C_A the number of coefficients we have

  $$C_A \leq S \cdot T.$$

  If we consider n the minimum power of two such that $n \geq ST$ we have that the time complexity for the Radix-2 FFT associated is $O(n \log_2 n)$ and we have

  $$ST \log_2[ST] \leq n \log_2 n$$

- **With Splitting:** we need to evaluate S polynomials of degree $\deg Q_i \leq T - 1$ so they have maximum T coefficients. The closest power of two is $n' = T + 1$ being that $T + 1$ is a power of two by contruction, then the time complexity is

  $$O(S(T+1) \cdot \log_2[T+1]).$$

The method that uses the splitting will be more efficient if

$$S(T+1) \cdot \log_2[T+1] < ST \log_2[ST]$$

given that $d_{max} \geq 2$ e $N \geq 1$ we have $S \geq 1$ and then splitting also the logarithm

$$(T+1) \cdot \log_2[T+1] < T \cdot (\log_2[S] + \log_2[T]).$$

So for the splitting to be more efficient we must have

$$\log_2 S > \frac{(T+1)}{T} \cdot \log_2[T+1] - \log_2[T].$$

We now prove that $f(T) = \frac{(T+1)}{T} \cdot \log_2[T+1] - \log_2[T]$ is a decreasing function, the derivative is

$$f'(T) = \frac{T-(T+1)}{T^2} \cdot \log_2[T+1] + \frac{T+1}{T} \cdot \frac{1}{(T+1)\ln 2} - \frac{1}{T \ln 2} = -\frac{\log_2[T+1]}{T^2}$$

that is always negative for $T > 0$ and then, given that for $T = 1$ we have

$$\log_2 S > 2 \cdot \log_2[2] - \log_2[1] = 2,$$

the splitting is more efficient when $S > 4$.

□

Remark 5.3.3 (Real Systems)
In the case of a ZK-VM that uses the RV32I architecture we have $N = 40$ instructions [RIS25, p. 23] so if we suppose a minimum degree for the constraint polynomials of $d_{max} = 2$ we get $S = d_{max} + N - 2 = 40$. This means the splitting is always convenient in real scenarios.

5.3.1 Low Degree Extension

The Prover needs to show that $\deg Q_j < T$ for each j, so he chooses n to compute the codeword $\mathbf{c}_j \in \mathrm{RS}(n, T)$ associated with the polynomial $Q_j(X)$.

To use a Radix-2 FFT and to make the codeword more redundant [AMGM24, p. 14], n is chosen as a power of two that is significantly larger than the number of coefficients k.

In particular, we choose $n = \beta(T+1)$ where $\beta = 2^s$ for some $s \in \mathbb{Z}^+$ is called *blowup factor*. This way using the MDS property we have

$$d_H(\mathrm{RS}(\beta(T+1), T)) = \beta(T+1) - (T-1) = \beta(T+1) - T + 1$$

that implies

$$\theta_0 = \frac{t}{n} = \frac{\left\lfloor \frac{\beta(T+1)-T}{2} \right\rfloor}{\beta(T+1)} = \begin{cases} \dfrac{1 - \dfrac{T+1}{\beta(T+1)}}{2} & \text{if } T \text{ is odd} \\ \dfrac{1 - \dfrac{T}{\beta(T+1)}}{2} & \text{if } T \text{ is even} \end{cases}$$

and because T is odd the unique decoding radius becomes $\theta_0 = \dfrac{1 - \frac{1}{\beta}}{2}$ so if we define $\rho := \dfrac{1}{\beta}$ we have

$$\theta_0 = \frac{1-\rho}{2}.$$

The codeword redundancy grows with the blowup factor β because for big values of T we have $\rho \approx \dfrac{k}{n}$ and so

$$1 - \rho \approx \frac{n-k}{n} = \frac{r}{n}$$

is a measure of the percentage of redundant terms.

Once the cardinality for the extended evaluation domain D has been chosen, we need to choose the extended evaluation domain such that $D \cap H = \emptyset$, this is because

$$Q(X) = \sum_{i=1}^{c} \alpha^{i-1} \frac{Check_{C_i}(X)}{Z_H(X)}$$

and so the evaluation is not well-defined on H being $Z_H(h) = 0$ for each $h \in H$.

Therefore a coset of a cyclic multiplicative subgroup of order n is chosen [Tea21, p. 14] utilizing a generator g of the multiplicative group $\mathbb{F}_q^*$ [Sze], for example we can choose $\langle \omega_n \rangle$ and $g = \omega_{q-1}$ and consider

$$D = \omega_{q-1}\langle \omega_n \rangle = \{\omega_{q-1}\omega_n^i \mid i \in \{0, \dots, n-1\}\}.$$

That's because ω_{q-1} has order $q-1$ and then it can't be in $H = \langle \omega_{T+1} \rangle$ or in $\langle \omega_n \rangle$ because $q-1$ does not divide n.

At last, if we assume by contradiction that $\omega_{q-1}\omega_n^i \in \langle \omega_n \rangle$ then we would have $\omega_{q-1}\omega_n^i = \omega_n^j$ and then $\omega_{q-1} = \omega_n^{j-1} \in \langle \omega_n \rangle$ that is absurd.

In the commitment procedures we will have to use an algebraic trick to use the FFT, if we want to commit a polynomial $P(X) = \sum_i a_i X^i$ over the extended domain, we can compute the FFT over $\langle \omega_n \rangle$ of $P'(X) = P(\omega_{q-1}X) = \sum_i (a_i \omega_{q-1}^i) X^i$ to obtain the evaluations with the same efficiency.

In particular, if we take the matrix $G_n \in \mathbb{F}_q^{n \times n}$ given by

$$G_n = \mathrm{diag}(1, g, g^2, \dots, g^{n-1})$$

we can consider the matrix $V_{\omega_n} G_n$ that does the evaluation over D.

The set of evaluations over the extended domain D is called *Low Degree Extension*.

5.3.2 Consistency Check DEEP

The last step before the proof of proximity is a consistency check [AMGM24, p. 15] between the trace polynomials $\mathbf{W}$ and quotient trace polynomials $Q_i(X)$, so the Prover commits the polynomials $Q_i(X)$ using the evaluations over the extended domain D.

The Verifier can't check all points so he sends a random z drawn uniformly from $\mathbb{F} \setminus (D \cup H)$ and the Prover answers with the evaluations $W_{j_1}(z), W_{j_1}(\sigma(z))$ where the indices $j_1 \in \{1, \dots, w\}$ are all the indices of the columns used in the constraints in $\mathcal{C}$ and the evaluations $Q_j(z^S)$ for each $j \in \{1, \dots, S\}$.

Now the Verifier has the data to verify that

$$\sum_{j=1}^{S} z^{j-1} Q_j(z^S) = \sum_{i=1}^{c} \alpha^{i-1} \frac{Check_{C_i}(z)}{Z_H(z)}.$$

We do a soundness analysis using a similar reasoning to [Tea21, p. 45]

Proposition 5.3.4 (Completeness and Soundness of Consistency Check)
Given $\mathcal{AIR} = (\mathbb{F}, T, w, H, \sigma(X), \mathcal{C})$, let $(x, p) \in \mathbb{F}^{(T+1)\times w}$ be a candidate discrete witness.

Given the quotient trace polynomials $Q_k(X)$ of the quotient polynomial $Q(X)$ built using the candidate $\mathbf{W}$ that interpolates the columns of x over H.

Then we have:

- ***Completeness:*** $(x,p) \in \mathcal{T}_{\mathcal{AIR}} \implies \sum_{j=1}^{S} z^{j-1} Q_j(z^S) = \sum_{i=1}^{c} \alpha^{i-1} \frac{Check_{C_i}(z)}{Z_H(z)}$ *for each* $z \in \mathbb{F}$.
- ***Soundness:*** *If* $(x,p) \notin \mathcal{T}_{\mathcal{AIR}}$ *then*

$$\Pr_{z \in \mathbb{F} \setminus (D \cup H)} \left[\sum_{j=1}^{S} z^{j-1} Q_j(z^S) = \sum_{i=1}^{c} \alpha^{i-1} \frac{Check_{C_i}(z)}{Z_H(z)} \right] \leq \frac{(S+1)T}{|\mathbb{F}| - |D \cup H|}$$

Proof. We prove the two properties

- **Completeness:** If $(x,p) \in \mathcal{T}_{\mathcal{AIR}}$ then by construction

$$\sum_{j=1}^{S} X^{j-1} Q_j(X^S) = Q(X) = \frac{\sum_{i=1}^{c} \alpha^{i-1} \text{Check}_{C_i}(X)}{Z_H(X)}.$$

- **Soundness:** If $(x,p) \notin \mathcal{T}_{\mathcal{AIR}}$ then given that $z \notin H$ we have

$$Z_H(z) \sum_{j=1}^{S} z^{j-1} Q_j(z^S) = \sum_{i=1}^{c} \alpha^{i-1} Check_{C_i}(z)$$

so if we define

$$F(X) := Z_H(X) \sum_{j=1}^{S} X^{j-1} Q_j(X^S) - \sum_{i=1}^{c} \alpha^{i-1} Check_{C_i}(X)$$

we get

$$\Pr_{z \in \mathbb{F} \setminus (D \cup H)} \left[\sum_{j=1}^{S} z^{j-1} Q_j(z^S) = \sum_{i=1}^{c} \alpha^{i-1} \frac{Check_{C_i}(z)}{Z_H(z)} \right] = \Pr_{z \in \mathbb{F} \setminus (D \cup H)} \left[F(z) = 0 \right].$$

Now $\deg F = \max\{\deg(Q \cdot Z_H), \max_i(\deg Check_{C_i})\}$ where

$$\deg(Q \cdot Z_H) = \deg Q + \deg Z_H \leq (ST - 1) + (T + 1) = (S+1)T$$

and

$$\max_i(\deg Check_{C_i}) = \max_i(\deg W_i \cdot \deg C_i) \leq T(S+1)$$

therefore we get

$$\deg F = \max\{(S+1)T, (S+1)T\} = (S+1)T.$$

Using the Schwartz-Zippel lemma we finally have

$$\Pr_{z \in \mathbb{F} \setminus (D \cup H)} \left[F(z) = 0 \right] \leq \frac{(S+1)T}{|\mathbb{F}| - |D \cup H|}$$

□

5.3.3 Soundness Analysis of Quotient Polynomial and Consistency Check

As observed in Remark 5.1.5 the Proposition 5.3.4 gives the guarantee that if the Prover uses a witness $\mathbf{W}(X)$ derived from a non valid trace then the consistency check will fail with high probability.

However, this analysis assumes that the Prover is using the same witness $\mathbf{W}(X)$ in the initial commitment and in the construction of $Q(X)$ and again the same quotient $Q(X)$ in the initial commitment and in the construction of the trace quotient polynomials $Q_j(X)$.

For each phase, a dishonest Prover can choose to commit to an evaluation that corresponds to another low-degree polynomial but is not far enough from the Reed-Solomon code $\mathrm{RS}(n,k)$ to be identified. The probability that it is not detected is therefore

$$\Pr\left[\bigcup_{i=1}^{\ell} E_i\right] \leq \sum_{i=1}^{\ell} \Pr\left[E_i\right]$$

where ℓ is the number of polynomials that are not far enough from the code and E_i is the event that the i-th of such polynomials is not detected. The number ℓ of such polynomials is given by the Johnson Bound Theorem found in [Tea21, p. 30] and is such that

$$\ell = \frac{1}{2\eta\sqrt{\rho}}$$

where $\eta = 1 - \sqrt{\rho} - \gamma$ and γ is the maximum fractional Hamming distance accepted, so we get $\eta \in (0, 1-\sqrt{\rho})$. Therefore we have the lower bound

$$\ell > \frac{1}{2(1-\sqrt{\rho})\sqrt{\rho}}$$

but the actual value is determined by choosing a maximum allowed fractional Hamming distance.

So using Proposition 5.1.4 for the construction of the quotient polynomial we have

$$\Pr_{\alpha\in\mathbb{F}}\left[\bigcup_{i=1}^{\ell} A_i\right] \leq \sum_{i=1}^{\ell} \Pr_{\alpha\in\mathbb{F}}\left[A_i\right] \leq \ell\left(\frac{c-1}{|\mathbb{F}|}\right)$$

therefore $\epsilon_{Quotient} = \ell\left(\frac{c-1}{|\mathbb{F}|}\right)$ and for the consistency check, given that it depends on the choice of $\mathbf{W}(X)$ and of $Q(X)$, using Proposition 5.3.4 we get

$$\Pr_{z\in\mathbb{F}\setminus(D\cup H)}\left[\bigcup_{k=1}^{\ell}\bigcup_{m=1}^{\ell} F_{k,m}(z) = 0\right] \leq \ell^2\frac{(S+1)T}{|\mathbb{F}| - |D\cup H|}$$

so we have $\epsilon_{DEEP} = \ell^2\dfrac{(S+1)T}{|\mathbb{F}| - |D\cup H|}$.

So the pre-FRI phase has soundness error

$$\epsilon_{pre-FRI} = \epsilon_{Quotient} + \epsilon_{DEEP} = \ell\left(\frac{c-1}{|\mathbb{F}|} + \ell\frac{(S+1)T}{|\mathbb{F}| - |D\cup H|}\right).$$

5.3.4 Checking The Evaluations

So it remains only to prove the authenticity of the evaluations sent by the Prover during the pre-FRI phase and the low degree of the polynomials involved. That can be done all at once by checking that the quotients

$$F_{j_1,1}(X) = \frac{W_{j_1}(X) - W_{j_1}(z)}{X - z}$$
$$F_{j_1,2}(X) = \frac{W_{j_1}(\sigma(X)) - W_{j_1}(\sigma(z))}{X - \sigma(z)}$$
$$F_j(X) = \frac{Q_j(X) - Q_j(z^S)}{X - z^S}$$

are respectively polynomials with degree less than T for the trace polynomials and $T-1$ for the trace quotient polynomials.

So the prover computes $F_{j_1,1}, F_{j_1,2}$ and $F_{j,1}$ for each j_1 and j, commits to their LDE and the protocol proceeds with the proximity test.

5.4 FRI Proof of Proximity

We now introduce the FRI (Fast Reed-Solomon Interactive Oracle Proof of Proximity) protocol. The protocol allows a Verifier to asses with high probability that a certain vector of evaluations $\mathbf{c}$ is θ_0-close to the code $\mathrm{RS}(n,k)$.

5.4.1 Commit Phase (Split and Fold)

Following [Hab22, p. 5], [AMGM24, pp. 8-9] and [RIS, Sze] we explain the commit phase.

Let $\mathbb{F}_q$ be a finite field, let $n_0 = 2^r$ for some $r \in \mathbb{Z}^+$ such that there exists a primitive n_0-th root of unity $\omega_{n_0} \in \mathbb{F}_q$ and let $D_0 = \omega_{q-1}\langle\omega_{n_0}\rangle$ be the extended domain.

Let $\mathbf{c}_0 \in \mathbb{F}_q^{n_0}$ and let $m_0(X) \in \mathbb{F}_q[X]$ be the degree $\deg m_0 < k_0 \leq n_0$ polynomial with coefficients $\mathbf{m_0} = G^{-1}V_{\omega_{n_0}}^{-1}\mathbf{c_0}$.

The protocol is recursive and uses a divide et impera approach similar to the Radix-2 FFT (hence the term Fast).

At each step i, the Prover splits the polynomial $m_{i-1}(X)$ of degree $\deg m_{i-1} < k_{i-1}$ in two polynomials

$$m_{i-1}(X) = f_{0,i-1}(X^2) + X \cdot f_{1,i-1}(X^2)$$

where $\deg f_{b,i-1} \leq \left\lfloor \frac{k_{i-1}}{2} \right\rfloor$ for $b \in \{0,1\}$.

Then the Verifier sends a random λ_i drawn uniformly from $\mathbb{F}_q$ and the Prover uses that value to compute the polynomial

$$m_i(Y) = f_{0,i-1}(Y) + \lambda_i \cdot f_{1,i-1}(Y)$$

that will have $\deg m_i < \left\lceil \frac{k_{i-1}}{2} \right\rceil = k_i$.

Finally the Prover computes the evaluations of $m_i(Y)$ over the extended domain [Sze]

$$D_i = D_{i-1}^2 = \left\{ (\omega_{q-1}\omega_{n_0}^j)^{2^i} \mid j \in \{0, \ldots, \frac{n_0}{2^i} - 1\} \right\}$$

obtaining a codeword $\mathbf{c_i} = V_{\omega_{n_0}^{2^i}} G_{\frac{n_0}{2^i}} \mathbf{m_i}$ of lenght $n_i = \frac{n_0}{2^i}$ and uses it to commit to the polynomial m_i, note that we have $\mathbf{c_i} \in \mathrm{RS}(n_i, k_i)$.

Iterating for $\log_2 n_0 = r$ rounds we obtain the constant polynomial m_r.

5.4.2 Query Phase

Following [Sze],[RIS],[AMGM24, pp. 8-9],[BBHR18a, pp. 10-11] and [Hab22, p. 7] we explain the query phase.

After the Prover has committed all the polynomials $m_0, \dots, m_r$, the Verifier verifies that each folding step was done correctly.

To reach that goal we use the relation

$$m_{i-1}(X) = f_{0,i-1}(X^2) + X \cdot f_{1,i-1}(X^2)$$

that gives the system of equations

$$\begin{cases} m_{i-1}(X) = f_{0,i-1}(X^2) + X \cdot f_{1,i-1}(X^2) \\ m_{i-1}(-X) = f_{0,i-1}(X^2) - X \cdot f_{1,i-1}(X^2) \end{cases}.$$

The Verifier then sends a random η uniformly drawn from D and, because $\eta^{2^i} \in D_i$, he computes the sequence of challenges $\eta_i = \eta^{2^i}$.

Then he asks the Prover for the evaluations $v_{i-1,+} = m_{i-1}(\eta_{i-1})$, $v_{i-1,-} = m_{i-1}(-\eta_{i-1})$ and $v_i = m_i(\eta_i)$ for each $i \in \{1, \dots, r\}$ and the Prover responds sending them and their Merkle Paths.

We note that the evaluation $m_{i-1}(-\eta_{i-1})$ is well-defined because $-\eta_{i-1} \in D_{i-1}$ as we show in the following remark.

Remark 5.4.1 ($x \in D_i \implies -x \in D_i$)
If $x \in D_i$ then $x = (\omega_{q-1}\omega_{n_0}^j)^{2^i}$ for some i, j and so $-x = -(\omega_{q-1}\omega_{n_0}^j)^{2^i}$. For the generator ω_{n_0} we have $\omega_{n_0}^{\frac{n_0}{2}} = -1$ because, using the fact that ω_{n_0} has order n_0, we have $\left(\omega_{n_0}^{\frac{n_0}{2}}\right)^2 = 1$ and $\omega_{n_0}^{\frac{n_0}{2}} \neq 1$. Then

$$-x = \omega_{n_0}^{\frac{n_0}{2}}(\omega_{q-1}\omega_{n_0}^j)^{2^i} = (\omega_{q-1}\omega_{n_0}^{j+\frac{n_0}{2^{i+1}}})^{2^i} = (\omega_{q-1}\omega_{n_0}^{\bar{j}})^{2^i} \in D$$

where we used $\bar{j} = j + \frac{n_0}{2^{i+1}}$.

Now the Verifier, utilizing the received values and starting from η_0, solves the system

$$\begin{cases} v_{0,+} = X_{0,0} + \eta_0 \cdot X_{0,1} \\ v_{0,-} = X_{0,0} - \eta_0 \cdot X_{0,1} \end{cases}$$

obtaining $X_{0,0} = \dfrac{v_{0,+} + v_{0,-}}{2}$ and $X_{0,1} = \dfrac{v_{0,+} - v_{0,-}}{2\eta_0}$.

Now given that the folding relation is

$$m_i(X^2) = f_{0,i-1}(X^2) + \lambda_i \cdot f_{1,i-1}(X^2)$$

the Verifier checks that the received value v_i is consistent with the values of the previous round by checking if

$$v_1 = X_{0,0} + \lambda_1 \cdot X_{0,1}$$

holds. If the identity checks then we iterate the procedure until the constant v_r is reached.

If $v_r = m_r$ then the phase concludes with a success.

To improve soundness the Verifier can repeat the phase for $s \in \mathbb{Z}^+$ queries and accept only if the test passed for all the s queries.

We will denote by $\text{FRI} : \mathbb{F}_q^{n_0} \times \mathbb{F}_q^r \times D_0^s \to \{0, 1\}$ the function that models the verification algorithm and that outputs 1 if the test passed and 0 otherwise.

5.4.3 Soundness end Efficiency Analysis

The security of the FRI protocol is based on the guarantee that the property of being far from the Reed-Solomon code is conserved between the folding rounds. The proof is based on the Correlated Agreement Theorem [BCI+23, p. 13] but, before stating the Theorem in its standard form, we present a naive case that uses stricter parameters and that can be proved without decoding algorithms or algebraic geometry tools.

This version is **not** powerful enough to be used to prove FRI soundness but we show it with the intent of explaining why such a result should hold.

Theorem 5.4.2 (Correlated Agreement for Lines, Naive)
Let $\mathrm{RS}(n,k)$ *be a Reed-Solomon code built using an evaluation doman D such that* $|D| = n$ *and let* $\theta \leq \frac{1-\rho}{3}$*. Let* $\mathbf{u_0}, \mathbf{u_1} \in \mathbb{F}_q^n$ *be two evaluation vectors, if the following holds*

$$\Pr_{\lambda \in \mathbb{F}_q}[\delta_H(\mathbf{u_0} + \lambda \mathbf{u_1}, \mathrm{RS}(n,k)) \leq \theta] > \frac{n}{q}$$

then there exist two codewords $\mathbf{v_0}, \mathbf{v_1} \in \mathrm{RS}(n,k)$ *such that*

$$|\{i \mid u_{0,i} \neq v_{0,i}\} \cup \{i \mid u_{1,i} \neq v_{1,i}\}| \leq (n+1)\theta$$

Proof. We begin by defining the set of λ for which the event verifies

$$\Lambda = \{\lambda \in \mathbb{F}_q \mid \delta_H(\mathbf{u_0} + \lambda \mathbf{u_1}, \mathrm{RS}(n,k)) \leq \theta\}$$

by hypotesis, computing the probability as favorable outcomes over total outcomes, we have that $\frac{|\Lambda|}{q} > \frac{n}{q}$ and so $|\Lambda| > n$.

Moreover $\theta \leq \frac{1-\rho}{3}$ implies that $\theta < \theta_0 = \frac{1-\rho}{2}$ and being inside the unique decoding radius of $\mathrm{RS}(n,k)$ for each $\lambda \in \Lambda$ there exists a unique $\mathbf{v}_\lambda \in \mathrm{RS}(n,k)$ and then a unique $m_\lambda(X) \in R_k[\mathbb{F}_q]$ such that

$$\delta_H(\mathbf{u_0} + \lambda \mathbf{u_1}, \mathbf{v}_\lambda) \leq \theta$$

that implies

$$d_H(\mathbf{u_0} + \lambda \mathbf{u_1}, \mathbf{v}_\lambda) \leq n\theta.$$

We will prove that all the polynomials $m_\lambda(X)$ are on the same line and will use that fact to obtain the conclusion.

- **Co-linearity:** We prove that, indipendently of the choice of $\lambda \in \Lambda$, there are two polynomials $m_0(X), m_1(X) \in R_k[\mathbb{F}_q]$ such that

  $$m_\lambda(X) = m_0(X) + \lambda m_1(X)$$

  and so all the polynomials $m_\lambda(X)$ are on the same line.

  We take distinct elements $\lambda_1, \lambda_2 \in \Lambda$ and compute the slope and the constant term

  $$\begin{aligned} m_1(X) &= \frac{m_{\lambda_1}(X) - m_{\lambda_2}(X)}{\lambda_1 - \lambda_2} \\ m_0(X) &= m_{\lambda_1}(X) - \lambda_1 m_1(X) \end{aligned}.$$

  Now it suffices to show that once a $\lambda_3 \in \Lambda$ is chosen we have

  $$m_{\lambda_3}(X) = m_0(X) + \lambda_3 m_1(X).$$

If we denote by $\mathbf{v}_{\lambda_3}$ the codeword of $m_{\lambda_3}(X)$ and with $\mathbf{v}^*_{\lambda_3}$ the codeword of $m_0(X) + \lambda_3 m_1(X)$, we would want to estimate the proximity of the two codewords using the triangular inequality. To make use of the set Λ characterization we need then an intermediate value, if we take in particular

$$\mathbf{u}_{\lambda_3} = \mathbf{u_0} + \lambda_3 \mathbf{u_1}$$

it can be written as a linear combination of $\mathbf{u}_{\lambda_1} = \mathbf{u_0} + \lambda_1 \mathbf{u_1}$ and $\mathbf{u}_{\lambda_2} = \mathbf{u_0} + \lambda_2 \mathbf{u_1}$ therefore we have

$$\mathbf{u}_{\lambda_3} = A\mathbf{u}_{\lambda_1} + B\mathbf{u}_{\lambda 2}$$

with $A + B = 1$ and making use of the same coefficients we obtain the codeword

$$\mathbf{v}^*_{\lambda_3} = \mathbf{v_0} + \lambda_3 \mathbf{v_1}$$

as

$$\mathbf{v}^*_{\lambda_3} = A\mathbf{v}_{\lambda_1} + B\mathbf{v}_{\lambda 2}.$$

We can then use the triangular inequality

$$d_H(\mathbf{v}_{\lambda_3}, \mathbf{v}^*_{\lambda_3}) \leq d_H(\mathbf{v}_{\lambda_3}, \mathbf{u}_{\lambda_3}) + d_H(\mathbf{u}_{\lambda_3}, \mathbf{v}^*_{\lambda_3})$$

where

$$d_H(\mathbf{v}_{\lambda_3}, \mathbf{u}_{\lambda_3}) \leq n\theta$$

while

$$\begin{aligned}
d_H(\mathbf{u}_{\lambda_3}, \mathbf{v}^*_{\lambda_3}) &= d_H(A\mathbf{u}_{\lambda_1} + B\mathbf{u}_{\lambda_2}, A\mathbf{v}_{\lambda_1} + B\mathbf{v}_{\lambda_2}) \\
&= d_H(A(\mathbf{u}_{\lambda_1} - \mathbf{v}_{\lambda_1}) + B(\mathbf{u}_{\lambda_2} - \mathbf{v}_{\lambda_2}), \mathbf{0}) \\
&= d_H(A(\mathbf{u}_{\lambda_1} - \mathbf{v}_{\lambda_1}), -B(\mathbf{u}_{\lambda_2} - \mathbf{v}_{\lambda_2})) \\
&\leq d_H(A(\mathbf{u}_{\lambda_1} - \mathbf{v}_{\lambda_1}), \mathbf{0}) + d_H(-B(\mathbf{u}_{\lambda_2} - \mathbf{v}_{\lambda_2}), \mathbf{0}) \\
&= d_H(\mathbf{u}_{\lambda_1}, \mathbf{v}_{\lambda_1}) + d_H(\mathbf{u}_{\lambda_2}, \mathbf{v}_{\lambda_2}) \\
&\leq n\theta + n\theta = 2n\theta.
\end{aligned}$$

Then using the hypotesis on θ and the unique decoding radius definition we have

$$d_H(\mathbf{v}_{\lambda_3}, \mathbf{v}^*_{\lambda_3}) \leq 3n\theta < 2n\frac{(1-\rho)}{2} = 2n\theta_0 = 2n\frac{\left\lfloor \frac{d_H(\mathrm{RS}(n,k))-1}{2} \right\rfloor}{n} < d_H(\mathrm{RS}(n,k))$$

that means that the two codewords must be the same one.

- **Cardinality:** Given that, indipendently of the choice of $\lambda \in \Lambda$, there are two polynomials $m_0(X), m_1(X) \in R_k[\mathbb{F}_q]$ such that

$$m_\lambda(X) = m_0(X) + \lambda m_1(X)$$

then using the evaluations

$$\mathbf{v}_\lambda = \mathbf{v_0} + \lambda \mathbf{v_1}$$

we have

$$d_H(\mathbf{u_0} + \lambda \mathbf{u_1}, \mathbf{v_0} + \lambda \mathbf{v_1}) \leq n\theta$$

and then

$$d_H((\mathbf{u_0} + \lambda \mathbf{u_1}) - (\mathbf{v_0} + \lambda \mathbf{v_1}), \mathbf{0}) \leq n\theta.$$

So we are interested in the cardinality $|A|$ of the set of disagreements

$$|A| = |\{i \mid u_{0,i} \neq v_{0,i}\} \cup \{i \mid u_{1,i} \neq v_{1,i}\}| = |\{i \mid (u_{0,i}, u_{1,i}) \neq (v_{0,i}, v_{1,i})\}|.$$

To find an estimate we define $E_{i,\lambda} = \{(i, \lambda) \mid (u_{0,i} - v_{0,i}) + \lambda(u_{1,i} - v_{1,i}) \neq 0\}$ then on one side we have

$$\begin{aligned} |E_{i,\lambda}| &= \sum_{i=1}^{n} |\{\lambda \in \Lambda \mid (u_{0,i} - v_{0,i}) + \lambda(u_{1,i} - v_{1,i}) \neq 0\}| \\ &\geq \sum_{i \in A} (|\Lambda| - 1) \geq |A| \cdot (|\Lambda| - 1) \end{aligned}$$

that's because if $i \in A$ the polynomial $(u_{0,i} - v_{0,i}) + \lambda(u_{1,i} - v_{1,i})$ does not vanish except for

$$\lambda = -\frac{u_{0,i} - v_{0,i}}{u_{1,i} - v_{1,i}}.$$

On the other side we have $d_H((\mathbf{u_0} + \lambda \mathbf{u_1}) - (\mathbf{v_0} + \lambda \mathbf{v_1}), \mathbf{0}) \leq n\theta$ and then we have a maximum of $n\theta$ errors that implies

$$|E_{i,\lambda}| \leq |\Lambda| n\theta.$$

Using the two inequalities we get

$$|A| \cdot (|\Lambda| - 1) \leq |E_{i,\lambda}| \leq |\Lambda| n\theta$$

that implies

$$|A| \leq \frac{|\Lambda|}{|\Lambda| - 1} n\theta = \left(1 + \frac{1}{|\Lambda| - 1}\right) n\theta \leq \left(1 + \frac{1}{n}\right) n\theta = (n+1)\theta.$$

□

We now state the complete result for the unique decoding regime found in [BCI+23, p. 14].

Theorem 5.4.3 (Correlated Agreement for Lines, Unique Decoding Regime)
Let $\mathrm{RS}(n, k)$ *be a Reed-Solomon code built using the evaluation domain* D *such that* $|D| = n$ *and let* $\theta \leq \theta_0 = \frac{1-\rho}{2}$. *Let* $\mathbf{u_0}, \mathbf{u_1} \in \mathbb{F}_q^n$ *be two evaluation vectors, if the following holds*

$$\Pr_{\lambda \in \mathbb{F}_q} [\delta_H(\mathbf{u_0} + \lambda \mathbf{u_1}, \mathrm{RS}(n, k)) \leq \theta] > \frac{n}{q}$$

then there exist two codewords $\mathbf{v_0}, \mathbf{v_1} \in \mathrm{RS}(n, k)$ *such that*

$$|\{i \mid u_{0,i} \neq v_{0,i}\} \cup \{i \mid u_{1,i} \neq v_{1,i}\}| \leq n\theta$$

We have that being

$$|\{i \mid u_{0,i} \neq v_{0,i}\} \cup \{i \mid u_{1,i} \neq v_{1,i}\}| \geq |\{i \mid u_{0,i} \neq v_{0,i}\}|$$

a sufficient condition to use the contrapositive of Theorem 5.4.3 is that at least one between $\mathbf{u_0}$ and $\mathbf{u_1}$ is θ-far from the code.

Indeed, if for example $\mathbf{u_0}$ is θ-far, its distance from the code is $d_H(\mathbf{u_0}, \mathrm{RS}(n, k)) > n\theta$. On the other side, if by contradiction we assume that the conclusion of the theorem is true, that would imply the existence of a codeword $\mathbf{v_0} \in \mathrm{RS}(n, k)$ such that

$$n\theta \geq |\{i \mid u_{0,i} \neq v_{0,i}\} \cup \{i \mid u_{1,i} \neq v_{1,i}\}| \geq |\{i \mid u_{0,i} \neq v_{0,i}\}| = d_H(\mathbf{u_0}, \mathbf{v_0}) \geq d_H(\mathbf{u_0}, \mathrm{RS}(n, k)).$$

that is absurd. Then with the hypothesis that $\mathbf{u_0}$ is θ-far, the conclusion of Theorem 5.4.3 is false and we can use the contrapositive.

Theorem 5.4.4 (FRI Soundness)
Let $FRI : \mathbb{F}_q^{n_0} \times \mathbb{F}_q^r \times D_0^s \to \{0,1\}$ *the function that models the verification algorithm.*
If $\mathbf{c}_0 \in \mathbb{F}_q^{n_0}$ *is* θ_0*-far from* $\mathrm{RS}(n_0, k_0)$*, then*

$$\Pr_{\substack{\boldsymbol{\lambda} \in \mathbb{F}_q^r \\ \boldsymbol{\eta} \in D_0^s}} [FRI(\mathbf{c_0}, \boldsymbol{\lambda}, \boldsymbol{\eta}) = 1] \leq \epsilon_{FRI} = \epsilon_C + \epsilon_Q$$

where $\epsilon_Q := (1-\theta_0)^s$ *is the soundness error of the query phase and* $\epsilon_C := \dfrac{n_0}{q}$ *is the soundness error of the commit phase.*

Proof. We denote by A the event of the dishonest Prover passing the FRI protocol and with B the event of the dishonest Prover passing the commit phase, we have

$$\Pr[A] = \Pr[A|B]\Pr[B] + \Pr[A|B^c]\Pr[B^c] \leq \Pr[B] + \Pr[A|B^c]\Pr[B^c]$$

We analyze the two cases:

- **Commit Phase:** Given that $\mathbf{c}_0 \in \mathbb{F}_q^{n_0}$ is θ_0-far from $\mathrm{RS}(n_0, k_0)$ using the contrapositive of Theorem 5.4.3, we have that if $\mathbf{u_{0,0}}, \mathbf{u_{1,0}} \in \mathbb{F}_q^{n_0}$ are the vectors of evaluations of the functions $f_{0,0}$ and $f_{1,0}$ over D_0 we have

  $$\Pr_{\lambda_1 \in \mathbb{F}_q} [\delta_H(\mathbf{u_{0,0}} + \lambda_1 \mathbf{u_{1,0}}, \mathrm{RS}(n_0, k_0)) \leq \theta_1] \leq \frac{n_0}{q}$$

  therefore $\mathbf{c_1}$ will be θ_1-far with probability of error $\dfrac{n_0}{q}$, iterating and using subadditivity

  $$\Pr_{\boldsymbol{\lambda} \in \mathbb{F}_q^r} \left[\bigcup_{i=1}^{r} \{\delta_H(\mathbf{u_{0,i-1}} + \lambda_i \mathbf{u_{1,i-1}}, \mathrm{RS}(n_i, k_i)) \leq \theta_i\} \right] \leq \sum_{i=1}^{r} \frac{n_i}{q} = \frac{1}{q} \sum_{i=1}^{r} \frac{n_0}{2^i} < \frac{n_0}{q} \sum_{i=1}^{\infty} \left(\frac{1}{2}\right)^i = \frac{n_0}{q}$$

  that is $P[B] < \epsilon_C$.

- **Query Phase:** Suppose that B^c happened, we compute the probability to pass s indipendent consistency checks. Given that

  $$\delta_H(\mathbf{u_{0,i-1}} + \lambda_1 \mathbf{u_{1,i-1}}, \mathrm{RS}(n_i, k_i)) > \theta_i$$

  and in particular

  $$d_H(\mathbf{u_{0,i-1}} + \lambda_1 \mathbf{u_{1,i-1}}, \mathrm{RS}(n_i, k_i)) > n_i \theta_i$$

  then for the candidate evaluation we have at most $n_i - n_i\theta_i$ elements that are equal to the ones of the correct vector. So for each $i \in \{1, \ldots, r\}$ the probability to pass the check is then given by $\dfrac{n_i - n_i\theta_i}{n_i} = (1-\theta_i)$. The probability of the event A_k to pass a single query is then bounded by the probability of passing the first check of the query

  $$\Pr[A_k] \leq (1-\theta_0).$$

  Therefore given that all the queries are indipendent we have

  $$\Pr[A \mid B^c] = \prod_{k=1}^{s} \Pr[A_k] \leq (1-\theta_0)^s.$$

Finally we have

$$\Pr[A] \leq \Pr[B] + \Pr[A|B^c]\Pr[B^c] \leq \epsilon_C + (1-\epsilon_C)\epsilon_Q \leq \epsilon_C + \epsilon_Q = \frac{n_0}{q} + (1-\theta_0)^s.$$

□

At this point, to improve efficiency, once an error bound is chosen $\epsilon_Q = 2^{-b}$ for $b \in \mathbb{Z}^+$ bit, we need to minimize the number of queries s, we can find $s(\theta)$ because

$$2^{-b} = (1-\theta)^s$$

so using the base 2 logarithm and solving for s we get

$$s(\theta) = \frac{b}{|\log_2(1-\theta)|}$$

to minimize s we then have to maximize θ.

The Theorem 5.4.3 does not hold for $\theta > \theta_0$, so we take the maximum value accepted

$$\theta = \theta_0 = \frac{1-\rho}{2}$$

and, given that $\rho = \frac{1}{\beta}$, to have large values for θ we need to choose an high blowup factor β.

The results we explained can be used until the limit $\theta = \lim_{\rho\to 0} \frac{1-\rho}{2} = \frac{1}{2}$.

Remark 5.4.5 (One Million Dollar Question)
Recent developments in the field have proven that this limit can be broken.

More advanced versions of the Correlated Agreement Theorem, based on list decoding techniques, extend the test validity to the Johnson bound $\theta < 1 - \sqrt{\rho}$ [BCI+23, p. 4]. For little ρ values, this limit is much higher than $\frac{1}{2}$ and allows to reduce significantly the number of queries s. The soundness estimates of FRI in real-world systems use this more advanced regime [Hab22, p. 7-8].

There is also an open conjecture that states that the Correlated Agreement Theorem validity can be extended to use $\theta < 1 - \rho$ [BCI+23, p. 42] and for proving (or disproving) that conjecture there is a prize of $\$1,000,000$ granted directly by the Ethereum Foundation [Eth25].

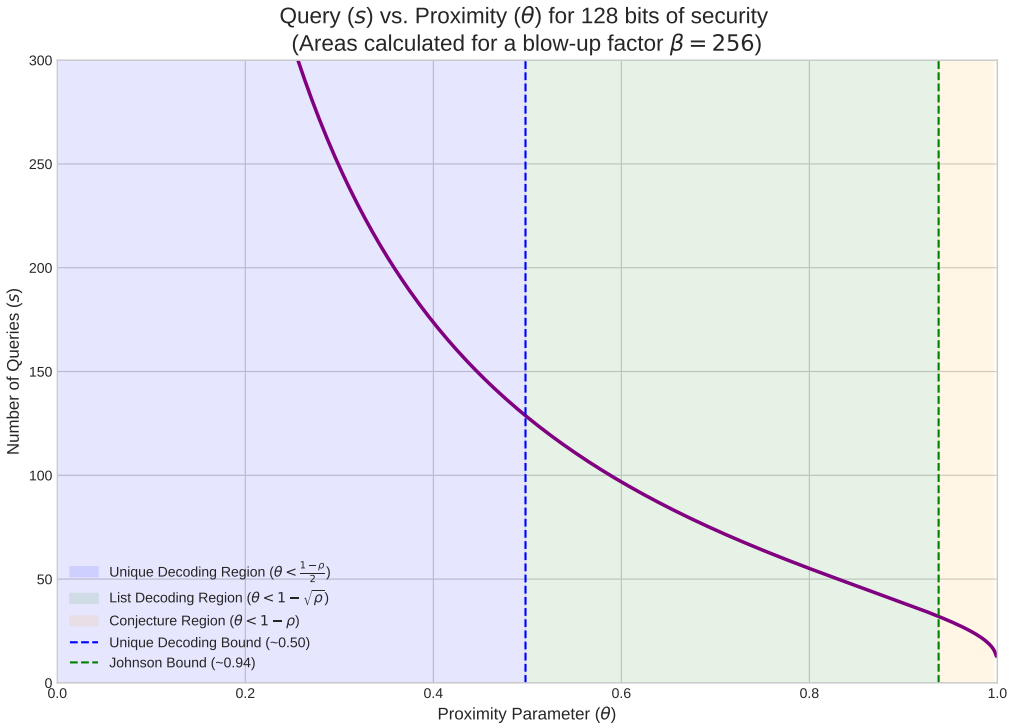
We then have that the Verifier faces a computational time complexity that is equal to the number of queries s multiplied by the cost of a single query. In each of the s queries, the Verifier needs to check r Merkle Path commitments and that has a cost of

$$O\left(\sum_{i=0}^{r-1} \log_2 n_i\right) = O\left(\sum_{i=0}^{\log_2 n_0 - 1} \log_2 n_0 - i\right) = O(\log_2^2 T)$$

So total time complexity for the Verifier is $O(s \log_2^2 T)$ and considering s as a constant once a value for θ is fixed we get $O(s \log_2^2 T) = O(\log_2^2 T)$.

We can now compute the Prover's time complexity. In the commit phase each of the r rounds must compute the evaluation $\mathbf{c_i} = V_{\omega_{n_0}^{2^i}} G_{\frac{n_0}{2^i}} \mathbf{m_i}$ that is multiplying by a diagonal $n_i \times n_i$ matrix and a DFT of the same dimension, so we get complexity

$$O\left(\sum_{i=0}^{r}(n_i \log_2 n_i + n_i)\right) = \sum_{i=0}^{r} O\left(n_i \log_2 n_i\right) = \sum_{i=0}^{r} O\left(\frac{n_0}{2^i} \log_2 \frac{n_0}{2^i}\right) = O(n_0 \log_2 n_0) = O(T \log_2 T).$$



In the query phase the Prover must compute the Merkle Paths of r evaluations so

$$O\left(\sum_{i=0}^{r-1}\log_2 n_i\right) = O\left(\sum_{i=0}^{\log_2 n_0 - 1}\log_2 n_0 - i\right) = O(\log_2^2 T)$$

and then this part isn't relevant to the total time complexty.

As seen in Remark 4.4.3 we obtained a model with $w + w_{perm} + w_{tot} + w_{tot,\mathcal{B}} + r$ private trace polynomials and then if we define $\bar{w} = w + w_{perm} + w_{tot} + w_{tot,\mathcal{B}}$ the total number of quotients to test is $2 \cdot (\bar{w} + r) + S$ and so the total computational complexity will be

$$O\left((2 \cdot (\bar{w} + r) + S)T \log_2 T\right) = O(T \log_2 T)$$

that is in the same asymptotic class.

Even if testing the quotients indipendently does not change the time complexity asymptotic class, this approach significantly worsens the soundness because for the total soundness we have $\epsilon_{FRI,tot} = (2 \cdot (\bar{w} + r) + S) \cdot \epsilon_{FRI}$ and that would require more FRI queries to be compensated.

This linear dependence from the trace width can be avoided using the Batched FRI approach.

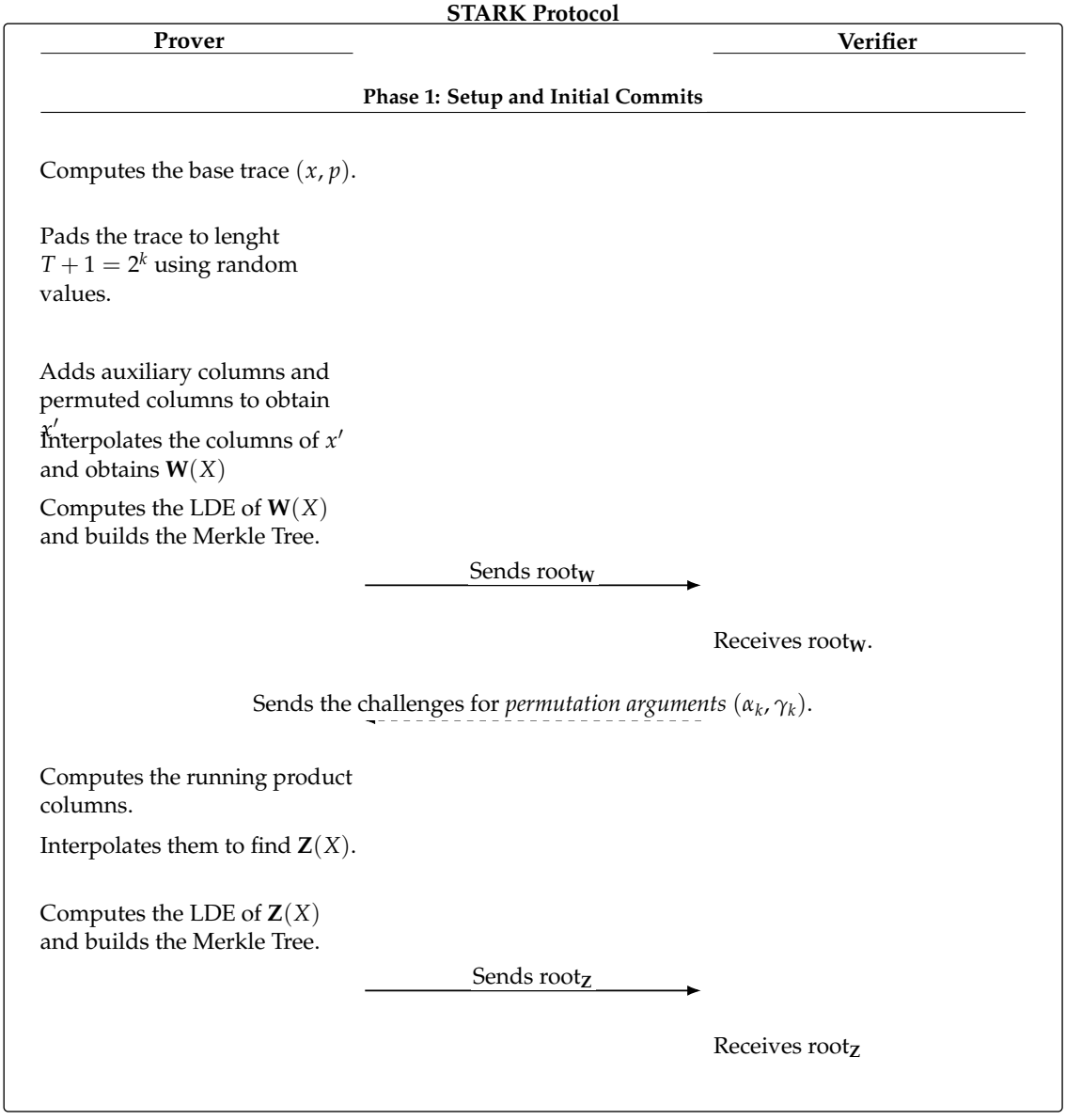
Remark 5.4.6 (Batched FRI)
All modern implementations [Hab22, p. 7], [AMGM24, p. 10] use a FRI version that does a random linear combination of all the polynomials to test and is called Batched FRI*, this way the asymptotic class remains the same but the time complexity is drastically reduced.*

In this version, the soundness analysis is much harder so we treated only the classic FRI version applied to the individual polynomials and not the batched variant.

Chapter 6

Summary, Complexity Estimates and Soundness

6.0.1 Setup and Initial Commits



Time Complexity for the Prover

We analyze the single steps:

- Let w and $T_{base}+1$ be the dimensions of the base trace and we denote by C_{max} the computational cost of executing the most costly instruction in the ISA. Generating the trace (x, p) has complexity

  $$O(C_{max}(T_{base}+1)) = O(T_{base}).$$

  Computing p requires a reordering and then has complexity $O(T_{base}\log_2 T_{base})$. Adding the padding has complexity $O(T(w+r)) = O(T)$.

- If we denote by $C_{op,max}$ the maximum number of elementary operations needed to unroll the constraints and by c_{max} the maximum number of constraints to verify in each row we get

  $$O(c_{max}\cdot C_{op,max}\cdot T_{base}) = O(T_{base}) = O(T).$$

  If we denote by w_{perm} the total number of columns in permuted order we get that the complexity is given by the copy in the order given by the permutations involved so we have $O(w_{perm}\cdot T) = O(T)$.

- The IFFT Radix-2 has complexity $O(T\log_2 T)$.

- The LDE has complexity $O(T\log_2 T)$ because it is done using an FFT and multiplying for a diagonal matrix.

- Computing the Merkle Tree is computing $2(T+1)-1$ hashes. Each hash has polynomial time complexity in the input length and that is a constant, so we get, choosing for example SHA256, a total of $O(T)$ for computing the Merkle Tree.

- Sending the root has complexity $O(1)$.

- Computing the running product involves doing two random linear combinations on a constant number of elements, a division and a multiplication for each row, so we have $O(T)$.

- The IFFT Radix-2 has complexity $O(T\log_2 T)$.

- Computing the Merkle Tree has complexity $O(T)$.

The total time complexity for the Prover is then $O(T\log T)$.

Time Complexity for the Verifier

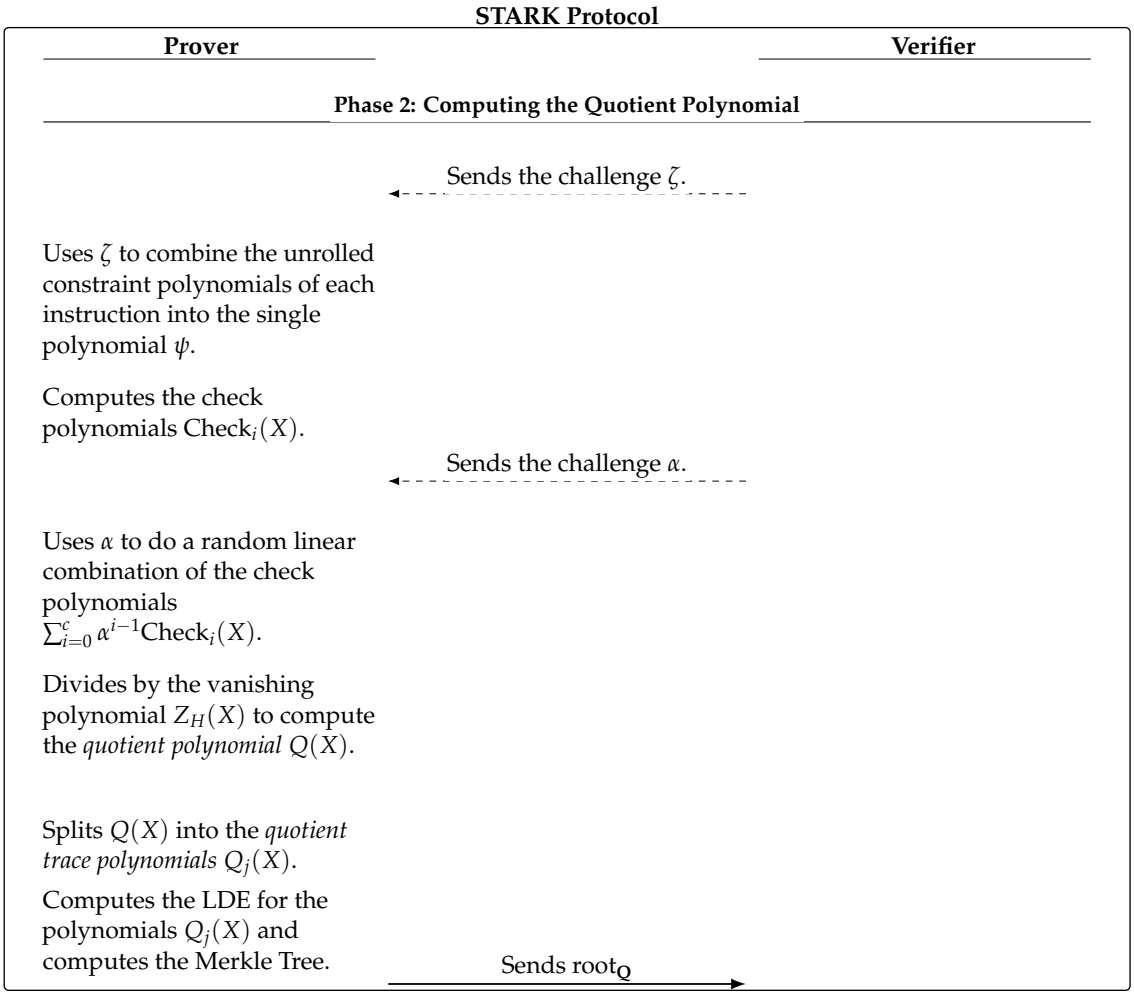
In this phase he only receives Merkle roots so we have $O(1)$.

Soundness

As seen in Proposition 4.2.8 the soundness error is $\epsilon_{ARG} = \sum_{k=1}^{r}\epsilon_k$ where

$$\epsilon_k = \begin{cases} \dfrac{T}{|\mathbb{F}|} & \text{if } v_k = 1 \\ \dfrac{(T+1)(v_{\pi_k}-1)}{|\mathbb{F}|} & \text{if } v_k \geq 2 \end{cases}.$$

6.0.2 Computing the Quotient Polynomial



Time Complexity for the Prover

We analyze the single steps:

- The linear combination is on a constant number of instructions and then we have $O(1)$.
- Computing the random linear combination for the numerator involves a constant number of polynomial of polynomials with degree at most S with polynomials of degree at most T and a random linear combination of a constant number of terms so we have $O(T)$.
- The division can be done using the Newton Iteration and using the FFT to perform multiplications inside the method giving a time complexity of $O(T \log T)$, see for example [vzGG99, p. 247].
- The computation of the trace quotient polynomials is done then by grouping the coefficients of the quotient polynomial and this can be performed by going through the coefficients only once, that gives $O(T)$.

- The LDE has complexity $O(T \log T)$.

Total time complexity is then $O(T \log T)$.

Time Complexity for the Verifier

The Verifier only receives or sends values so the complexity is $O(1)$.

Soundness

As seen in Section 4.4.1 for the aggregation of the constraint polynomials the soundness error is

$$\epsilon_{ISA} = \frac{\sum_{i=1}^{N}(c_{tot,i} - 1)}{|\mathbb{F}|}$$

while, as seen in Section 5.3.3, we have $\epsilon_{Quotient} = \ell \left(\frac{c-1}{|\mathbb{F}|} \right)$

6.0.3 DEEP Consistency Check and Preparing for FRI

STARK Protocol

Prover		**Verifier**
	Phase 3: DEEP Consistency Check	
		Receives $\text{root}_{\mathbf{Q}}$.
	Sends the challenge $z \in \mathbb{F} \setminus (D \cup H)$.	
Computes the evaluations $(\mathbf{W}(z), \mathbf{Z}(z))$. Computes the evaluations $Q_j(z^S)$.		
	Sends $\mathbf{W}(z), \mathbf{W}(\sigma(z)), \mathbf{Z}(z), \mathbf{Z}(\sigma(z)), \mathbf{Q}(z^S)$.	
	Sends the Merkle Paths.	
		Verifies the validity of Merkle Paths. Verifies that the identity $\sum_{j=1}^{S} z^{j-1} Q_j(z^S) = \sum_{i=1}^{c} \alpha^{i-1} \frac{Check_{C_i}(z)}{Z_H(z)}$ checks.
		If the check fails, **Rejects**.
	Phase 4: Preparing for FRI	
	For each j_1, k_1, j	
Computes the quotients $F_{j_1,1}(X) = \frac{W_{j_1}(X) - W_{j_1}(z)}{X - z}$ and $F_{j_1,2}(X) = \frac{W_{j_1}(\sigma(X)) - W_{j_1}(\sigma(z))}{X - \sigma(z)}$ Computes the quotients $F_{k_1,1}(X) = \frac{Z_{k_1}(X) - Z_{k_1}(z)}{X - z}$ and $F_{k_1,2}(X) = \frac{Z_{k_1}(\sigma(X)) - Z_{k_1}(\sigma(z))}{X - \sigma(z)}$ Computes the quotients $F_j(X) = \frac{Q_j(X) - Q_j(z^S)}{X - z^S}$ For each of the quotients he computes the LDE and computes the Merkle Tree		
	Sends $\text{root}_{F_{j_1,1}}$,$\text{root}_{F_{j_2,1}}$	
	Sends $\text{root}_{F_{k_1,1}}$, $\text{root}_{F_{k_1,2}}$	
	Sends root_{F_j}	

Time Complexity for the Prover

We analyze the two phases:

- In the DEEP consistency check phase the Prover computes only a constant number of evaluations on a single point of univariate polynomials of degree at most T, the total time complexity is then $O(T)$.
- Computing the quotients involves a simple division by a linear polynomial that can be done using the method of synthetic division with complexity $O(T)$, this is done for a constant number of iterations.

  Computing the LDE has complexity $O(T \log T)$ and computing the Merkle Tree has complexity $O(T)$.

Total time complexity is then $O(T \log T)$.

Time Complexity for the Verifier

The Verifier checks the validity of the Merkle Paths with complexity $O(\log T)$ and verifies the identity

$$\sum_{j=1}^{S} z^{j-1} Q_j(z^S) = \sum_{i=1}^{c} \alpha^{i-1} \frac{Check_{C_i}(z)}{Z_H(z)}$$

using a constant number of operations.

Total complexity is then $O(\log T)$.

Soundness

As seen in Section 5.3.3 we have $\epsilon_{DEEP} = \ell^2 \dfrac{(S+1)T}{|\mathbb{F}| - |D \cup H|}$

6.0.4 FRI

STARK Protocol

Prover		**Verifier**
	For each $F_{j_1,1}, F_{j_1,2}, F_{k_1,1}, F_{k_1,2}, F_j$	
	Phase 5: FRI — Commit Phase	
	For $i = 1, \ldots, r$ **where** $r = \log_2[\beta(T+1)]$	
Splits $m_{i-1}(X)$.		
	Sends the challenge λ_i. $\longleftarrow$	
Computes $m_i(Y)$.		
Computes the LDE of $m_i(Y)$ and the Merkle Tree.		
	Sends root_{m_i}. $\longrightarrow$	
	Sends m_r. $\longrightarrow$	
	Phase 6: FRI Query Phase	
	Sends s queries $\{\eta_k\}_{k=1}^{s} \subset D$. $\longleftarrow$	
	For $k = 1, \ldots, s$	
	Sends $\{v_i = m_i(\eta_k^{2^i})\}_{i=0}^{r}$ and the Merkle paths.	
		Verifies that $v_r = m_r$.
	For $i = 1, \ldots, r$	
		Verifies the validity of Merkle Paths.
		Computes $X_{i-1,0} = \frac{v_{i-1,+}+v_{i-1,-}}{2}$ and $X_{i-1,1} = \frac{v_{i-1,+}-v_{i-1,-}}{2\eta_{i-1}}$.
		Checks that $v_i = X_{i-1,0} + \lambda_i X_{i-1,1}$
		Accepts if all the s queries are valid, **Rejects** instead.

Time Complexity for the Prover

As seen in Section 5.4.3 we have total complexity $O(T \log T)$.

Time Complexity for the Verifier

As seen in Section 5.4.3 we have total complexity $O(\log^2 T)$.

Soundness

As seen in section 5.4.3 we have soundness error

$$\epsilon_{FRI,tot} = (2(\bar{w}+r)+S) \cdot \left(\frac{\beta(T+1)}{|\mathbb{F}|} + (1-\theta_0)^s \right).$$

6.1 Total Soundness and Complexity Analysis

Time Complexity for the Prover

The Prover has total time complexity $O(T \log T)$.

Time Complexity for the Verifier

The Verifier has total time complexity $O(\log^2 T)$.

Soundness

The total soundness error is

$$\begin{aligned} \epsilon_{TOT} = \sum_{k=1}^{r} \epsilon_k + \sum_{i=1}^{N} \frac{(c_{tot,i}-1)}{|\mathbb{F}|} + \ell \left(\frac{c-1}{|\mathbb{F}|} + \ell \frac{(S+1)T}{|\mathbb{F}| - |D \cup H|} \right) + \\ + (2(\bar{w}+r)+S) \cdot \left(\frac{\beta(T+1)}{|\mathbb{F}|} + (1-\theta_0)^s \right) \end{aligned}.$$

where

$$\epsilon_k = \begin{cases} \dfrac{T}{|\mathbb{F}|} & \text{if } v_{\pi_k} = 1 \\ \dfrac{(T+1)(v_{\pi_k}-1)}{|\mathbb{F}|} & \text{if } v_{\pi_k} \geq 2 \end{cases}.$$

Bibliography

[Alo99] Noga Alon. Combinatorial nullstellensatz. *Combinatorics, Probability and Computing*, 8(1–2):7–29, 1999.

[AMGM24] Héctor Masip Ardevol, Jordi Baylina Melé, Marc Guzmán-Albiol, and Jose Luis Muñoz-Tapia. estark: extending starks with arguments. *Des. Codes Cryptogr.*, 92(11):3677–3721, 2024.

[Ant23] A.M. Antonopoulos. *Mastering Bitcoin: Programming the Open Blockchain*. O'Reilly Media, 2023.

[AS92] Sanjeev Arora and Shmuel Safra. Probabilistic checking of proofs; A new characterization of NP. In *33rd Annual Symposium on Foundations of Computer Science, Pittsburgh, Pennsylvania, USA, October 24-27, 1992*, pages 2–13. IEEE Computer Society, 1992.

[BBCM92] Roberto Bevilacqua, Dario Andrea Bini, Milvio Capovani, and Ornella Menchi. *Metodi numerici*. Collana di matematica. Testi e manuali. Zanichelli, 1992.

[BBHR17] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Fast reed-solomon interactive oracle proofs of proximity. *Electron. Colloquium Comput. Complex.*, TR17-134, 2017.

[BBHR18a] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Fast reed-solomon interactive oracle proofs of proximity. In Ioannis Chatzigiannakis, Christos Kaklamanis, Dániel Marx, and Donald Sannella, editors, *45th International Colloquium on Automata, Languages, and Programming, ICALP 2018, July 9-13, 2018, Prague, Czech Republic*, volume 107 of *LIPIcs*, pages 14:1–14:17. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2018.

[BBHR18b] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Scalable, transparent, and post-quantum secure computational integrity. *IACR Cryptol. ePrint Arch.*, page 46, 2018.

[BBHR19] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Scalable zero knowledge with no trusted setup. In Alexandra Boldyreva and Daniele Micciancio, editors, *Advances in Cryptology - CRYPTO 2019 - 39th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2019, Proceedings, Part III*, volume 11694 of *Lecture Notes in Computer Science*, pages 701–732. Springer, 2019.

[BCI+23] Eli Ben-Sasson, Dan Carmon, Yuval Ishai, Swastik Kopparty, and Shubhangi Saraf. Proximity gaps for reed-solomon codes. *J. ACM*, 70(5):31:1–31:57, 2023.

[BGt23] Jeremy Bruestle, Paul Gafni, and the RISC Zero Team. RISC Zero zkVM: Scalable, Transparent Arguments of RISC-V Integrity, 2023. Technical Documentation.

[Bin] Dario Andrea Bini. Trasformata discreta di fourier e trasformata veloce (fft). Lecture Notes for the course of Numerical Analysis, University of Pisa.

[Cap16] Stefano Serra Capizzano. Interpolazione e minimi quadrati, 2016. Lecture Notes, University of Insubria.

[CBBZ22] Binyi Chen, Benedikt Bünz, Dan Boneh, and Zhenfei Zhang. Hyperplonk: Plonk with linear-time prover and high-degree custom gates. *IACR Cryptol. ePrint Arch.*, page 1355, 2022.

[CBBZ23] Binyi Chen, Benedikt Bünz, Dan Boneh, and Zhenfei Zhang. Hyperplonk: Plonk with linear-time prover and high-degree custom gates. In Carmit Hazay and Martijn Stam, editors, *Advances in Cryptology - EUROCRYPT 2023 - 42nd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Lyon, France, April 23-27, 2023, Proceedings, Part II*, volume 14005 of *Lecture Notes in Computer Science*, pages 499–530. Springer, 2023.

[Com15] A.M. Comparetti. *Introduzione ai sistemi dinamici. Con CD-ROM*. Pisa University Press. Pisa University Press, 2015.

[DF04] D.S. Dummit and R.M. Foote. *Abstract Algebra*. John Wiley & Sons, Incorporated, 2004.

[Eth25] Ethereum Foundation. The proximity prize, 2025. Official Prize Website.

[Fac21] Facebook (Meta Platforms, Inc.). Winterfell: Math module, 2021. GitHub repository, Source code.

[Gab] Ariel Gabizon. From airs to raps - how plonk-style arithmetization works.

[Giu06] L. Giuzzi. *Codici correttori: Un'introduzione*. UNITEXT. Springer Milan, 2006.

[Hab22] Ulrich Haböck. A summary on the FRI low degree test. *IACR Cryptol. ePrint Arch.*, page 1216, 2022.

[HLP24] Ulrich Haböck, David Levit, and Shahar Papini. Circle starks. *IACR Cryptol. ePrint Arch.*, page 278, 2024.

[KL14] Jonathan Katz and Yehuda Lindell. *Introduction to Modern Cryptography, Second Edition*. CRC Press, 2014.

[Knu98] Donald Ervin Knuth. *The art of computer programming, Volume II: Seminumerical Algorithms, 3rd Edition*. Addison-Wesley, 1998.

[LN97] R. Lidl and H. Niederreiter. *Finite Fields*. Number v. 20,pt. 1 in EBL-Schweitzer. Cambridge University Press, 1997.

[MF23] Tiago Martins and João Farinha. Study of arithmetization methods for starks. *IACR Cryptol. ePrint Arch.*, page 661, 2023.

[MR95] Rajeev Motwani and Prabhakar Raghavan. *Randomized Algorithms*. Cambridge University Press, 1995.

[MR04] David K. Maslen and Daniel N. Rockmore. The Cooley-Tukey FFT and group theory. In *Modern signal processing.*, pages 281–300. Cambridge: Cambridge University Press, 2004.

[MvOV96] Alfred Menezes, Paul C. van Oorschot, and Scott A. Vanstone. *Handbook of Applied Cryptography*. CRC Press, 1996.

[Nat12] National Institute of Standards and Technology (NIST). Nvd - cve-2012-2459, 2012. National Vulnerability Database.

[Nus82] Henri J. Nussbaumer. *Number Theoretic Transforms*, pages 211–240. Springer Berlin Heidelberg, Berlin, Heidelberg, 1982.

[Ram25] Bianca Rampazzo. Post-quantum anonymous credentials and zk-starks. Master's thesis in mathematical engineering, Politecnico di Torino, 2025.

[RIS] RISC Zero. Stark by hand. RISC Zero Developer Documentation.

[RIS25] RISC-V International. The RISC-V instruction set manual, volume I: Unprivileged architecture. Technical report, RISC-V International, 2025. Version 20250508, Ratified Standard.

[Sip12] M. Sipser. *Introduction to the Theory of Computation*. Cengage Learning, 2012.

[SSM+23] Ardianto Satriawan, Infall Syafalni, Rella Mareta, Isa Anshori, Wervyan Shalannanda, and Aleams Barra. Conceptual review on number theoretic transform and comprehensive review on its implementations. *IEEE Access*, 11:70288–70316, 2023.

[Sta] StarkWare. Starknet docs: Cryptography. Official Starknet Protocol Documentation.

[Sze] Alan Szepieniec. Stark anatomy.

[Tea21] StarkWare Team. ethstark documentation. *IACR Cryptol. ePrint Arch.*, page 582, 2021.

[Tha22] Justin Thaler. Proofs, arguments, and zero-knowledge. *Found. Trends Priv. Secur.*, 4(2-4):117–660, 2022.

[The22] The Plonky3 Authors. Plonky3, 2022. GitHub repository.

[TW20] W. Trappe and L.C. Washington. *Introduction to Cryptography: With Coding Theory*. Pearson Education, 2020.

[VL92] Charles F. Van Loan. *Computational frameworks for the fast Fourier transform*, volume 10 of *Front. Appl. Math.* Philadelphia, PA: SIAM, 1992.

[vzGG99] Joachim von zur Gathen and Jürgen Gerhard. *Modern Computer Algebra*. Cambridge University Press, 1st edition, 1999.